

键落星辰

ACM 算法竞赛模板

定稿时间:

2026年9月

队员:

许嘉昊

郭金庚

曾逸轩

Contents

1 动态规划

1.1 四边形不等式与树形背包优化 3
1.1.1 树形背包优化 3
1.2 O(n · max ai) Subset Sum 3

2 字符串

2.1 最小表示法 3
2.2 Manacher 3
2.3 KMP, exKMP 3
2.4 AC 自动机 3
2.5 Lydon Word Decomposition 3
2.6 后缀自动机 3
2.7 广义后缀自动机 3
2.8 SAMSA & 后缀树 4
2.9 后缀数组 4
2.10 Total LCS 4
2.11 回文树 4
2.12 双端插入回文树 4
2.13 Runs 4
2.14 字符串 Hash 类 5
2.15 String Conclusions 5

3 数论 (不含博弈论)

3.1 Long Long O(1) 乘, Barrett 6
3.2 exgcd, 逆元 6
3.3 CRT 中国剩余定理 6
3.4 Miller Rabin, Pollard Rho 6
3.5 扩展卢卡斯 6
3.6 阶乘取模 6
3.7 类欧几里得直线下格点统计 6
3.8 万能欧几里德 6
3.9 平方剩余 7
3.10 原根 7
3.11 min25筛 7
3.12 Pell 方程 7
3.13 解一元三次方程 7
3.14 Formulas 公式表 7
3.14.1 Mobius Inversion 7
3.14.2 杜教筛 7
3.14.3 降幂公式 7
3.14.4 其他常用公式 7
3.14.5 单位根反演 7
3.14.6 Arithmetic Function 7
3.14.7 Binomial Coefficients 8
3.14.8 Fibonacci Numbers, Lucas Numbers 8
3.14.9 Sum of Powers 8
3.14.10 Catalan Numbers 1, 1, 2, 5, 14, 42, 132, 429, 1430... 8
3.14.11 Motzkin Numbers 1, 1, 2, 4, 9, 21, 51, 127, 323, 835... 8
3.14.12 Derangement 错排数 0, 1, 2, 9, 44, 265, 1854, 14833... 8
3.14.13 Bell Numbers 1, 1, 2, 5, 15, 52, 203, 877, 4140 ... 8
3.14.14 Stirling Numbers 8
3.14.15 Eulerian Numbers 9
3.14.16 Harmonic Numbers, 1, 3/2, 11/6, 25/12, 137/60... 9
3.14.17 卡迈克尔函数 9
3.14.18 求拆分数 9
3.14.19 Bernoulli Numbers 1, 1/2, 1/6, 0, -1/30, 0, 1/42 ... 9
3.14.20 kMAX-MIN反演 9
3.14.21 伍德伯里矩阵不等式 9
3.14.22 Sum of Squares 9
3.14.23 枚举勾股数 Pythagorean Triple 9
3.14.24 四面体体积 Tetrahedron Volume 9
3.14.25 杨氏矩阵与钩子公式 9
3.14.26 常见博弈游戏 9
3.14.27 概率相关 9
3.14.28 邻接矩阵行列式的意义 9
3.14.29 Others (某些近似数值公式在这里) 9
3.15 Calculus, Integration Table 导数积分表 10
3.16 Zeller 日期公式 10
3.17 有理数二分: Stern-Brocot 树, Farey 序列 10
3.18 Constant Table 常数表 10

4 多项式与生成函数

4.1 FFT 10
4.2 NTT 11
4.3 MTT 任意模数卷积 11
4.4 多项式运算 11
4.4.1 多项式求逆 开根 ln exp 11
4.4.2 多项式除法 取模 11
4.4.3 多点求值 12
4.4.4 插值 12
4.5 线性递推 12
4.6 Berlekamp-Massey 最小多项式 12
4.7 FWT 13
4.8 K 进制 FWT 13

5 线性代数

5.1 Simplex 单纯形 13
5.2 高斯消元最小范数解 13

6 数据结构

6.1 bitset 14
6.2 FHQ treap 14
6.3 RangeBIT 14
6.4 李超线段树 14

6.5 线段树分裂 14
6.6 左偏树 14
6.7 非递归线段树 15
6.7.1 区间加, 区间求最大值 15
6.8 LCT 动态树 15
6.9 Hash Table 15
6.10 基数排序 15

7 树上问题

7.1 虚树 15
7.2 树哈希 15

8 计算几何

8.1 声明与宏 16
8.2 点与向量 16
8.3 线 16
8.4 圆 16
8.5 凸包与闵可夫斯基和 17
8.6 旋转卡壳 17
8.7 多边形基础 18
8.8 直线半平面交 18
8.9 整数半平面交 18
8.10 凸包询问: 凸包内、切点、交点、最近点 19
8.11 简单多边形内最长线段 19
8.12 O(n^2) Ear Clipping 三角剖分 20
8.13 单插入动态凸包 20
8.14 Delaunay 三角剖分 20
8.15 圆反演, 阿波罗尼亚圆 21
8.16 圆并 21
8.17 多边形与圆交 22
8.18 球面基础, 经纬度球面距离 22
8.19 最近点对 22
8.20 圆上整点 22
8.21 三角形 22
8.22 相关公式: 三角形心, 海伦公式, 欧拉定理, 内接球, 外接圆 23
8.22.1 超球坐标系 23
8.22.2 三维旋转公式 23
8.22.3 立体角公式 23
8.22.4 常用体积公式 23
8.22.5 扇形与圆弧重心 23
8.22.6 高维球体积 23
8.23 三维几何基础操作 23
8.24 三维凸包 24
8.25 最小覆盖球 24
8.26 黄金三分 24
8.27 自适应 Simpson 24

9 图论

9.1 Hopcroft-Karp O(sqrt(V)E) 25
9.2 Hungarian O(VE/w) 25
9.3 Shuffle 一般图最大匹配 O(VE) 25
9.4 极大团计数 25
9.5 KM 最大权匹配 O(V^3) 25
9.6 欧拉回路 25
9.7 Blossom 一般图最大匹配 O(|V||E|) 25
9.8 2-SAT, 强连通分量 / Bitset Kosaraju 26
9.9 Tarjan 点双, 边双 26
9.10 Dominator Tree 支配树 26
9.11 环计数 26
9.12 Dinic 最大流 26
9.13 原始对偶费用流 27
9.14 网络流总结 27
9.15 网络流总结 27
9.16 Gomory-Hu 无向图最小割树 O(V^3E) 27
9.17 Stoer-Wagner 无向图最小割 O(VE + V^2 log V) 27
9.18 弦图 28
9.19 Minimum Mean Cycle 最小平均值环 O(n^2) 28
9.20 斯坦纳树 28
9.21 图论结论 28
9.21.1 最小乘积问题原理 28
9.21.2 最小环 28
9.21.3 度序列的可图性 28
9.21.4 切比雪夫距离与曼哈顿距离转化 28
9.21.5 树链的交 29
9.21.6 带修改MST 29
9.21.7 差分约束 29
9.21.8 李超线段树 29
9.21.9 Segment Tree Beats 29
9.21.10 二分图 29
9.21.11 稳定婚姻问题 29
9.21.12 三元环 29
9.21.13 图同构 29
9.21.14 竞赛图 Landau's Theorem 29
9.21.15 Ramsey Theorem R(3,3)=6, R(4,4)=18 29
9.21.16 树的计数 Prufer序列 29
9.21.17 有根树计数 1,1,2,4,9,20,48,115,286,719,1842,4766 29
9.21.18 无根树计数 29
9.21.19 生成树计数 Kirchhoff's Matrix-Tree Theorem 29
9.21.20 有向图欧拉回路计数 BEST Theorem 29
9.21.21 Tutte Matrix 29
9.21.22 Edmonds Matrix 29
9.21.23 有向图无环定向, 色多项式 29
9.21.24 拟阵交问题 29
9.21.25 双极定向 29
9.21.26 图中的环 30

10 离线算法 (如 CDQ、莫队)

10.1	莫队二次离线	30
11	随机化与博弈论	30
11.1	Python Hint	30
11.2	Hacks: 指令集, 读入优化, Bitset, builtin, 随机种子	30
11.3	试机赛与纪律文件	30

1. 动态规划

1.1 四边形不等式与树形背包优化

```
1 //  $a \leq b \leq c \leq d: w(b, c) \leq w(a, d)$ ,
  //  $\hookrightarrow w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$ 
2 for (int len = 2; len <= n; ++len) {
3     for (int l = 1, r = len; r <= n; ++l, ++r) {
4         f[l][r] = INF;
5         for (int k = m[l][r - 1]; k <= m[l + 1][r]; ++k) {
6             if (f[l][r] > f[l][k] + f[k + 1][r] + w(l, r)) {
7                 f[l][r] = f[l][k] + f[k + 1][r] + w(l, r);
8                 m[l][r] = k;
9             } } } }
```

1.1.1 树形背包优化

限制: 必须取与根节点相连的一个连通块。

转化: 一个点的子树对应于DFS序中的一个区间。则每个点的决策为, 取该点, 或者舍弃该点对应的区间。从后往前dp, 设 $f(i, v)$ 表示从后往前考虑到 i 号点, 总体积为 V 的最优价值, 设 i 号点对应的区间为 $[i, i + \text{size}_i - 1]$, 转移为 $f(i, v) = \max\{f(i + 1, V - v_i) + w_i, f(i + \text{size}_i, v)\}$ 。

如果要求任意连通块, 则点分治后转为指定根的连通块问题即可。

1.2 $O(n \cdot \max a_i)$ Subset Sum

```
1 int SubsetSum(vector<int> &a, int t) {
2     int B = *max_element(a.begin(), a.end());
3     int n = (int) a.size(), s = 0, i = 0;
4     while (i < n && s + a[i] <= t) s += a[i++];
5     if (i == n) return s;
6     vector<int> f(2 * B + 1, -1), pre (B + 1, -1);
7     f[s - (t - B)] = i;
8     for (; i < n; i++) {
9         s += a[i];
10        for (int d = 0; d <= B; d++) pre[d] = max(0, f[d + B]);
11        for (int d = B; d >= 0; d--)
12            f[d + a[i]] = max(f[d + a[i]], f[d]);
13        for (int d = 2 * B; d > B; --d)
14            for (int j = pre[d - B]; j < f[d]; j++)
15                f[d - a[j]] = max(f[d - a[j]], j);
16        for (i = 0; i <= B; i++) if (f[B - i] >= 0) return t - i; }
```

2. 字符串

2.1 最小表示法

```
1 int minimal_string(){// 下标从0开始
2     int k = 0, i = 0, j = 1;
3     while(k < n && i < n && j < n){
4         if(a[(i+k)%n] == a[(j+k)%n]) {k++; continue;}
5         if(a[(i+k)%n] > a[(j+k)%n]) i = i+k+1;
6         else j = j+k+1;
7         i += (i == j); k = 0; }
8     return min(i, j); }
```

2.2 Manacher

```
1 void read(){ // 两倍空间
2     s[0] = '~', s[++n] = '|';
3     char c = getchar();
4     while(c < 'a' || c > 'z') c = getchar();
5     while('a' <= c && c <= 'z') s[++n] = c, s[++n] = '|', c =
        getchar();
6 }
7 void manacher(){
8     int mx = 0, mid = -1, ans = 0;
9     for(int i=1; i<=n; i++){
10        if(i <= mx) h[i] = min(h[(mid<<1)-i], mx-i+1);
11        else h[i] = 1;
12        while(s[i-h[i]] == s[i+h[i]]) ++h[i];
13        if(h[i] + i - 1 > mx) mx = h[i] + i - 1, mid = i; } }
```

2.3 KMP, exKMP

```
1 void kmp(const char *s, int n) { // 1-based
2     fail[0] = fail[1] = 0;
3     for (int i = 1; i < n; i++) { int j = fail[i];
4         while (j && s[i + 1] != s[j + 1]) j = fail[j];
5         if (s[i + 1] == s[j + 1]) fail[i + 1] = j + 1;
6         else fail[i + 1] = 0; } }
7 void exkmp(const char *s, int *a, int n) { // 1-based
8     int l = 0, r = 0; a[1] = n;
9     for (int i = 2; i <= n; i++) {
10        a[i] = i > r ? 0 : min(r - i + 1, a[i - 1 + 1]);
11        while (i+a[i] <= n && s[i+a[i]] == s[i+a[i]]) a[i]++;
12        if (i + a[i] - 1 > r) {l = i; r = i + a[i] - 1;}} }
```

2.4 AC 自动机

注意代码是以 0 为根的, 如果要 1-base 的话要改一下没有儿子时的逻辑。

```
1 struct AC{
2     int ch[N][26], fail[N], tag[N], tot;
3     void ins(string s, int _idx){
4         int x = 0;
5         for(auto c : s){
6             int v = c - 'a';
7             if(!ch[x][v]) ch[x][v] = ++tot;
8             x = ch[x][v]; tag[x] = _idx; }
9     void build(){
10        queue<int> q;
11        for(int i=0; i<26; i++) if(ch[0][i]) q.push(ch[0][i]);
12        while(!q.empty()){
13            int x = q.front(); q.pop();
14            for(int i=0; i<26; i++) {
15                if(ch[x][i]) fail[ch[x][i]] = ch[fail[x]][i],
16                    q.push(ch[x][i]);
17                else ch[x][i] = ch[fail[x]][i]; } } }
18 } ac;
```

2.5 Lyndon Word Decomposition

```
1 //满足s的最小后缀等于s本身的串称为Lyndon串。
2 //等价于: s是它自己的所有循环移位中唯一最小的一个。
3 //任意字符串s可以分解为  $s = s_1 s_2 s_k$ , 其中  $s_i$  是Lyndon串,
  //  $\hookrightarrow s_i \geq s_{i+1}$ . 且这种分解方法是唯一的。
4 //后缀排序后, 排名的所有前缀最小值构成了  $Ly$  分解的左端点。
5 void mnsuf(char *s, int *mn, int n){ // 每个前缀的最小后缀
6     //1 - base, 求 Lyndon 分解去掉 mn 即可
7     for(int i = 1; i <= n; i++)
8         for(int j = i + 1, k = i; mn[i] = i;
9             for(; j <= n && s[k] <= s[j]; j++){
10                if(s[k] < s[j]) k = mn[j] = i;
11                else mn[j] = mn[k] + j - k, k++;
12                for(; i <= k; i += j - k) {} } //lyn+=s[i..i+k-j-1]
13 void mxsuf(char *s, int *mx, int n){ // 每个前缀的最大后缀
14     fill(mx + 1, mx + n + 1, 0); // 1 - base
15     for(int i = 1; i <= n; i++){
16         int j = i + 1, k = i; !mx[i] ? mx[i] = i : 0;
17         for(; j <= n && s[k] >= s[j]; j++){
18             !mx[j] ? mx[j] = j : 0;
19             s[k] > s[j] ? k = i : k++; }
20         for(; i <= k; i += j - k) {} }
```

2.6 后缀自动机

```
1 struct SAM{ // 别忘记空间是两倍 改字符集记得全改了
2     int last = 1, tot = 1;
3     int ch[MAXN<<1][26], len[MAXN<<1], f[MAXN<<1];
4     void ins(char c){
5         c -= 'a';
6         int p = last, cur = last = ++tot;
7         len[cur] = len[p] + 1;
8         for(; p && !ch[p][c]; p=f[p]) ch[p][c] = cur;
9         if(!p) {f[cur] = 1; return;}
10        int q = ch[p][c];
11        if(len[q] == len[p] + 1) {f[cur] = q; return;}
12        int clone = ++tot;
13        for(int i=0; i<26; i++) ch[clone][i] = ch[q][i];
14        f[clone] = f[q], len[clone] = len[p] + 1;
15        f[q] = f[cur] = clone;
16        for(; p && ch[p][c] == q; p=f[p]) ch[p][c] = clone;
17    }
18 } sam;
```

2.7 广义后缀自动机

```
1 struct SAM{
2     int ch[MAXN<<1][26], f[MAXN<<1], len[MAXN<<1], tot = 1;
3     int ins(int c, int last){
4         c -= 'a';
5         int cur, p;
6         if(ch[last][c]){
7             p = last, cur = ch[p][c];
8             if(len[p] + 1 == len[cur]) return cur;
9             int q = cur, clone = ++tot;
10            for(int i=0; i<26; i++) ch[clone][i] = ch[q][i];
11            len[clone] = len[p] + 1;
12            f[clone] = f[q], f[q] = clone;
13            for(; p && ch[p][c] == q; p=f[p]) ch[p][c] = clone;
14            return clone;
15        }
16        p = last, cur = ++tot;
17        len[cur] = len[p] + 1;
```

```

7 |         if (s[i] == t[j]) {
8 |             H[i][j] = V[i][j - 1]; V[i][j] = H[i - 1][j];
9 |         } else {
10 |             H[i][j] = max(H[i-1][j], V[i][j-1]);
11 |             V[i][j] = min(H[i-1][j], V[i][j-1]); } } }
12 | for (int i = 1; i <= m; ++ i) { int ans = 0;
13 |     for (int j = 1; j <= n; ++ j) { ans += H[n][j] < i;
14 |         printf("%d%c", ans, " \n"[j == m]); } } }

```

0 的子树是长为偶数的串, 1 的子树是长为奇数的. 0 代表空串, 1 没有意义.

```

1 struct PAM { // sum[i]: 以i结尾的回文串的个数
2 |   int ch[N][26], len[N], fail[N], tot = 1, now = 1, sum[N];
3 |   void build(){
4 |       fail[0] = 1, len[1] = -1;
5 |       s[0] = '#';
6 |       for(int i=1; s[i]; i++){
7 |           int j, v = (s[i] - 'a');
8 |           while(s[i-len[now]-1] != s[i]) now = fail[now];
9 |           if(!ch[now][v]){
10 |               len[++tot] = len[now] + 2;
11 |               j = fail[now];
12 |               while(s[i-len[j]-1] != s[i]) j = fail[j];
13 |               fail[tot] = ch[j][v];
14 |               ch[now][v] = tot;
15 |               sum[tot] = sum[fail[tot]] + 1;
16 |           }
17 |           now = ch[now][v];
18 |       }
19 |   }
20 } pam; // 定理：长度为n的字符串s最多有n个本质不同回文串

```

```

1 // 之前 PAM 的板子中，通过设置位置 0 为哨兵节点，因此不需要限制
   ↳ 范围
2 // 本板子采用 while 循环中限制范围的做法来规避风险
3 struct PAM_bidirectional {
4     int ch[N][26], len[N], fail[N], tot = 1;
5     int last_front = 1, last_back = 1; // 维护前后端指针
6     char s[N * 2]; // 双向字符数组，首尾
   ↳ 留足 buffer
7     int L = N, R = N - 1; // 初始指针居中
8
9     void init() {
10         fail[0] = 1; len[1] = -1;
11         memset(ch, 0, sizeof ch);
12         memset(len, 0, sizeof len);
13         memset(fail, 0, sizeof fail);
14         tot = 1;
15         last_front = last_back = 1;
16         L = N; R = N - 1;
17         fail[0] = 1; len[1] = -1;
18
19     void push_back(char c) {
20         s[++R] = c;
21         int v = c - 'a', p = last_back;
22         while (s[R - len[p] - 1] != s[R]) p = fail[p];
23         if (!ch[p][v]) {
24             int cur = ++tot, j = fail[p];
25             len[cur] = len[p] + 2;
26             while (s[R - len[j] - 1] != s[R]) j = fail[j];
27             fail[cur] = ch[j][v];
28             ch[p][v] = cur;
29         }
30         last_back = ch[p][v];
31         if (len[last_back] == R - L + 1) last_front =
           ↳ last_back;
32     }
33     void push_front(char c) {
34         s[--L] = c;
35         int v = c - 'a', p = last_front;
36         while (s[L + len[p] + 1] != s[L]) p = fail[p];
37         if (!ch[p][v]) {
38             int cur = ++tot, j = fail[p];
39             len[cur] = len[p] + 2;
40             while (s[L + len[j] + 1] != s[L]) j = fail[j];
41             fail[cur] = ch[j][v];
42             ch[p][v] = cur;
43         }
44         last_front = ch[p][v];
45         if (len[last_front] == R - L + 1) last_back =
           ↳ last_front;

```

2.10 Total LCS

```

1 const int N = 2005; int H[N][N], V[N][N]; string s, t;
2 int main() { cin >> s >> t;
3   | int n = (int) (s.size() + 1), m = (int) (t.size() + 1); s
   |   ↳ = " " + s; t = " " + t;
4   | for (int i = 1; i <= m; ++ i) H[0][i] = i;
5   | for (int i = 1; i <= n; ++ i) {
6   |   | for (int j = 1; j <= m; ++ j) {

```

```

46     }
47 };

```

2.13 Runs

```

1 struct Runs{
2     int l,r,p;
3 };vector<Runs> run;
4 bool operator==(Runs x,Runs y){return x.l==y.l && x.r==y.r;}
5 int gl(int x,int y); // 求 S[1,x],S[1,y] 的最长公共后缀
6 int gr(int x,int y); // 求 S[x,n],S[y,n] 的最长公共前缀
7 //上面两个可以用 二分 + Hash 或者后缀数组实现。
8 bool getcmp(int x,int y){//S[x,n]<S[y,n]
9     | int len=gr(x,y);
10    | return st[x+len]<st[y+len];}
11 int ly[N];
12 void lyndon(bool type){//后缀排序法求 Lyndon
13     | stack<PII> stk;stk.push({n,n});ly[n]=n;
14     | for(int i=n-1;i>=1;i--){
15         | int now=i;
16         | while(!stk.empty() && getcmp(i,stk.top().first)!=type)
17         | | now=stk.top().second,stk.pop();
18         | ly[i]=now;
19         | stk.push({i,now});
20     | } }
21 void getrun(){
22     | for(int l=1;l<=n;l++){
23         | int r=ly[l],ll=l,rr=r;
24         | if(l!=1)ll=gl(l-1,r);
25         | if(r!=n)rr+=gr(l,r+1);
26         | if(rr-ll+1>=2*(r-l+1))run.push_back({ll,rr,r-l+1}); }
27         | }
28 void solve(){
29     | st[n+1] = '\0'; run.clear();
30     | init();//Hash 或者 SA 的启动
31     | for(int op=0;op<=1;op++){//0正常字典序,1反序
32         | lyndon(op); getrun(); }
33     | sort(run.begin(),run.end(),[](Runs x, Runs y){
34         | return x == y ? x.p < y.p : (x.l != y.l ? x.l < y.l
35         | | : x.r < y.r);});
36     | run.erase(unique(run.begin(),run.end()),run.end()); }

```

2.14 字符串 Hash 类

Random primes generated at Mon Aug 31 22:29:26 2026
1e12 991234123427 992345232803 992345233853 1023452340373
1e13 9912341233903 9945674567623 9956785678529 10345634561341
2e13 19234523456303 19234523458961 19345634561839 20123412348593
1e15 991234123419107 994567456749287 995678567857759
1e17 10123412341234073 102345234523454491 102345234523458847
1e18 994567456745678273 1045674567456746371 1056785678567850079

```

1 static constexpr u128 inv = []() {
2     | u128 ret = P;
3     | for (int i = 0; i < 6; i++) ret *= 2 - ret * P;
4     | return ret; }();
5 constexpr u128 chk = u128(-1) / P;
6 bool check(i128 a, i128 b) { // 注意: i128
7     | if (a < b) swap(a, b);
8     | return (a - b) * inv <= chk; }

```

```

1 struct StringHash {
2     int B = 131;
3     int M = 1e9 + 7;
4     int n = 0;
5     vector<int> h, p;
6     StringHash() {}
7     StringHash(int base, int mod) : B(base), M(mod) {}
8     void init(int base, int mod) {
9         B = base;
10        M = mod;
11    }
12    void build(const string& s) {
13        n = s.size();
14        h.assign(n + 1, 0);
15        p.assign(n + 1, 1);
16        for (int i = 1; i <= n; i++) {
17            p[i] = 1LL * p[i - 1] * B % M;
18            h[i] = (1LL * h[i - 1] * B + s[i - 1]) % M;
19        }
20    }
21    void build(const string& s, int base, int mod) {
22        init(base, mod);
23        build(s);
24    }

```

```

25 int get(int l, int r) const {
26     int res = (h[r] - 1LL * h[l - 1] * p[r - l + 1]) %
27         | M;
28     if (res < 0) res += M;
29     return res;
30 };

```

2.15 String Conclusions

双回文串

若字符串 s 满足 $s = x_1x_2 = y_1y_2 = z_1z_2$ (长度满足 $|x_1| < |y_1| < |z_1|$), 且 x_2, y_1, y_2, z_1 均为回文串, 则 x_1 与 z_2 也必然是回文串。

Border 和周期

若 r 是 S 的一个 Border, 则 $|S| - r$ 必然是 S 的一个周期。

弱周期引理 (Weak Periodicity Lemma): 若 p 和 q 均为 S 的周期, 且满足 $p + q \leq |S|$, 则 $\gcd(p, q)$ 也是 S 的周期。

强周期引理 (Strong Periodicity Lemma): 若 p 和 q 均为 S 的周期, 且满足 $p + q \leq |S| + \gcd(p, q)$, 则 $\gcd(p, q)$ 也是 S 的周期。

字符串匹配与 Border

当模式串与文本串满足 $2|S| \geq |T|$ 时, S 在 T 中的所有匹配位置构成一个等差数列。

若 S 在 T 中的匹配次数大于 2, 则该等差数列的公差恰好等于 S 的最小周期。

Border 的结构

字符串 S 所有长度不小于 $|S|/2$ 的 Border 构成一个单等差数列。

字符串 S 的所有 Border 按长度排序后, 最多可以拆分为 $O(\log |S|)$ 段等差数列。

回文串 Border 与 slink 优化

回文串长度为 t 的后缀是回文后缀, 当且仅当 t 是该回文串的 Border。因此, 回文后缀的长度分布同样满足拆分为 $O(\log |S|)$ 段等差数列的性质, 可利用此性质实现 $O(N \log N)$ 回文划分 DP 优化。

具体地, 引入以下两个辅助变量:

$\text{diff}[u] = \text{len}[u] - \text{len}[\text{fail}[u]]$: 表示节点 u 与它最长真回文后缀的长度差。

$\text{slink}[u]$ (Serial Link): 指向 u 的 fail 链上第一个 diff 值与 u 不同的祖先节点:

$$\text{slink}[u] = \begin{cases} \text{slink}[\text{fail}[u]], & \text{if } \text{diff}[u] == \text{diff}[\text{fail}[u]] \\ \text{fail}[u], & \text{otherwise} \end{cases}$$

同一条等差数列上的状态可以 $O(1)$ 继承。

Run 理论

三元组 (l, r, p) 称为字符串 S 的一个 Run, 当且仅当子串 $S[l..r]$ 的最小周期为 p , 且满足:

1. 周期重复: $r - l + 1 \geq 2p$ (至少重复 2 次)。
2. 极大性: 无法向左拓展 ($S[l - 1] \neq S[l - 1 + p]$) 且无法向右拓展 ($S[r + 1] \neq S[r + 1 - p]$)。

长度为 N 的字符串中, 不同的 Runs 数量 $< 1.83N$, 所有 Runs 的长度 / 周期之和 $< 3N$ 。

Run 的 $O(N)$ 求解

针对每一个位置 $i \in [1, N]$, 分别在正序 ($\leq$) 和反序 ($\geq$) 字典序下求出以 i 开头的最长 Lyndon 前缀长度 $\text{Lyn}[i]$ 。正反各产生 N 个, 共计 $2N$ 个候选周期 $p = \text{Lyn}[i]$, 提取所有 $(i, i + p)$ 候选对。

对于每一个切分点形成的相邻位置 i 与 j ($p = j - i$): 利用 ExKMP (或 SA / Hash + 二分) 求出 $L_1 = \text{LCS}(S[1..i - 1], S[1..j - 1])$, $L_2 = \text{LCP}(S[i..N], S[j..N])$ 。若扩展总长 $L_1 + L_2 \geq p$, 则构造出了一个合法 Run $[i - L_1, j + L_2 - 1]$, 其周期为 p 。最后将所有提取到的三元组 (l, r, p) 放入 vector 中, 通过 sort + unique 排序去重, 即可得到全串所有的 Runs。

子串最小后缀

设 $s[p..n]$ 是所有 $s[i..n]$ ($l \leq i \leq r$) 中字典序最小者, 则 $\text{minsub}(l, r)$ 等于 $s[p..r]$ 的最短非空 Border。

满足倍增递推式: $\text{minsub}(l, r) = \min\{s[p..r], \text{minsub}(r - 2^k + 1, r)\}$ ($2^k < r - l + 1 \leq 2^{k+1}$)。

子串最大后缀

从左往右扫描字符串, 使用 std::set 维护字典序递减的单调队列。每一次插入字符的时候, 先弹出队尾一定小于其字典序的位置, 然后在对应时刻插入“小于事件”点以供后续修改队列; 可以使用查找 LCP 后一位字符相对大小的方法找到这个对应的事件。删除的时候重新获取前驱后继计算事件插入即可。区间查询时直接在 set 上运行 lower_bound。

LCP 性质与查询

对于任意三个字符串 A, B, C , 若满足字典序关系 $A \leq B \leq C$, 则必有:

$$\text{LCP}(A, C) = \min(\text{LCP}(A, B), \text{LCP}(B, C))$$

因此, 对于位置 l 和 r 的串的 LCP 查询, 配合 ST 表和 height 数组可实现 $O(1)$ 查询。

哈希与杂项

字符串 Hash 配合数据结构可动态维护可变字符串的 Hash。

引入行进制 B_1 与列进制 B_2 可实现二维矩阵 Hash。

3. 数论（不含博弈论）

3.1 Long Long $O(1)$ 乘, Barrett

```

1 LL modmul(LL a, LL b, LL M) { // skip2004, M < 63bit
2   | LL ret = a * b - M * LL(1.L * a / M * b + 0.5);
3   | return ret < 0 ? ret + M : ret; }
4 ULL modmul(ULL a, ULL b, LL M) { // orz@CF, M in 63 bit
5   | ULL c = (long double)a * b / M;
6   | LL ret = LL(a * b - c * M) % LL(M); // must be signed
7   | return ret < 0 ? ret + M : ret; }
8 // use int128 instead if M > 63 bit
9 struct DIV {
10  | ULL p, ip;
11  | void init (ULL _p) { p = _p; ip = -1llu / p; }
12  | int mod (ULL x) { // x < 2 ^ 64
13    | | ULL q = ULL((u128)ip * x) >> 64;
14    | | ULL r = x - q * p;
15    | | return int(r >= p ? r - p : r);
16  }; // speedup only when mod is not const

```

3.2 exgcd, 逆元

```

1 LL exgcd(LL a, LL b, LL &x, LL &y) {
2   | if (b == 0) return x = 1, y = 0, a;
3   | LL t = exgcd(b, a % b, y, x);
4   | y -= a / b * x; return t; }
5 LL inv(LL x, LL m) {
6   | LL a, b; exgcd(x, m, a, b); return (a % m + m) % m; }

```

递推逆元: $\text{inv}(i) \equiv (P - P/i) \cdot \text{inv}(P \bmod i)$

3.3 CRT 中国剩余定理

```

1 bool crt_merge(LL a1, LL m1, LL a2, LL m2, LL &A, LL &M) {
2   | LL c = a2 - a1, d = __gcd(m1, m2); //合并两个模方程
3   | if(c % d) return 0; // gcd(m1, m2) | (a2 - a1) 时才有解
4   | c = (c % m2 + m2) % m2; c /= d; m1 /= d; m2 /= d;
5   | c = c * inv(m1 % m2, m2) % m2; //0逆元可任意值
6   | M = m1*m2*d; A = (c * m1 % M * d % M + a1) % M; return 1; //有解

```

3.4 Miller Rabin, Pollard Rho

```

1 mt19937_64 rng(123);
2 #define rand() (LL)(rng() & LONGLONG_MAX)
3 const LL BASE[] = {2, 7, 61}; //int(7,3e9)
4 // {2,325,9375,28178,450775,9780504,1795265022}LL(37)
5 struct miller_rabin {
6   | bool check (LL M, LL base) {
7     | LL a = M - 1;
8     | while (~a & 1) a >>= 1;
9     | LL w = qpow (base, a, M); // qpow should use mul
10    | for (; a != M - 1 && w != 1 && w != M - 1; a <<= 1)
11      | w = mul (w, w, M);
12    | return w == M - 1 || (a & 1) == 1; }
13   | bool solve (LL a) { //O((3 or 7) * log n * mul)
14     | if (a < 4) return a > 1;
15     | if (~a & 1) return false;
16     | for (int i = 0; i < sizeof(BASE)/sizeof(BASE[0]); ++i) {
17       | LL chk = BASE[i] % a;
18       | if (chk == 0) return true;
19       | if (!check (a, chk)) return false;
20     }
21     | return true; } };
22 miller_rabin is_prime;
23 LL get_factor (LL a, LL seed) { //O(n^{1/4} * log n * mul)
24   | LL x = rand() % (a - 1) + 1, y = x;
25   | for (int head = 1, tail = 2; ; ) {
26     | | x = mul (x, x, a); x = (x + seed) % a;
27     | | if (x == y) return a;
28     | | LL ans = gcd (abs (x - y), a);
29     | | if (ans > 1 && ans < a) return ans;
30     | | if (++head == tail) { y = x; tail <= 1; } } }
31   | void factor (LL a, vector<LL> &d) {
32     | if (a <= 1) return;
33     | if (is_prime.solve (a)) d.push_back (a);
34     | else {
35       | | LL f = a;
36       | | for (; f >= a; f = get_factor (a, rand() % (a - 1) + 1));
37       | | factor (a / f, d);
38       | | factor (f, d); } }

```

3.5 扩展卢卡斯

```

1 int l,a[33],p[33],P[33];

```

```

2 U fac(int k,LL n){// 求 n! mod pk^tk, 返回值 U{ 不包含 pk 的
   | 值 ,pk 出现的次数 }
3   | if (!n)return U{1,0};LL x=n/p[k],y=n/P[k],ans=1;int i;
4   | if(y){// 求出循环节的答案
5     | | for(i=2;i<P[k];i++)if(i%p[k])ans=ans*i%P[k];
6     | | ans=Pw(ans,y,P[k]);
7   }for(i=y*P[k];i<=n;i++) if(i%p[k])ans=ans*i%M; // 求零散部
   | 分
8   | U z=fac(k,x);return U{ans*z.x%M,x+z.z};
9 }LL get(int k,LL n,LL m){// 求 C(n,m) mod pk^tk
10  | U a=fac(k,n),b=fac(k,m),c=fac(k,n-m); // 分三部分求解
11  | return Pw(p[k],a.z-b.z-c.z,P[k])*a.x%P[k]*
   | inv(b.x,P[k])%P[k]*inv(c.x,P[k])%P[k];
12 }LL CRT(){// CRT 合并答案
13  | LL d,w,y,x,ans=0;
14  | fr(i,1,l)w=M/P[i],exgcd(w,P[i],x,y),
   | ans=(ans+w*x%M*a[i])%M;
15  | return (ans+M)%M;
16 }LL C(LL n,LL m){// 求 C(n,m)
17  | fr(i,1,l)a[i]=get(i,n,m);
18  | return CRT();
19 }LL exLucas(LL n,LL m,int M){
20  | int jj=M,i //求 C(n,m)mod M,M=prod(pi^ki), 0(pi^kilg^2n)
21  | for(i=2;i*i<=jj;i++)if(jj%i==0)
22  | | for(p[1]=i,P[1]=1;jj%i==0;P[1]*=p[1])jj/=i;
23  | if(jj>1)l++,p[l]=P[l]=jj;
24  | return C(n,m);}

```

3.6 阶乘取模

```

1 // n! mod p^q Time : O(pq^2 log^2 n)
2 // Output : {a, b} means a*p^b
3 using Val=unsigned long long; //Val 需要 mod p^q 意义下 + *
4 typedef vector<Val> poly;
5 poly polymul(const poly &a,const poly &b){
6   | int n = (int) a.size(); poly c (n, Val(0));
7   | for (int i = 0; i < n; ++ i) {
8     | | for (int j = 0; i + j < n; ++ j) {
9       | | | c[i + j] = c[i + j] + a[i] * b[j]; } }
10  | return c; } Val choo[70][70];
11 poly polyshift(const poly &a, Val delta) {
12  | int n = (int) a.size(); poly res (n, Val(0));
13  | for (int i = 0; i < n; ++ i) { Val d = 1;
14    | | for (int j = 0; j <= i; ++ j) {
15      | | | res[i - j] = res[i - j] + a[i] * choo[i][j] * d;
16      | | | d = d * delta; } } return res; }
17 void prepare(int q) {
18  | for (int i = 0; i < q; ++ i) { choo[i][0] = Val(1);
19    | | for (int j = 1; j <= i; ++ j)
20      | | | choo[i][j]=choo[i-1][j-1]+choo[i-1][j]; } }
21 pair<Val, LL> fact(LL n, LL p, LL q) { Val ans = 1;
22   | for (int r = 1; r < p; ++ r) {
23     | | poly x (q, Val(0)), res (q, Val(0));
24     | | res[0] = 1; LL _res = 0; x[0] = r; LL _x = 0;
25     | | if (q > 1) x[1] = p, _x = 1; LL m = (n - r + p) / p;
26     | | while (m) { if (m & 1) {
27       | | | res=polymul(res,polyshift(x,_res)); _res+=_x; }
28       | | | m >>= 1; x = polymul(x, polyshift(x, _x)); _x+=_x;
   | } }
29   | ans = ans * res[0]; }
30   | LL cnt = n / p; if (n >= p) { auto tmp=fact(n / p, p, q);
31     | | ans = ans * tmp.first; cnt += tmp.second; }
32   | return {ans, cnt}; }

```

3.7 类欧几里得直线下格点统计

```

1 // \sum_{i=0}^{n-1} \lfloor \frac{a+bi}{m} \rfloor, n, m, a, b > 0
2 LL solve(LL n, LL a, LL b, LL m){
3   | if (b == 0) return n * (a / m);
4   | if (a >= m) return n * (a / m) + solve(n, a % m, b, m);
5   | if (b >= m) return (n-1)*n/2*(b/m) + solve(n,a,b%m,m);
6   | return solve((a + b * n) / m, (a + b * n) % m, m, b); }

```

3.8 万能欧几里德

```

1 Val work(LL P, LL R, LL Q, LL n, Val VU, Val VR) {
2   | // (Px+R)/Q, 1<=x<=i, 经过整数先U再R
3   | if(!(((i128)n * P + R) / Q)) return ksm(VR, n);
4   | if(P>=Q) return work(P%Q,R,Q,n, VU, ksm(VU, P/Q) * VR);
5   | Val res; swap(VU,VR);
6   | res = ksm(VU, (Q-R-1)/P)*VR;
7   | LL m = ((i128)n * P + R) / Q;
8   | res = res * work(Q, (Q-R-1)%P, P, m-1, VU, VR);
9   | return res * ksm(VU, n - ((i128)m*Q - R - 1) / P); }

```

3.9 平方剩余

```

1 // x^2=a (mod p), 0 <=a<p, 返回 true or false 代表是否存在解
2 // p必须是质数, 若是多个单质数的乘积, 可以分别求解再用CRT合并
3 // 复杂度为 O(log n)
4 void multiply(ll &c, ll &d, ll a, ll b, ll w) {
5     int cc = (a * c + b * d % MOD * w) % MOD;
6     int dd = (a * d + b * c) % MOD; c = cc, d = dd; }
7 bool solve(int n, int &x) {
8     if (n==0) return x=0, true; if (MOD==2) return x=1, true;
9     if (power(n, MOD / 2, MOD) == MOD - 1) return false;
10    ll c = 1, d = 0, b = 1, a, w;
11    // finding a such that a^2 - n is not a square
12    do { a = rand() % MOD; w = (a * a - n + MOD) % MOD;
13        if (w == 0) return x = a, true;
14        while (power(w, MOD / 2, MOD) != MOD - 1);
15        for (int times = (MOD + 1) / 2; times; times >>= 1) {
16            if (times & 1) multiply(c, d, a, b, w);
17            multiply(a, b, a, b, w); }
18        // x = (a + sqrt(w)) ^ ((p + 1) / 2)
19        return x = c, true; }

```

3.10 原根

定义 使得 $a^x \bmod m = 1$ 的最小的 x , 记作 $\delta_m(a)$. 若 $a \equiv g^s \bmod m$, 其中 g 为 m 的一个原根. 则虽然 s 随 g 的不同取值有所不同, 但是必然满足 $\delta_m(a) = \gcd(s, \varphi(m))$.

性质 $\delta_m(a^k) = \frac{\delta_m(a)}{\gcd(\delta_m(a), k)}$

k 次剩余 给定方程 $x^k \equiv a \bmod m$, 求所有解. 若 k 与 $\varphi(m)$ 互质, 则可以直接求出 k 对 $\varphi(m)$ 的逆元. 否则, 将 k 拆成两部分, $k = uv$, 其中 $u \perp \varphi(m)$, $v | \varphi(m)$, 先求 $x^v \equiv a \bmod m$, 则 $ans = x^{u^{-1}}$. 下面讨论 $k | \varphi(m)$ 的问题. 任取一原根 g , 对两侧取离散对数, 设 $x = g^s, a = g^t$, 其中 t 可以用BSGS求出, 则问题转化为求出所有的 s 满足 $ks \equiv t \bmod \varphi(m)$, exgcd 即可求解, 显然有解的条件是 $k | \delta_m(a)$.

3.11 min25筛

```

1 typedef long long ll;
2
3 ll n;
4 ll pri[N], sta[M], g[M]; int tot, mod;
5 int getid(ll x) {
6     if (x <= sqrt(n)) return x;
7     return tot + 1 - (n / x);
8 }
9
10 int solve(ll n, int id) {
11     if (n < pri[id]) return 0;
12     if (n / pri[id] < pri[id])
13         return (g[getid(n)] - (id > 1 ? g[pri[id - 1]] : 0) +
14             mod) % mod;
15     ll e = n / pri[id]; int d = pri[id], Sum = 0;
16     while (e) {
17         Sum = (Sum + (ll)d * (d - 1 + mod) % mod * (solve(e,
18             id + 1) + 1)) % mod;
19         d = (ll)d * pri[id] % mod; e /= pri[id];
20     }
21     return (Sum + solve(n, id + 1)) % mod;
22 }
23
24 void init() {
25     for (int i = 1; i <= pri[0]; i++)
26         for (int j = tot; j >= 1; j--) {
27             if (sta[j] / pri[i] < pri[i]) break;
28             int num = (g[getid(sta[j] / pri[i])] - g[pri[i]] -
29                 mod + mod) % mod;
30             g[j] = (g[j] - (ll)pri[i] * pri[i] % mod * num %
31                 mod + mod) % mod;
32         }
33 }

```

3.12 Pell 方程

```

1 // x^2 - n * y^2 = 1 最小正整数根, n 为完全平方数时无解
2 // x_{k+1} = x_0 x_k + n y_0 y_k
3 // y_{k+1} = x_0 y_k + y_0 x_k
4 pair<LL, LL> pell(LL n) {
5     static LL p[N], q[N], g[N], h[N], a[N];
6     p[1] = q[0] = h[1] = 1; p[0] = q[1] = g[1] = 0;
7     a[2] = (LL)(floor(sqrt(1.0 * n + 1e-7L)));
8     for (int i = 2; ; i++) {
9         g[i] = -g[i - 1] + a[i] * h[i - 1];

```

```

10 | | h[i] = (n - g[i] * g[i]) / h[i - 1];
11 | | a[i + 1] = (g[i] + a[2]) / h[i];
12 | | p[i] = a[i] * p[i - 1] + p[i - 2];
13 | | q[i] = a[i] * q[i - 1] + q[i - 2];
14 | | if (p[i] * p[i] - n * q[i] * q[i] == 1)
15 | |     return {p[i], q[i]}; }

```

3.13 解一元三次方程

```

1 double a(p[3]), b(p[2]), c(p[1]), d(p[0]);
2 double k(b / a), m(c / a), n(d / a);
3 double p(-k * k / 3. + m);
4 double q(2. * k * k * k / 27 - k * m / 3. + n);
5 Complex omega[3] = {Complex(1, 0), Complex(-0.5, 0.5 *
6     sqrt(3)), Complex(-0.5, -0.5 * sqrt(3))};
7 Complex r1, r2; double delta(q * q / 4 + p * p * p / 27);
8 if (delta > 0) {
9     r1 = cubrt(-q / 2. + sqrt(delta));
10    r2 = cubrt(-q / 2. - sqrt(delta));
11 } else {
12     r1 = pow(-q / 2. + pow(Complex(delta), 0.5), 1. / 3);
13     r2 = pow(-q / 2. - pow(Complex(delta), 0.5), 1. / 3);
14 }
15 for (int _ = 0; _ < 3; _++) {
16     Complex x = -k/3. + r1*omega[_] + r2*omega[_*2%3]; }

```

3.14 Formulas 公式表

3.14.1 Mobius Inversion

$$F(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) F\left(\frac{n}{d}\right)$$

$$F(n) = \sum_{n|d} f(d) \Rightarrow f(n) = \sum_{n|d} \mu\left(\frac{d}{n}\right) F(d) \quad \text{引理}$$

$$[x = 1] = \sum_{d|x} \mu(d), \quad x = \sum_{d|x} \mu(d)$$

3.14.5 单位根反演

$$\sum_{i=0}^{\lfloor \frac{n}{k} \rfloor} C_n^{ik}$$

$$\frac{1}{k} \sum_{i=0}^{k-1} \omega_k^{in} = [k | n]$$

3.14.2 杜教筛

反演

$$S_\varphi(n) = \frac{n(n+1)}{2} - \sum_{d=2}^n S_\varphi\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$$

$$Ans = \sum_{i=0}^n C_n^i [k | i]$$

$$S_\mu(n) = 1 - \sum_{d=2}^n S_\mu\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$$

$$= \sum_{i=0}^n C_n^i \left(\frac{1}{k} \sum_{j=0}^{k-1} \omega_k^{ij}\right)$$

3.14.3 降幂公式

$$a^k \equiv a^{k \bmod \varphi(p) + \varphi(p)}, \quad k \geq \varphi(p)$$

$$= \frac{1}{k} \sum_{i=0}^n C_n^i \sum_{j=0}^{k-1} \omega_k^{ij}$$

3.14.4 其他常用公式

$$\sum_{i=1}^n [(i, n) = 1] i = n \frac{\varphi(n) + e(n)}{2}$$

$$= \frac{1}{k} \sum_{j=0}^{k-1} \left(\sum_{i=0}^n C_n^i (\omega_k^j)^i\right)$$

$$\sum_{i=1}^n \sum_{j=1}^i [(i, j) = d] = S_\varphi\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$$

$$= \frac{1}{k} \sum_{j=0}^{k-1} (1 + \omega_k^j)^n$$

$$\sum_{i=1}^n \sum_{j=1}^m [(i, j) = d] = \sum_{d|k} \mu\left(\frac{k}{d}\right) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor$$

另, 如果要求的是 $[n\%k = t]$, 其实就是 $[k | (n - t)]$. 同理推式子即可.

$$\sum_{i=1}^n f(i) \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} g(j) = \sum_{i=1}^n g(i) \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} f(j)$$

3.14.6 Arithmetic Function

$$(p-1)! \equiv -1 \pmod{p}$$

$$a > 1, m, n > 0, \text{ then } \gcd(a^m - 1, a^n - 1) = a^{\gcd(m, n)} - 1$$

$$\mu^2(n) = \sum_{d^2|n} \mu(d)$$

$$a > b, \gcd(a, b) = 1, \text{ then } \gcd(a^m - b^m, a^n - b^n) = a^{\gcd(m, n)} - b^{\gcd(m, n)}$$

$$\prod_{k=1, \gcd(k, m)=1}^m k \equiv \begin{cases} -1 & \text{mod } m, m = 4, p^q, 2p^q \\ 1 & \text{mod } m, \text{ otherwise} \end{cases}$$

$$\sigma_k(n) = \sum_{d|n} d^k = \prod_{i=1}^{\omega(n)} \frac{p_i^{(a_i+1)k} - 1}{p_i^k - 1}$$

$$J_k(n) = n^k \prod_{p|n} \left(1 - \frac{1}{p^k}\right)$$

$J_k(n)$ is the number of k -tuples of positive integers all less than or equal to n that form a coprime $(k+1)$ -tuple together with n .

$$\sum_{\delta|n} J_k(\delta) = n^k$$

$$\sum_{i=1}^n \sum_{j=1}^n [\gcd(i, j) = 1] ij = \sum_{i=1}^n i^2 \varphi(i)$$

$$\sum_{\delta|n} \delta^s J_r(\delta) J_s\left(\frac{n}{\delta}\right) = J_{r+s}(n)$$

$$\begin{aligned} \sum_{\delta|n} \varphi(\delta) d\left(\frac{n}{\delta}\right) &= \sigma(n), \sum_{\delta|n} |\mu(\delta)| = 2^{\omega(n)} \\ \sum_{\delta|n} 2^{\omega(\delta)} &= d(n^2), \sum_{\delta|n} d(\delta^2) = d^2(n) \\ \sum_{\delta|n} d\left(\frac{n}{\delta}\right) 2^{\omega(\delta)} &= d^2(n), \sum_{\delta|n} \frac{\mu(\delta)}{\delta} = \frac{\varphi(n)}{n} \\ \sum_{\delta|n} \frac{\mu(\delta)}{\varphi(\delta)} &= d(n), \sum_{\delta|n} \frac{\mu^2(\delta)}{\varphi(\delta)} = \frac{n}{\varphi(n)} \\ n|\varphi(a^n - 1) \\ \sum_{\substack{1 \leq k \leq n \\ \gcd(k, n)=1}} f(\gcd(k-1, n)) &= \varphi(n) \sum_{d|n} \frac{(\mu * f)(d)}{\varphi(d)} \\ \varphi(\operatorname{lcm}(m, n)) \varphi(\gcd(m, n)) &= \varphi(m) \varphi(n) \\ \sum_{\delta|n} d^3(\delta) &= \left(\sum_{\delta|n} d(\delta)\right)^2 \\ d(uv) &= \sum_{\delta|\gcd(u, v)} \mu(\delta) d\left(\frac{u}{\delta}\right) d\left(\frac{v}{\delta}\right) \\ \sigma_k(u) \sigma_k(v) &= \sum_{\delta|\gcd(u, v)} \delta^k \sigma_k\left(\frac{uv}{\delta^2}\right) \\ \mu(n) &= \sum_{k=1}^n [\gcd(k, n) = 1] \cos 2\pi \frac{k}{n} \\ \varphi(n) &= \sum_{k=1}^n [\gcd(k, n) = 1] = \sum_{k=1}^n \gcd(k, n) \cos 2\pi \frac{k}{n} \\ \begin{cases} S(n) = \sum_{k=1}^n (f * g)(k) \\ \sum_{k=1}^n S(\lfloor \frac{n}{k} \rfloor) = \sum_{i=1}^n f(i) \sum_{j=1}^{\lfloor \frac{n}{i} \rfloor} (g * 1)(j) \end{cases} \\ \begin{cases} S(n) = \sum_{k=1}^n (f \cdot g)(k), g \text{ completely multiplicative} \\ \sum_{k=1}^n S\left(\left\lfloor \frac{n}{k} \right\rfloor\right) g(k) = \sum_{k=1}^n (f * 1)(k) g(k) \end{cases} \end{aligned}$$

3.14.7 Binomial Coefficients

C	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	3	1							
4	1	4	6	4	1						
5	1	5	10	10	5	1					
6	1	6	15	20	15	6	1				
7	1	7	21	35	35	21	7	1			
8	1	8	28	56	70	56	28	8	1		
9	1	9	36	84	126	126	84	36	9	1	
10	1	10	45	120	210	252	210	120	45	10	1

$$\binom{n}{k} \equiv [n \& k = k] \pmod{2}$$

$$\begin{aligned} \sum_{k=0}^n \binom{k}{m} &= \binom{n+1}{m+1} \\ \sqrt{1+z} &= 1 + \sum_{k=1}^{\infty} \frac{(-1)^{k-1}}{k} \times \frac{2k-2}{2^{2k-1}} \binom{2k-2}{k-1} z^k \\ \sum_{k=0}^r \binom{r-k}{m} \binom{s+k}{n} &= \binom{r+s+1}{m+n+1} \\ C_{n,m} &= \binom{n+m}{m} - \binom{n+m}{m-1}, n \geq m \end{aligned}$$

$$\begin{aligned} \binom{n}{k} &= (-1)^k \binom{k-n-1}{k}, \sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{n} \\ \binom{n_1 + \dots + n_p}{m} &= \sum_{k_1 + \dots + k_p = m} \binom{n_1}{k_1} \dots \binom{n_p}{k_p} \end{aligned}$$

3.14.8 Fibonacci Numbers, Lucas Numbers

$$\begin{aligned} F(z) &= \frac{z}{1-z-z^2} \\ \hat{\phi} &= \frac{1-\sqrt{5}}{2} \\ \sum_{k=1}^n f_k &= f_{n+2} - 1, \sum_{k=1}^n f_k^2 = f_n f_{n+1} \\ \sum_{k=0}^n f_k f_{n-k} &= \frac{1}{5}(n-1)f_n + \frac{2}{5}f_{n-1} \\ \frac{f_{2n}}{f_n} &= f_{n-1} + f_{n+1} \\ f_1 + 2f_2 + 3f_3 + \dots + nf_n &= nf_{n+2} - f_{n+3} + 2 \end{aligned}$$

```

def fib(n): # F(n), F(n+1)
    if not n: return (0, 1)
    a, b = fib(n >> 1)
    c = a * (2 * b - a)
    d = a * a + b * b
    if n & 1:
        return (d, c + d)
    else:
        return (c, d)

```

$$\text{Modulo } f_n, f_{mn+r} \equiv \begin{cases} f_r, & m \bmod 4 = 0; \\ (-1)^{r+1} f_{n-r}, & m \bmod 4 = 1; \\ (-1)^n f_r, & m \bmod 4 = 2; \\ (-1)^{r+1+n} f_{n-r}, & m \bmod 4 = 3. \end{cases}$$

$$L_0 = 2, L_1 = 1, L_n = L_{n-1} + L_{n-2} = \left(\frac{1+\sqrt{5}}{2}\right)^n + \left(\frac{1-\sqrt{5}}{2}\right)^n$$

$$L(x) = \frac{2-x}{1-x-x^2}$$

除了 $n = 0, 4, 8, 16, L_n$ 是素数, 则 n 是素数.

$$\begin{aligned} \phi &= \frac{1+\sqrt{5}}{2}, \hat{\phi} = \frac{1-\sqrt{5}}{2} \\ F_n &= \frac{\phi^n - \hat{\phi}^n}{\sqrt{5}}, L_n = \phi^n + \hat{\phi}^n \\ \frac{L_n + F_n \sqrt{5}}{2} &= \left(\frac{1+\sqrt{5}}{2}\right)^n \end{aligned}$$

3.14.9 Sum of Powers

$$\begin{aligned} \sum_{i=1}^n i^2 &= \frac{n(n+1)(2n+1)}{6}, \sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2}\right)^2 \\ \sum_{i=1}^n i^4 &= \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30} \\ \sum_{i=1}^n i^5 &= \frac{n^2(n+1)^2(2n^2+2n-1)}{12} \end{aligned}$$

3.14.10 Catalan Numbers 1, 1, 2, 5, 14, 42, 132, 429, 1430...

$$\begin{aligned} c_0 = 1, c_n &= \sum_{i=0}^{n-1} c_i c_{n-1-i} = c_{n-1} \frac{4n-2}{n+1} = \frac{\binom{2n}{n}}{n+1} = \binom{2n}{n} - \binom{2n}{n-1} \\ c(x) &= \frac{1-\sqrt{1-4x}}{2x} \end{aligned}$$

Usage: n 括号序列; n 个点满二叉树; $n \times n$ 的方格左下到右上不过对角线方案数; 凸 $n+2$ 边形三角形分割数; n 个数的出栈方案数; $2n$ 个顶点连接, 线段两两不交的的方案数.

类卡特兰数 从 $(1, 1)$ 出发走到 (n, m) , 只能向右或者向上走, 不能越过 $y = x$ 这条线 (即保证 $x \geq y$), 合法方案数是 $C_{n+m-2}^n - C_{n+m-2}^{n-1}$.

3.14.11 Motzkin Numbers 1, 1, 2, 4, 9, 21, 51, 127, 323, 835...

圆上 n 点间画不相交弦的方案数. 选 n 个数 $k_1, k_2, \dots, k_n \in \{-1, 0, 1\}$, 保证 $\sum_i^a k_i (1 \leq a \leq n)$ 非负且所有数总和为 0 的方案数.

$$M_{n+1} = M_n + \sum_{i=1}^{n-1} M_i M_{n-1-i} = \frac{(2n+3)M_n + 3nM_{n-1}}{n+3}$$

$$M_n = \sum_{i=0}^{\lfloor \frac{n}{2} \rfloor} \binom{n}{2k} \text{Catlan}(k)$$

$$M(X) = \frac{1-x-\sqrt{1-2x-3x^2}}{2x^2}$$

3.14.12 Derangement 错排数 0, 1, 2, 9, 44, 265, 1854, 14833...

$$\begin{aligned} D_1 = 0, D_2 = 1, D_n &= n! \left(\frac{1}{0!} - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots + \frac{(-1)^n}{n!} \right) \\ D_n &= (n-1)(D_{n-1} + D_{n-2}) \end{aligned}$$

3.14.13 Bell Numbers 1, 1, 2, 5, 15, 52, 203, 877, 4140 ...

n 个元素集合划分的方案数.

$$B_n = \sum_{k=1}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\}, B_{n+1} = \sum_{k=0}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} B_k$$

$$B_{p^m+n} \equiv mB_n + B_{n+1} \pmod{p}$$

$$B(x) = \sum_{n=0}^{\infty} \frac{B_n}{n!} x^n = e^{e^x-1}$$

3.14.14 Stirling Numbers

第一类 n 个元素集合分作 k 个非空轮换方案数.

$$\begin{aligned} \left[\begin{matrix} n+1 \\ k \end{matrix} \right] &= n \left[\begin{matrix} n \\ k \end{matrix} \right] + \left[\begin{matrix} n \\ k-1 \end{matrix} \right] \\ \left[\begin{matrix} n+1 \\ 2 \end{matrix} \right] &= n! H_n \text{ (see 3.14.16)} \\ x^n &= \sum_k \left[\begin{matrix} n \\ k \end{matrix} \right] (-1)^{n-k} x^k \\ x^{\bar{n}} &= \sum_k \left[\begin{matrix} n \\ k \end{matrix} \right] x^k \end{aligned}$$

n\k	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	1	1				
3	0	2	3	1			
4	0	6	11	6	1		
5	0	24	50	35	10	1	
6	0	120	274	225	85	15	1
7	0	720	1764	1624	735	175	21

For fixed k , EGF:

$$\sum_{n=0}^{\infty} \left[\begin{matrix} n \\ k \end{matrix} \right] \frac{x^n}{n!} = \frac{x^k}{k!} \left(\frac{\ln(1-x)}{x} \right)^k$$

第二类 把 n 个元素集合分作 k 个非空子集方案数.

$$\begin{aligned} \left\{ \begin{matrix} n+1 \\ k \end{matrix} \right\} &= k \left\{ \begin{matrix} n \\ k \end{matrix} \right\} + \left\{ \begin{matrix} n \\ k-1 \end{matrix} \right\} \\ m! \left\{ \begin{matrix} n \\ m \end{matrix} \right\} &= \sum_k \binom{m}{k} k^n (-1)^{m-k} \\ x^n &= \sum_k \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^k \\ &= \sum_k \left\{ \begin{matrix} n \\ k \end{matrix} \right\} (-1)^{n-k} x^{\bar{k}} \end{aligned}$$

For fixed k , EGF and OGF:

$$\begin{aligned} \sum_{n=0}^{\infty} \left\{ \begin{matrix} n \\ k \end{matrix} \right\} \frac{x^n}{n!} &= \frac{x^k}{k!} \left(\frac{e^x - 1}{x} \right)^k \\ \sum_{n=0}^{\infty} \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^n &= x^k \prod_{i=1}^k (1 - ix)^{-1} \end{aligned}$$

n\k	0	1	2	3	4	5	6
0	1						
1	0	1					
2	0	1	1				
3	0	1	3	1			
4	0	1	7	6	1		
5	0	1	15	25	10	1	
6	0	1	31	90	65	15	1
7	0	1	63	301	350	140	21

3.14.15 Eulerian Numbers

n\k	0	1	2	3	4	5	6
1	1						
2	1	1					
3	1	4	1				
4	1	11	11	1			
5	1	26	66	26	1		
6	1	57	302	302	57	1	
7	1	120	1191	2416	1191	120	1

3.14.16 Harmonic Numbers, 1, 3/2, 11/6, 25/12, 137/60...

$$H_n = \sum_{k=1}^n \frac{1}{k}, \sum_{k=1}^n H_k = (n+1)H_n - n$$

$$\sum_{k=1}^n kH_k = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}$$

$$\sum_{k=1}^n \binom{k}{m} H_k = \binom{n+1}{m+1} (H_{n+1} - \frac{1}{m+1})$$

3.14.18 求拆分数

```
def penta(k):
    return k*(3*k-1)//2
def compute_partition(goal):
    p = [1]
    for n in range(1,goal+1):
        p.append(0)
        for k in range(1,n+1):
            c = (-1)**(k+1)
            for t in [penta(k), penta(-k)]:
                if (n-t) >= 0:
                    p[n] = p[n] + c*p[n-t]
    return p
```

$$\Phi(x) = \prod_{n=1}^{\infty} (1 - x^n)$$

$$= \sum_{n=-\infty}^{\infty} (-1)^k x^{k(3k-1)/2}$$

3.14.19 Bernoulli Numbers 1, 1/2, 1/6, 0, -1/30, 0, 1/42 ...

$$B(x) = \sum_{i \geq 0} \frac{B_i x^i}{i!} = \frac{x}{e^x - 1}$$

$$B_n = [n=0] - \sum_{i=0}^{n-1} \binom{n}{i} \frac{B_i}{n-k+1}, \sum_{i=0}^n \binom{n+1}{i} B_i = 0$$

$$S_n(m) = \sum_{i=0}^{m-1} i^n = \sum_{i=0}^n \binom{n}{i} B_{n-i} \frac{m^{i+1}}{i+1}$$

$$B_0 = 1, B_1 = -\frac{1}{2}, B_4 = -\frac{1}{30}, B_6 = \frac{1}{42}, B_8 = -\frac{1}{30}, \dots$$

(除了 $B_1 = -\frac{1}{2}$ 以外, 伯努利数的奇数项都是 0.)

自然数幂次和关于次数的EGF:

$$F(x) = \sum_{k=0}^{\infty} \frac{\sum_{i=0}^n i^k}{k!} x^k$$

$$= \sum_{i=0}^n e^{ix} = \frac{e^{(n+1)x-1}}{e^x - 1}$$

3.14.20 kMAX-MIN反演

$$k\text{MAX}(S) = \sum_{T \subset S, T \neq \emptyset} (-1)^{|T|-k} C_{|T|-1}^{k-1} \text{MIN}(T)$$

代入 $k=1$ 即为MAX-MIN反演:

$$\text{MAX}(S) = \sum_{T \subset S, T \neq \emptyset} (-1)^{|T|-1} \text{MIN}(T)$$

3.14.21 伍德伯里矩阵不等式

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}$$

该等式可以动态维护矩阵的逆, 令 $C = [1]$, U, V 分别为 $1 \times n$ 和 $n \times 1$ 的向量, 这样可以构造出 UCV 为只有某行或者某列不为0的矩阵, 一次修改复杂度为 $O(n^2)$.

3.14.22 Sum of Squares

$r_k(n)$ 表示用 k 个平方数组成 n 的方案数. 假设:

$$n = 2^{a_0} p_1^{2a_1} \dots p_r^{2a_r} q_1^{b_1} \dots q_s^{b_s}$$

其中 $p_i \equiv 3 \pmod{4}$, $q_i \equiv 1 \pmod{4}$, 那么

$$r_2(n) = \begin{cases} 0 & \text{if any } a_i \text{ is a half-integer} \\ 4 \prod_{i=1}^r (b_i + 1) & \text{if all } a_i \text{ are integers} \end{cases}$$

$r_3(n) > 0$ 当且仅当 n 不满足 $4^a(8b+7)$ 的形式 (a, b 为整数).

3.14.23 枚举勾股数 Pythagorean Triple

枚举 $x^2 + y^2 = z^2$ 的三元组: 可令 $x = m^2 - n^2$, $y = 2mn$, $z = m^2 + n^2$, 枚举 m 和 n 即可 $O(n)$ 枚举勾股数. 判断素勾股数方法: m, n 至少一个为偶数并且 m, n 互质, 那么 x, y, z 就是素勾股数.

3.14.24 四面体体积 Tetrahedron Volume

If U, V, W, u, v, w are lengths of edges of the tetrahedron (first three form a triangle; u opposite to U and so on)

$$V = \frac{\sqrt{4u^2v^2w^2 - \sum_{cyc} u^2(v^2 + w^2 - U^2)^2 + \prod_{cyc} (v^2 + w^2 - U^2)}}{12}$$

3.14.25 杨氏矩阵与钩子公式

满足: 格子 (i, j) 没有元素, 则它右边和上边相邻格子也没有元素; 格子 (i, j) 有元素 $a[i][j]$, 则它右边和上边相邻格子要么没有元素, 要么有元素且比 $a[i][j]$ 大.

计数: $F_1 = 1, F_2 = 2, F_n = F_{n-1} + (n-1)F_{n-2}, F(x) = e^{x + \frac{x^2}{2}}$

钩子公式: 对于给定形状 λ , 不同杨氏矩阵的个数为:

$$d_\lambda = \frac{n!}{\prod h_\lambda(i, j)}$$

$h_\lambda(i, j)$ 表示该格子右边和上边的格子数量加1.

3.14.26 常见博弈游戏

Nim-K游戏 n 堆石子轮流拿, 每次最多可以拿 k 堆石子, 谁走最后一步输. 结论: 把每一堆石子的sg值(即石子数量)二进制分解, 先手必败当且仅当每一位二进制位上1的个数是 $(k+1)$ 的倍数.

Anti-Nim游戏 n 堆石子轮流拿, 谁走最后一步输. 结论: 先手胜当且仅当1. 所有堆石子数都为1且游戏的SG值为0 (即有偶数个孤单堆-每堆只有1个石子数) 2. 存在某堆石子数大于1且游戏的SG值不为0.

斐波那契博弈 有一堆物品, 两人轮流取物品, 先手最少取一个, 至多无上限, 但不能把物品取完, 之后每次取物品数不能超过上一次取的物品数的二倍且至少为一件, 取走最后一件物品的人获胜. 结论: 先手胜当且仅当物品数 n 不是斐波那契数.

威佐夫博弈 有两堆石子, 博弈双方每次可以取一堆石子中的任意个, 不能不取, 或

者取两堆石子中的相同个. 先取完者赢. 结论: 求出两堆石子 A 和 B 的差值 C , 如果 $\left\lfloor C * \frac{\sqrt{5}+1}{2} \right\rfloor = \min(A, B)$ 那么后手赢, 否则先手赢.

约瑟夫环 n 个人 $0, \dots, n-1$, 令 $f_{i,m}$ 为 i 个人报 m 的胜利者, 则 $f_{i,m} = 0, f_{i,m} = (f_{i-1,m} + m) \bmod i$.

阶梯Nim 在一个阶梯上, 每次选一个台阶上任意个石子移到下一个台阶上, 不可移动者输. 结论: SG值等于奇数层台阶上石子数的异或和. 对于树形结构也适用, 奇数层节点上所有石子数异或起来即可.

图上博弈 给定无向图, 先手从某点开始走, 只能走相邻且未走过的点, 无法移动者输. 对该图求最大匹配, 若某个点不一定在最大匹配中则先手必败, 否则先手必胜.

最大最小定理求纳什均衡点 在二人零和博弈中, 可以用以下方式求出一个纳什均衡点: 在博弈双方中任选一方, 求混合策略 p 使得对方选择任意一个纯策略时, 己方的最小收益最大(等价于对方的最大收益最小). 据此可以求出双方在局面下的最优期望得分, 分别等于己方最大的最小收益和对方最小的最大收益. 一般而言, 可以得到形如

$$\max_p \min_i \sum_{j \in p} p_j w_{i,j}, \text{ s.t. } p_j \geq 0, \sum p_j = 1$$

的形式. 当 $\sum p_j w_{i,j}$ 可以表示成只与 i 有关的函数 $f(i)$ 时, 可以令初始时 $p_i = 0$, 不断调整 $\sum p_j w_{i,j}$ 最小的那个 i 的概率 p_i , 直至无法调整或者 $\sum p_j = 1$ 为止.

3.14.27 概率相关

$$D(X) = E(X - E(X))^2 = E(X^2) - (E(X))^2, D(X+Y) = D(X) + D(Y), D(aX) = a^2 D(X)$$

$$E[x] = \sum_{i=1}^{\infty} P(X \geq i)$$

m 个数的方差:

$$s^2 = \frac{\sum_{i=1}^m x_i^2}{m} - \bar{x}^2$$

3.14.28 邻接矩阵行列式的意义

在无向图中取若干个环, 一种取法权值就是边权的乘积, 对行列式的贡献是 $(-1)^{\text{even}}$, 其中 even 是偶环的个数.

3.14.29 Others (某些近似数值公式在这里)

$$S_j = \sum_{k=1}^n x_k^j$$

$$h_m = \sum_{1 \leq j_1 < \dots < j_m \leq n} x_{j_1} \dots x_{j_m}, H_m = \sum_{1 \leq j_1 \leq \dots \leq j_m \leq n} x_{j_1} \dots x_{j_m}$$

$$h_n = \frac{1}{n} \sum_{k=1}^n (-1)^{k+1} S_k h_{n-k}$$

$$H_n = \frac{1}{n} \sum_{k=1}^n S_k H_{n-k}$$

$$\sum_{k=0}^n kc^k = \frac{nc^{n+2} - (n+1)c^{n+1} + c}{(c-1)^2}$$

$$\sum_{i=1}^n \frac{1}{n} \approx \ln\left(n + \frac{1}{2}\right) + \frac{1}{24(n+0.5)^2} + \Gamma, (\Gamma \approx 0.577215664901532860605)$$

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \frac{1}{12n} + \frac{1}{288n^2} + O\left(\frac{1}{n^3}\right)\right)$$

$$\max \{x_a - x_b, y_a - y_b, z_a - z_b\} - \min \{x_a - x_b, y_a - y_b, z_a - z_b\}$$

$$= \frac{1}{2} \sum_{cyc} |(x_a - y_a) - (x_b - y_b)|$$

$$(a+b)(b+c)(c+a) = \frac{(a+b+c)^3 - a^3 - b^3 - c^3}{3}$$

$$a^3 + b^3 = (a+b)(a^2 - ab + b^2), a^3 - b^3 = (a-b)(a^2 + ab + b^2)$$

$n \bmod 2 = 1$:

$$a^n + b^n = (a+b)(a^{n-1} - a^{n-2}b + a^{n-3}b^2 - \dots - ab^{n-2} + b^{n-1})$$

划分问题: n 个 $k-1$ 维向量最多把 k 维空间分为 $\sum_{i=0}^k C_n^i$ 份.

3.15 Calculus, Integration Table 导数积分表

$$\begin{aligned}
\left(\frac{u}{v}\right)' &= \frac{u'v - uv'}{v^2} & (\arctan x)' &= \frac{1}{1+x^2} & (\operatorname{arcsinh} x)' &= \frac{1}{\sqrt{1+x^2}} \\
(a^x)' &= (\ln a)a^x & (\operatorname{arccot} x)' &= -\frac{1}{1+x^2} & (\operatorname{arccosh} x)' &= \frac{1}{\sqrt{x^2-1}} \\
(\tan x)' &= \sec^2 x & (\operatorname{arccsc} x)' &= -\frac{1}{x\sqrt{1-x^2}} & (\operatorname{arctanh} x)' &= \frac{1}{1-x^2} \\
(\cot x)' &= -\csc^2 x & (\operatorname{arcsec} x)' &= \frac{1}{x\sqrt{1-x^2}} & (\operatorname{arccoth} x)' &= \frac{1}{x^2-1} \\
(\sec x)' &= \tan x \sec x & (\tanh x)' &= \operatorname{sech}^2 x & (\operatorname{arcsch} x)' &= -\frac{1}{|x|\sqrt{1+x^2}} \\
(\csc x)' &= -\cot x \csc x & (\coth x)' &= -\operatorname{csch}^2 x & (\operatorname{arcsech} x)' &= -\frac{1}{x\sqrt{1-x^2}} \\
(\arcsin x)' &= \frac{1}{\sqrt{1-x^2}} & (\operatorname{sech} x)' &= -\operatorname{sech} x \tanh x & & \\
(\arccos x)' &= -\frac{1}{\sqrt{1-x^2}} & (\operatorname{csch} x)' &= -\operatorname{csch} x \coth x & &
\end{aligned}$$

$$\cdot ax^2 + bx + c (a > 0)$$

$$\int \frac{dx}{ax^2+bx+c} = \begin{cases} \frac{2}{\sqrt{4ac-b^2}} \arctan \frac{2ax+b}{\sqrt{4ac-b^2}} + C & (b^2 < 4ac) \\ \frac{1}{\sqrt{b^2-4ac}} \ln \left| \frac{2ax+b-\sqrt{b^2-4ac}}{2ax+b+\sqrt{b^2-4ac}} \right| + C & (b^2 > 4ac) \end{cases}$$

$$\int \frac{x}{ax^2+bx+c} dx = \frac{1}{2a} \ln |ax^2 + bx + c| - \frac{1}{2a} \int \frac{dx}{ax^2+bx+c}$$

$$\cdot \sqrt{\pm ax^2 + bx + c} (a > 0)$$

$$\int \frac{dx}{\sqrt{ax^2+bx+c}} = \frac{1}{\sqrt{a}} \ln |2ax + b + 2\sqrt{a}\sqrt{ax^2 + bx + c}| + C$$

$$\int \sqrt{ax^2 + bx + c} dx = \frac{2ax+b}{4a} \sqrt{ax^2 + bx + c} + \frac{4ac-b^2}{8\sqrt{a^3}} \ln |2ax + b + 2\sqrt{a}\sqrt{ax^2 + bx + c}| + C$$

$$\int \frac{x}{\sqrt{ax^2+bx+c}} dx = \frac{1}{a} \sqrt{ax^2 + bx + c} - \frac{b}{2\sqrt{a^3}} \ln |2ax + b + 2\sqrt{a}\sqrt{ax^2 + bx + c}| + C$$

$$\int \frac{dx}{\sqrt{c+bx-ax^2}} = -\frac{1}{\sqrt{a}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C$$

$$\int \sqrt{c+bx-ax^2} dx = \frac{2ax-b}{4a} \sqrt{c+bx-ax^2} + \frac{b^2+4ac}{8\sqrt{a^3}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C$$

$$\int \frac{x}{\sqrt{c+bx-ax^2}} dx = -\frac{1}{a} \sqrt{c+bx-ax^2} + \frac{b}{2\sqrt{a^3}} \arcsin \frac{2ax-b}{\sqrt{b^2+4ac}} + C$$

$$\cdot \sqrt{\pm \frac{x-a}{x-b}} \text{ 或 } \sqrt{(x-a)(x-b)}, (a < b)$$

$$\int \frac{dx}{\sqrt{(x-a)(b-x)}} = 2 \arcsin \sqrt{\frac{x-a}{b-x}} + C$$

$$\int \sqrt{(x-a)(b-x)} dx = \frac{2x-a-b}{4} \sqrt{(x-a)(b-x)} + \frac{(b-a)^2}{4} \arcsin \sqrt{\frac{x-a}{b-x}} + C$$

三角函数的积分 (省略+C)

$$\begin{aligned}
\int \tan x dx &= -\ln |\cos x| & \int \sec^2 x dx &= \tan x \\
\int \cot x dx &= \ln |\sin x| & \int \csc^2 x dx &= -\cot x \\
\int \sec x dx &= \ln \left| \tan \left(\frac{x}{4} + \frac{\pi}{2} \right) \right| & \int \sec x \tan x dx &= \sec x \\
&= \ln |\sec x + \tan x| & \int \csc x \cot x dx &= -\csc x \\
\int \csc x dx &= \ln \left| \tan \frac{x}{2} \right| & \int \sin^2 x dx &= \frac{x}{2} - \frac{1}{4} \sin 2x \\
&= \ln |\csc x - \cot x| & \int \cos^2 x dx &= \frac{x}{2} + \frac{1}{4} \sin 2x
\end{aligned}$$

$$\int \sin^n x dx = -\frac{1}{n} \sin^{n-1} x \cos x + \frac{n-1}{n} \int \sin^{n-2} x dx$$

$$\int \cos^n x dx = \frac{1}{n} \cos^{n-1} x \sin x + \frac{n-1}{n} \int \cos^{n-2} x dx$$

$$\int \frac{dx}{\sin^n x} = -\frac{1}{n-1} \frac{\cos x}{\sin^{n-1} x} + \frac{n-2}{n-1} \int \frac{dx}{\sin^{n-2} x}$$

$$\int \frac{dx}{\cos^n x} = \frac{1}{n-1} \frac{\sin x}{\cos^{n-1} x} + \frac{n-2}{n-1} \int \frac{dx}{\cos^{n-2} x}$$

$$\int \cos^m x \sin^n x dx$$

$$= \frac{1}{m+n} \cos^{m-1} x \sin^{n+1} x + \frac{m-1}{m+n} \int \cos^{m-2} x \sin^n x dx$$

$$= -\frac{1}{m+n} \cos^{m+1} x \sin^{n-1} x + \frac{n-1}{m+1} \int \cos^m x \sin^{n-2} x dx$$

$$\int \frac{dx}{a+b \sin x} = \begin{cases} \frac{2}{\sqrt{a^2-b^2}} \arctan \frac{a \tan \frac{x}{2} + b}{\sqrt{a^2-b^2}} & (a^2 > b^2) \\ \frac{1}{\sqrt{b^2-a^2}} \ln \left| \frac{a \tan \frac{x}{2} + b - \sqrt{b^2-a^2}}{a \tan \frac{x}{2} + b + \sqrt{b^2-a^2}} \right| & (a^2 < b^2) \end{cases}$$

$$\int \frac{dx}{a+b \cos x} = \begin{cases} \frac{2}{a+b} \sqrt{\frac{a+b}{a-b}} \arctan \left(\sqrt{\frac{a-b}{a+b}} \tan \frac{x}{2} \right) & (a^2 > b^2) \\ \frac{1}{a+b} \sqrt{\frac{a+b}{a-b}} \ln \left| \frac{\tan \frac{x}{2} + \sqrt{\frac{a+b}{a-b}}}{\tan \frac{x}{2} - \sqrt{\frac{a+b}{a-b}}} \right| & (a^2 < b^2) \end{cases}$$

$$\int \frac{dx}{a^2 \cos^2 x + b^2 \sin^2 x} = \frac{1}{ab} \arctan \left(\frac{b}{a} \tan x \right)$$

$$\int \frac{dx}{a^2 \cos^2 x - b^2 \sin^2 x} = \frac{1}{2ab} \ln \left| \frac{b \tan x + a}{b \tan x - a} \right|$$

$$\int x \sin ax dx = \frac{1}{a^2} \sin ax - \frac{1}{a} x \cos ax$$

$$\int x^2 \sin ax dx = -\frac{1}{a^2} x^2 \cos ax + \frac{2}{a^2} x \sin ax + \frac{2}{a^3} \cos ax$$

$$\int x \cos ax dx = \frac{1}{a^2} \cos ax + \frac{1}{a} x \sin ax$$

$$\int x^2 \cos ax dx = \frac{1}{a^2} x^2 \sin ax + \frac{2}{a^2} x \cos ax - \frac{2}{a^3} \sin ax$$

反三角函数的积分 (其中 $a > 0$)

$$\int \arcsin \frac{x}{a} dx = x \arcsin \frac{x}{a} + \sqrt{a^2 - x^2} + C$$

$$\int x \arcsin \frac{x}{a} dx = \left(\frac{x^2}{2} - \frac{a^2}{4} \right) \arcsin \frac{x}{a} + \frac{x}{4} \sqrt{x^2 - x^2} + C$$

$$\int x^2 \arcsin \frac{x}{a} dx = \frac{x^3}{3} \arcsin \frac{x}{a} + \frac{1}{9} (x^2 + 2a^2) \sqrt{a^2 - x^2} + C$$

$$\int \arccos \frac{x}{a} dx = x \arccos \frac{x}{a} - \sqrt{a^2 - x^2} + C$$

$$\int x \arccos \frac{x}{a} dx = \left(\frac{x^2}{2} - \frac{a^2}{4} \right) \arccos \frac{x}{a} - \frac{x}{4} \sqrt{a^2 - x^2} + C$$

$$\int x^2 \arccos \frac{x}{a} dx = \frac{x^3}{3} \arccos \frac{x}{a} - \frac{1}{9} (x^2 + 2a^2) \sqrt{a^2 - x^2} + C$$

$$\int \arctan \frac{x}{a} dx = x \arctan \frac{x}{a} - \frac{a}{2} \ln(a^2 + x^2) + C$$

$$\int x \arctan \frac{x}{a} dx = \frac{1}{2} (a^2 + x^2) \arctan \frac{x}{a} - \frac{a^2}{2} x + C$$

$$\int x^2 \arctan \frac{x}{a} dx = \frac{x^3}{3} \arctan \frac{x}{a} - \frac{a}{6} x^2 + \frac{a^3}{6} \ln(a^2 + x^2) + C$$

指数函数的积分

$$\int a^x dx = \frac{1}{\ln a} a^x + C$$

$$\int e^{ax} dx = \frac{1}{a} e^{ax} + C$$

$$\int x e^{ax} dx = \frac{1}{a^2} (ax - 1) e^{ax} + C$$

$$\int x^n e^{ax} dx = \frac{1}{a} x^n e^{ax} - \frac{n}{a} \int x^{n-1} e^{ax} dx$$

$$\int x a^x dx = \frac{x}{\ln a} a^x - \frac{1}{(\ln a)^2} a^x + C$$

$$\int x^n a^x dx = \frac{1}{\ln a} x^n a^x - \frac{n}{\ln a} \int x^{n-1} a^x dx$$

$$\int e^{ax} \sin bxdx = \frac{1}{a^2+b^2} e^{ax} (a \sin bx - b \cos bx) + C$$

$$\int e^{ax} \cos bxdx = \frac{1}{a^2+b^2} e^{ax} (b \sin bx + a \cos bx) + C$$

$$\int e^{ax} \sin^n bxdx = \frac{1}{a^2+b^2n^2} e^{ax} \sin^{n-1} bx (a \sin bx - nb \cos bx) +$$

$$\frac{n(n-1)b^2}{a^2+b^2n^2} \int e^{ax} \sin^{n-2} bxdx$$

$$\int e^{ax} \cos^n bxdx = \frac{1}{a^2+b^2n^2} e^{ax} \cos^{n-1} bx (a \cos bx + nb \sin bx) +$$

$$\frac{n(n-1)b^2}{a^2+b^2n^2} \int e^{ax} \cos^{n-2} bxdx$$

对数函数的积分

$$\int \ln x dx = x \ln x - x + C$$

$$\int \frac{dx}{x \ln x} = \ln |\ln x| + C$$

$$\int x^n \ln x dx = \frac{1}{n+1} x^{n+1} \left(\ln x - \frac{1}{n+1} \right) + C$$

$$\int (\ln x)^n dx = x (\ln x)^n - n \int (\ln x)^{n-1} dx$$

$$\int x^m (\ln x)^n dx = \frac{1}{m+1} x^{m+1} (\ln x)^n - \frac{n}{m+1} \int x^m (\ln x)^{n-1} dx$$

3.16 Zeller 日期公式

```

1 // weekday=(id+1)%7;{Sun=0,Mon=1,...}   getId(1, 1, 1) = 0
2 int getId(int y, int m, int d) {
3   | if (m < 3) { y --; m += 12; }
4   | return 365 * y + y / 4 - y / 100 + y / 400 + (153 * (m -
   |   ↪ 3) + 2) / 5 + d - 307; }
5 // y<0: 统一加400的倍数年
6 auto date(int id) {
7   | int x=id+1789995, n, i, j, y, m, d;
8   | n = 4 * x / 146097; x -= (146097 * n + 3) / 4;
9   | i = (4000 * (x + 1)) / 1461001; x -= 1461 * i / 4 - 31;
10  | j = 80 * x / 2447; d = x - 2447 * j / 80; x = j / 11;
11  | m = j + 2 - 12 * x; y = 100 * (n - 49) + i + x;
12  | return make_tuple(y, m, d); }

```

3.17 有理数二分: Stern-Brocot 树, Farey 序列

```

1 def build(a, b, c, d, level = 1):
2   | x = a + c; y = b + d
3   | build(a, b, x, y, level + 1)
4   | build(x, y, c, d, level + 1)
5 build(0, 1, 1, 0) # 最简分数, Stern-Brocot
6 build(0, 1, 1, 1) # 最简真分数, Farey

```

3.18 Constant Table 常数表

Random primes generated at Mon Aug 31 22:29:30 2026

5e2 397 421 457 467 487 491 499 547 569 571 577 593 601 619

1e3 977 991 1021 1039 1049 1061 1063 1093 1097 1129 1163

3e4 27919 28163 29303 29401 29483 29833 30763 31607 32083

1e5 93257 93487 95093 95443 97729 98479 103769 105767 106591

1e6 971387 975977 977021 1006003 1008701 1030511 1056577

5e6 4786321 4809289 4842251 4855211 4981261 5160283 5230513

1e7 9332759 9641297 9901159 10276571 10279697 10512611 10549633

2e7 19406059 20004367 20163911 20532641 20704529 21019849

1e9 989704273 1013742689 1057797271 1063983257 1068461731

2e9 1860735307 2067028681 2108354531 2126623231 2132345497

NTT 976224257 r=3 (20) 985661441 r=3 (22) 998244353 r=3 (23)

1004535809 r=3 (21) 1007681537 r=3 (20) 1012924417 r=5 (21)

1045430273 r=3 (20) 1051721729 r=6 (20) 1053818881 r=7 (20)

4. 多项式与生成函数

4.1 FFT

```

1 using cp = complex<double>; const double PI = acos(-1.0);
2 vector<cp> omega[25]; // 单位根
3 // n 是 DFT 的最大长度, 例如如果最多有两个长为 m 的多项式相乘,
4 // 或者求逆的长度为 m, 那么 n 需要 >= 2m
5 void fft_init(int n) { // n = 2^k
6   | for (int k = 2, d = 0; k <= n; k *= 2, d++) {
7     | | omega[d].resize(k + 1);
8     | | for (int i = 0; i <= k; i++) // polar 是用模和辐角求复
   |   |   ↪ 数
9     | | | omega[d][i] = polar(1.0, 2 * PI * i / k); } }
10 void fft(cp* a, int n, int t) {
11   | for (int i = 1, j = 0; i < n - 1; i++) {
12     | | int k = n; do j ^= (k >= 1); while (j < k);
13     | | if (i < j) swap(a[i], a[j]); }
14   | for (int k = 1, d = 0; k < n; k *= 2, d++)
15     | | for (int i = 0; i < n; i += k * 2)
16       | | | for (int j = 0; j < k; j++) {
17         | | | | cp w = omega[d][t > 0 ? j : k * 2 - j];

```

```

18 | | | cp u = a[i + j], v = w * a[i + j + k];
19 | | | a[i + j] = u + v; a[i + j + k] = u - v; }
20 | if (t < 0) for (int i = 0; i < n; i++) a[i] /= n; }

```

4.2 NTT

```

1 vector<int> omega[25]; // 单位根
2 // n 是 DFT 的最大长度, 例如如果最多有两个长为 m 的多项式相乘,
3 // 或者求逆的长度为 m, 那么 n 需要 >= 2m
4 void ntt_init(int n) { // n = 2^k
5     for (int k = 2, d = 0; k <= n; k *= 2, d++) {
6         omega[d].resize(k + 1);
7         int wn = qpow(3, (p - 1) / k), tmp = 1;
8         for (int i = 0; i <= k; i++) { omega[d][i] = tmp;
9             tmp = (LL)tmp * wn % p; } } }
10 // 传入的数必须是 [0, p) 范围内, 不能有负的
11 // 否则把 d == 16 改成 d % 8 == 0 之类, 多取几次模
12 void ntt(int *c, int n, int tp) {
13     static ULL a[N];
14     for (int i = 0; i < n; i++) a[i] = c[i];
15     for (int i = 1, j = 0; i < n - 1; i++) {
16         int k = n; do j ^= (k >= 1); while (j < k);
17         if (i < j) swap(a[i], a[j]); }
18     for (int k = 1, d = 0; k <= n; k *= 2, d++) {
19         if (d == 16) for (int i = 0; i < n; i++) a[i] %= p;
20         for (int i = 0; i < n; i += k * 2)
21             for (int j = 0; j < k; j++) {
22                 int w = omega[d][tp > 0 ? j : k * 2 - j];
23                 ULL u = a[i + j], v = w * a[i + j + k] % p;
24                 a[i + j] = u + v;
25                 a[i + j + k] = u - v + p; } }
26     if (tp > 0) for (int i = 0; i < n; i++) c[i] = a[i] % p;
27     else { int inv = qpow(n, p - 2);
28         for (int i = 0; i < n; i++) c[i] = a[i] * inv % p; }

```

4.3 MTT 任意模数卷积

```

1 void dft(cp* a, cp* b, int n) { static cp c[MXN];
2     for (int i = 0; i < n; i++)
3         c[i] = cp(a[i].real(), b[i].real());
4     fft(c, n, 1);
5     for (int i = 0; i < n; i++) { int j = (n - i) & (n - 1);
6         a[i] = (c[i] + conj(c[j])) * 0.5;
7         b[i] = (c[i] - conj(c[j])) * -0.5i; } }
8 void idft(cp* a, cp* b, int n) { static cp c[MXN];
9     for (int i = 0; i < n; i++) c[i] = a[i] + 1i * b[i];
10    fft(c, n, -1);
11    for (int i = 0; i < n; i++) {
12        a[i] = c[i].real(); b[i] = c[i].imag(); } }
13 vector<int> multiply(const vector<int>& u,
14     const vector<int>& v, int mod) { // 任意模数卷积
15     static cp a[2][MXN], b[2][MXN], c[3][MXN];
16     int base = ceil(sqrt(mod));
17     int n = (int)u.size(), m = (int)v.size();
18     int fft_n = 1; while (fft_n < n + m - 1) fft_n *= 2;
19     for (int i = 0; i < 2; i++) {
20         fill(a[i], a[i] + fft_n, 0);
21         fill(b[i], b[i] + fft_n, 0); }
22     for (int i = 0; i < 3; i++)
23         fill(c[i], c[i] + fft_n, 0);
24     for (int i = 0; i < n; i++) { // 一定要取模!
25         a[0][i] = (u[i] % mod) % base;
26         a[1][i] = (u[i] % mod) / base; }
27     for (int i = 0; i < m; i++) { // 一定要取模!
28         b[0][i] = (v[i] % mod) % base;
29         b[1][i] = (v[i] % mod) / base; }
30     dft(a[0], a[1], fft_n); dft(b[0], b[1], fft_n);
31     for (int i = 0; i < fft_n; i++) {
32         c[0][i] = a[0][i] * b[0][i];
33         c[1][i] = a[0][i] * b[1][i] + a[1][i] * b[0][i];
34         c[2][i] = a[1][i] * b[1][i]; }
35     fft(c[1], fft_n, -1); idft(c[0], c[2], fft_n);
36     int base2 = base * base % mod;
37     vector<int> ans(n + m - 1);
38     for (int i = 0; i < n + m - 1; i++)
39         ans[i] = ((LL)(c[0][i].real() + 0.5) +
40             (LL)(c[1][i].real() + 0.5) % mod * base +
41             (LL)(c[2][i].real() + 0.5) % mod * base2) % mod;
42     return ans; }

```

4.4 多项式运算

4.4.1 多项式求逆 开根 ln exp

```

1 using poly = vector<int>;
2 poly poly_calc(const poly& u, const poly& v, // 长度要相同

```

```

3     function<int(int, int)> op) { // 返回长度是两倍
4     static int a[MXN], b[MXN], c[MXN];
5     int n = (int)u.size();
6     memcpy(a, u.data(), sizeof(int) * n);
7     fill(a + n, a + n * 2, 0);
8     memcpy(b, v.data(), sizeof(int) * n);
9     fill(b + n, b + n * 2, 0);
10    ntt(a, n * 2, 1); ntt(b, n * 2, 1);
11    for (int i = 0; i < n * 2; i++) c[i] = op(a[i], b[i]);
12    ntt(c, n * 2, -1); return poly(c, c + n * 2); }
13 poly poly_mul(const poly& u, const poly& v) { // 乘法
14     return poly_calc(u, v, [](int a, int b)
15         { return (LL)a * b % p; }); // 返回长度是两倍
16 poly poly_inv(const poly& a) { // 求逆, 返回长度不变
17     poly c(qpow(a[0], p - 2)); // 常数项一般都是 1
18     for (int k = 2; k <= (int)a.size(); k *= 2) {
19         c.resize(k); poly b(a.begin(), a.begin() + k);
20         c = poly_calc(b, c, [](int bi, int ci) {
21             return ((2 - (LL)bi * ci) % p + p) * ci % p; });
22         memset(c.data() + k, 0, sizeof(int) * k); }
23     c.resize(a.size()); return c; }
24 poly poly_sqrt(const poly& a) { // 开根, 返回长度不变
25     poly c[1]; // 常数项不是 1 的话要写二次剩余
26     for (int k = 2; k <= (int)a.size(); k *= 2) {
27         c.resize(k); poly b(a.begin(), a.begin() + k);
28         b = poly_mul(b, poly_inv(c));
29         for (int i = 0; i < k; i++) // inv_2 是 2 的逆元
30             c[i] = (LL)(c[i] + b[i]) * inv_2 % p; }
31     c.resize(a.size()); return c; }
32 poly poly_derivative(const poly& a) { poly c(a.size());
33     for (int i = 1; i < (int)a.size(); i++) // 求导
34         c[i - 1] = (LL)a[i] * i % p; return c; }
35 poly poly_integrate(const poly& a) { poly c(a.size());
36     for (int i = 1; i < (int)a.size(); i++) // 不定积分
37         c[i] = (LL)a[i - 1] * inv[i] % p; return c; }
38 poly poly_ln(const poly& a) { // ln, 常数项非 0, 返回长度不变
39     auto c = poly_mul(poly_derivative(a), poly_inv(a));
40     c.resize(a.size()); return poly_integrate(c); }
41 // exp, 常数项必须是 0, 返回长度不变
42 // 常数很大并且总代码很长, 一般可以改用分治 FFT
43 // 依据: 设  $G(x) = \exp F(x)$ , 则  $g_i = \frac{1}{i} \sum_{k=1}^{i-1} g_{i-k} k f_k$ 
44 poly poly_exp(const poly& a) { poly c[1];
45     for (int k = 2; k <= (int)a.size(); k *= 2) {
46         c.resize(k); auto b = poly_ln(c);
47         for (int i = 0; i < k; i++)
48             b[i] = (a[i] - b[i] + p) % p;
49         (++b[0]) %= p; c = poly_mul(b, c);
50         memset(c.data() + k, 0, sizeof(int) * k); }
51     c.resize(a.size()); return c; }

```

4.4.2 多项式除法 取模

需要抄求逆。

```

1 poly poly_auto_mul(poly a, poly b) { // 自动判断长度的乘法
2     int res_len = (int)a.size() + (int)b.size() - 1;
3     int ntt_n = 1; while (ntt_n < res_len) ntt_n *= 2;
4     a.resize(ntt_n); b.resize(ntt_n);
5     ntt(a.data(), ntt_n, 1); ntt(b.data(), ntt_n, 1);
6     for (int i = 0; i < ntt_n; i++)
7         a[i] = (LL)a[i] * b[i] % p;
8     ntt(a.data(), ntt_n, -1); a.resize(res_len); return a; }
9 // 多项式除法, a 和 b 长度可以任意
10 // 商的长度是 n - m + 1, 余数的长度是 m - 1
11 poly poly_div(const poly& a, const poly& b) {
12     int n = (int)a.size(), m = (int)b.size();
13     if (n < m) return {};
14     int ntt_n = 1; while (ntt_n < n - m + 1) ntt_n *= 2;
15     poly f(ntt_n), g(ntt_n);
16     for (int i = 0; i < n - m + 1; i++) f[i] = a[n - i - 1];
17     for (int i = 0; i < m && i < n - m + 1; i++)
18         g[i] = b[m - i - 1];
19     auto g_inv = poly_inv(g);
20     fill(g_inv.begin() + n - m + 1, g_inv.end(), 0);
21     auto c = poly_mul(f, g_inv); c.resize(n - m + 1);
22     reverse(c.begin(), c.end()); return c; }
23 // 多项式取模, a 和 b 长度可以任意, 返回 (余数, 商)
24 pair<poly, poly> poly_mod(const poly& a, const poly& b) {
25     int n = (int)a.size(), m = (int)b.size();
26     if (n < m) return {a, {}};
27     auto d = poly_div(a, b); auto c = poly_auto_mul(b, d);
28     poly r(m - 1);
29     for (int i = 0; i < m - 1; i++)
30         r[i] = (a[i] - c[i] + p) % p;
31     return {r, d}; }

```


4.4.3 多点求值

需要抄取模。

```

1 struct poly_eval { poly f; vector<int> x; // 函数和询问点
2 | vector<poly> gs; vector<int> ans; // gs 是预处理数组
3 | poly_eval(poly f, vector<int> x) : f(f), x(x) {}
4 | void pretreat(int l, int r, int o) { poly& g = gs[o];
5 | | if (l == r) { g = poly{p - x[l], 1}; return; }
6 | | int mid = (l + r) / 2; pretreat(l, mid, o * 2);
7 | | pretreat(mid + 1, r, o * 2 + 1);
8 | | if (o > 1)
9 | | | g = poly_auto_mul(gs[o * 2], gs[o * 2 + 1]); }
10 | void solve(int l, int r, int o, const poly& f) {
11 | | if (l == r) { ans[l] = f[0]; return; }
12 | | int mid = (l + r) / 2;
13 | | solve(l, mid, o * 2, poly_mod(f, gs[o * 2])).first();
14 | | solve(mid + 1, r, o * 2 + 1,
15 | | | poly_mod(f, gs[o * 2 + 1])).first(); }
16 | vector<int> operator() () { // 包装好的接口
17 | | int n = (int)f.size(), m = (int)x.size();
18 | | if (m <= n) x.resize(m = n + 1);
19 | | else if (n < m - 1) f.resize(n = m - 1);
20 | | int bit_ceil = 1; while (bit_ceil < m) bit_ceil *= 2;
21 | | ntt_init(bit_ceil * 2); // 注意这里 ntt_init 过了
22 | | gs.resize(2 * bit_ceil + 1); pretreat(0, m - 1, 1);
23 | | ans.resize(m); solve(0, m - 1, 1, f); return ans; } };

```

4.4.4 插值

牛顿插值 实现时可以用 k 次差分替代右边的式子，也可以卷积。

$$f(n) = \sum_{i=0}^k \binom{n}{i} r_i \iff r_i = \sum_{j=0}^i (-1)^{i-j} \binom{i}{j} f(j)$$

拉格朗日插值

$$f(x) = \sum_i f(x_i) \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

4.5 线性递推

 $O(k^2 \log n)$

```

1 // Complexity: init O(n^2 log) query O(n^2 log k)
2 // Requirement: const LOG const MOD
3 // Example: In: {1, 3} {2, 1} an = 2an-1 + an-2
4 // | | Out: calc(3) = 7
5 typedef vector<int> poly;
6 struct LinearRec {
7 | int n; poly first, trans; vector<poly> bin;
8 | poly add(poly &a, poly &b) {
9 | | poly res(n * 2 + 1, 0);
10 | | // 不要每次新开 vector, 可以使用矩阵乘法优化
11 | | for (int i = 0; i <= n; ++i) {
12 | | | for (int j = 0; j <= n; ++j) {
13 | | | | (res[i+j] += (LL)a[i] * b[j] % MOD) %= MOD;
14 | | | for (int i = 2 * n; i > n; --i) {
15 | | | | for (int j = 0; j < n; ++j) {
16 | | | | | (res[i-1-j] += (LL)res[i] * trans[j] % MOD) %= MOD;
17 | | | | res[i] = 0; }
18 | | | res.erase(res.begin() + n + 1, res.end());
19 | | | return res; }
20 | | LinearRec(poly &first, poly &trans): first(first),
21 | | | trans(trans) {
22 | | | n = first.size(); poly a(n + 1, 0); a[1] = 1;
23 | | | bin.push_back(a); for (int i = 1; i < LOG; ++i)
24 | | | | bin.push_back(add(bin[i - 1], bin[i - 1])); }
25 | | int calc(int k) { poly a(n + 1, 0); a[0] = 1;
26 | | | for (int i = 0; i < LOG; ++i)
27 | | | | if (k >> i & 1) a = add(a, bin[i]);
28 | | | int ret = 0; for (int i = 0; i < n; ++i)
29 | | | | if ((ret += (LL)a[i + 1] * first[i] % MOD) >= MOD)
30 | | | | | ret -= MOD;
31 | | | return ret; } };

```

 $O(k \log k \log n)$ - Bostan-Mori

```

1 int bostan_mori(int k, poly a, poly b) {
2 | int n = (int)a.size(); while (k) { poly c = b;
3 | | for (int i = 1; i < n; i += 2) c[i] = (p - c[i]) % p;
4 | | a = poly_mul(a, c); b = poly_mul(b, c);
5 | | for (int i = 0; i < n; i++) {
6 | | | a[i] = a[i * 2 + k % 2]; b[i] = b[i * 2]; }
7 | | a.resize(n); b.resize(n); k /= 2; }
8 | return (LL)a[0] * qpow(b[0], p - 2) % p; }
9 // a_n = \sum_{i=1}^m f_i a_{n-i} (f_0 = 0), f.size() = a.size() + 1

```

```

10 int linear_recurrance(int n, poly f, poly a) {
11 | int m = (int)a.size(), ntt_n = 1;
12 | while (ntt_n <= m) ntt_n *= 2; ntt_init(ntt_n * 2);
13 | f.resize(ntt_n); a.resize(ntt_n); f[0] = 1;
14 | for (int i = 1; i <= m; i++) f[i] = (p - f[i]) % p;
15 | a = poly_mul(a, f); a.resize(ntt_n);
16 | fill(a.data() + m, a.data() + ntt_n, 0);
17 | return bostan_mori(n, a, f); }

```

 $O(k \log k \log n)$ - 多项式取模需要抄前面的多项式取模。预处理 $O(k \log k \log n)$ ，固定 n 和系数只改变初始值的话，询问一次 $O(k)$ 。注意只询问一次的话不如 Bostan-Mori 快。

```

1 poly poly_power_mod(LL k, const poly& m) { // x^k mod m
2 | poly ans{1}, a{0, 1}; while (k) { if (k & 1)
3 | | | ans = poly_mod(poly_auto_mul(ans, a), m).first();
4 | | a = poly_mod(poly_auto_mul(a, a), m).first(); k /= 2; }
5 | return ans; }
6 // a_n = \sum_{i=1}^m c_i a_{n-i} (c_0 = 0)
7 struct linear_recurrence { poly f; // f 是预处理结果
8 | linear_recurrence(const poly& c, LL n) {
9 | | assert(c[0] == 0); // c[0] 是没有用的
10 | | int m = (int)c.size() - 1;
11 | | int ntt_n = 1; while (ntt_n < m * 2) ntt_n *= 2;
12 | | ntt_init(ntt_n); // 图省事就直接 ntt_init(1 << 18)
13 | | poly t(m + 1); t[m] = 1;
14 | | for (int i = 0; i < m; i++) t[i] = (p - c[m - i]) % p;
15 | | f = poly_power_mod(m, t); }
16 | int operator()(const vector<int>& a) { // 0~m-1 项初始值
17 | | assert(a.size() == f.size()); int ans = 0;
18 | | for (int i = 0; i < (int)a.size(); i++)
19 | | | ans = (ans + (LL)f[i] * a[i]) % p;
20 | | return ans; } };

```

4.6 Berlekamp-Massey 最小多项式

如果要求出一个次数为 k 的递推式，则输入的数列需要至少有 $2k$ 项。返回的内容满足 $\sum_{j=0}^{m-1} a_{i-j} c_j = 0$ ，并且 $c_0 = 1$ 。如果不加最后的处理的话，代码返回的结果会变成 $a_i = \sum_{j=0}^{m-1} c_{j-1} a_{i-j}$ ，有时候这样会方便接着跑递推，需要的话就删掉最后的处理。

```

1 vector<int> berlekamp_massey(const vector<int>& a) {
2 | vector<int> v, last; // v is the answer, 0-based
3 | int k = -1, delta = 0;
4 | for (int i = 0; i < (int)a.size(); i++) { int tmp = 0;
5 | | for (int j = 0; j < (int)v.size(); j++)
6 | | | tmp = (tmp + (LL)a[i - j - 1] * v[j]) % p;
7 | | if (a[i] == tmp) continue;
8 | | if (k < 0) { k = i; delta = (a[i] - tmp + p) % p;
9 | | | v = vector<int>(i + 1); continue; }
10 | | vector<int> u = v;
11 | | int val = (LL)(a[i] - tmp + p) *
12 | | | qpow(delta, p - 2) % p;
13 | | if (v.size() < last.size() + i - k)
14 | | | v.resize(last.size() + i - k);
15 | | | (v[i - k - 1] += val) %= p;
16 | | for (int j = 0; j < (int)last.size(); j++) {
17 | | | v[i - k + j] = (v[i - k + j] -
18 | | | | (LL)val * last[j]) % p;
19 | | | if (v[i - k + j] < 0) v[i - k + j] += p; }
20 | | if ((int)u.size() - i < (int)last.size() - k) {
21 | | | last = u; k = i; delta = a[i] - tmp;
22 | | | if (delta < 0) delta += p; } }
23 | for (auto &x : v) x = (p - x) % p;
24 | v.insert(v.begin(), 1); // 一般是需要最小递推式的，处理一下
25 | return v; } // \forall i, \sum_{j=0}^m a_{i-j} v_j = 0

```

如果要求向量序列的递推式，就把每位乘一个随机权值（或者说是乘一个随机行向量 v^T ）变成求数列递推式即可。如果是矩阵序列的话就随机一个行向量 u^T 和列向量 v ，然后把矩阵变成 $u^T A v$ 的数列。优化矩阵快速幂 DP 假设 f_i 有 n 维，先暴力求出 $f_{0 \sim 2n-1}$ ，然后跑 Berlekamp-Massey，最后调用快速线性递推即可。求矩阵最小多项式 矩阵 A 的最小多项式是次数最小的并且 $f(A) = 0$ 的多项式 f 。实际上最小多项式就是 $\{A^i\}$ 的最小递推式，所以直接调用 Berlekamp-Massey 就好了，显然它的次数不超过 n 。瓶颈在于求出 A^i ，实际上我们只要处理 $A^i v$ 就行了，每次对向量做递推。求稀疏矩阵的行列式 如果能求出特征多项式，则常数项乘上 $(-1)^n$ 就是行列式，但是最小多项式不一定是特征多项式。把 A 乘上一个随机对角阵 B ，则 AB 的最小多项式有很大概率就是特征多项式，最后再除掉 $\det B$ 就行了。

求稀疏矩阵的秩 设 A 是一个 $n \times m$ 的矩阵, 首先随机一个 $n \times n$ 的对角阵 P 和一个 $m \times m$ 的对角阵 Q , 然后计算 QAP^TQ 的最小多项式即可。实际上不用计算这个矩阵, 因为求最小多项式时要用它乘一个向量, 我们依次把这几个矩阵乘到向量里就行了。答案就是最小多项式除掉所有 x 因子后剩下的次数。

解稀疏方程组 $Ax = b$, 其中 A 是一个 $n \times n$ 的满秩稀疏矩阵, b 和 x 是 $1 \times n$ 的列向量, A, b 已知, 需要解出 x 。

做法: 显然 $x = A^{-1}b$. 如果我们能求出 $\{A^i b\} (i \geq 0)$ 的最小递推式 $\{r_0 \dots r_{m-1}\} (m \leq n)$, 那么就有结论

$$A^{-1}b = -\frac{1}{r_{m-1}} \sum_{i=0}^{m-2} A^i b r_{m-2-i}$$

因为 A 是稀疏矩阵, 直接按定义递推出 $b \dots A^{2n-1}b$ 即可。

```
1 vector<int> solve_sparse_equations(const vector<tuple<int,
2   int, int> > &A, const vector<int> &b) {
3   int n = (int)b.size(); // 0-based
4   vector<vector<int> > f({b});
5   for (int i = 1; i < 2 * n; i++) {
6     vector<int> v(n); auto &u = f.back();
7     for (auto [x, y, z] : A) // [x, y, value]
8       v[x] = (v[x] + (long long)u[y] * z) % p;
9     f.push_back(v);
10    vector<int> w(n); mt19937 gen;
11    for (auto &x : w)
12      x = uniform_int_distribution<int>(1, p - 1)(gen);
13    vector<int> a(2 * n);
14    for (int i = 0; i < 2 * n; i++)
15      for (int j = 0; j < n; j++)
16        a[i] = (a[i] + (long long)f[i][j] * w[j]) % p;
17    auto c = berlekamp_massey(a); int m = (int)c.size();
18    vector<int> ans(n);
19    for (int i = 0; i < m - 1; i++)
20      for (int j = 0; j < n; j++)
21        ans[j] = (ans[j] +
22          (long long)c[m - 2 - i] * f[i][j]) % p;
23    int inv = qpow(p - c[m - 1], p - 2);
24    for (int i = 0; i < n; i++)
25      ans[i] = (long long)ans[i] * inv % p;
26    return ans; }
```

4.7 FWT

```
1 /* |
2 And: (1,1) (1,-1) Or: (1,0) (0,1) Xor: (1,1) (0,5,0,5)
3 IFFT的矩阵时FWT的逆, 对于任意运算 ⊕, 满足FWT的矩阵需要:
4 C[i][j] × C[i][k] = C[i][j ⊕ k]
5 对于不存在FWT矩阵的运算: 通过映射01变成另外一个可行的运算。*/
6 const LL XOR[2][2] = {{1, 1}, {1, M-1}};
7 const LL i2 = (M+1)/2, iXOR[2][2] = {{i2, i2}, {i2, M-i2}};
8 void FWT(LL f[], const LL C[2][2], int n) {
9   for (int t = 1; t < n; t <= 1) {
10    for (int l = 0; l < n; l += t + t) {
11      for (int i = 0; i < t; i++) {
12        LL x = f[l + i], y = f[l + t + i];
13        f[l + i] = (C[0][0] * x + C[0][1] * y) % M;
14        f[l + t + i] = (C[1][0] * x + C[1][1] * y) % M;
15      } } }
```

4.8 K进制 FWT

```
1 // n : power of k, omega[i] : (primitive kth root) ^ i
2 void fwt(int* a, int k, int type) {
3   static int tmp[K];
4   for (int i = 1; i < n; i *= k)
5     for (int j = 0, len = i * k; j < n; j += len)
6       for (int low = 0; low < i; low++) {
7         for (int t = 0; t < k; t++)
8           tmp[t] = a[j + t * i + low];
9         for (int t = 0; t < k; t++){
10          int x = j + t * i + low;
11          a[x] = 0;
12          for (int y = 0; y < k; y++)
13            a[x] = (a[x] + 1ll * tmp[y] * omega[(k + type) *
14              t * y % k] % MOD);
15        } }
16   if (type == -1)
17     for (int i = 0, invn = inv(n); i < n; i++)
18       a[i] = (1ll * a[i] * invn % MOD); }
```

5. 线性代数

5.1 Simplex 单纯形

```
1 const LD eps = 1e-9, INF = 1e9; const int N = 105;
2 namespace Simplex {
3   int n, m, id[N], tp[N]; LD a[N][N];
4   void pivot(int r, int c) {
5     swap(id[r + n], id[c]);
6     LD t = -a[r][c]; a[r][c] = -1;
7     for (int i = 0; i <= n; i++) a[r][i] /= t;
8     for (int i = 0; i <= m; i++) if (a[i][c] && r != i) {
9       t = a[i][c]; a[i][c] = 0;
10      for (int j = 0; j <= n; j++) a[i][j] += t*a[r][j];
11    }
12   bool solve() {
13     for (int i = 1; i <= n; i++) id[i] = i;
14     for ( ; ; ) {
15       int i = 0, j = 0; LD w = -eps;
16       for (int k = 1; k <= m; k++)
17         if (a[k][0] < w || (a[k][0] < -eps && rand() & 1))
18           if (!i) break;
19       for (int k = 1; k <= n; k++)
20         if (a[i][k] > eps) {j = k; break;}
21       if (!j) { printf("Infeasible"); return 0; }
22       pivot(i, j);
23     }
24     for ( ; ; ) {
25       int i = 0, j = 0; LD w = eps, t;
26       for (int k = 1; k <= n; k++)
27         if (a[0][k] > w) w = a[0][j = k];
28       if (!j) break;
29       for (int k = 1; k <= m; k++)
30         if (a[k][j] < -eps && (t = -a[k][0]/a[k][j]) < w)
31           if (!i) { printf("Unbounded"); return 0; }
32       pivot(i, j);
33     }
34     return 1;
35   } LD ans() {return a[0][0];}
36   void output() {
37     for (int i = n + 1; i <= n + m; i++) tp[id[i]] = i - n;
38     for (int i = 1; i <= n; i++) printf("%.9lf ", tp[i] ?
39       a[tp[i]][0] : 0);
40   } using namespace Simplex;
41   int main() { int K; cin >> n >> m >> K;
42     for (int i = 1; i <= n; i++) {LD x; cin >> x; a[0][i] = x;}
43     for (int i = 1; i <= m; i++) {LD x;
44       for (int j = 1; j <= n; j++) cin >> x, a[i][j] = -x;
45       cin >> x; a[i][0] = x;}
46     if (solve()) { printf("%.9lf\n", (LD)ans()); if (K)
47       output(); }
```

5.2 高斯消元最小范数解

```
1 typedef vector<LD> vec; /* sum a[i][0..d] = 0 */
2 pair<vec, vector<vec>> gauss(vector<vec> &a, int n, int d) {
3   vector<int> pivot(d, -1);
4   for (int i = 0, o = 0; i < d; i++) {
5     int j = o; while (j < n && abs(a[j][i]) < eps) j++;
6     if (j == n) continue;
7     swap(a[j], a[o]); LD w = a[o][i];
8     for (int k = 0; k <= d; k++) a[o][k] /= w;
9     for (int x = 0; x < n; x++)
10       if (x != o && abs(a[x][i]) > eps) {
11         w = a[x][i];
12         for (int k = 0; k <= d; k++)
13           a[x][k] -= a[o][k] * w;
14       }
15     pivot[i] = o++;
16   } vec x0(d); vector<vec> t; int free = 0;
17   for (int i = 0; i < d; i++)
18     if (pivot[i] != -1) x0[i] = -a[pivot[i]][d];
19     else free++;
20   for (int i = 0; i < d; i++) if (pivot[i] == -1) {
21     vec x(d); x[i] = -1;
22     for (int j = 0; j < d; j++)
23       if (pivot[j] != -1) x[j] = a[pivot[j]][i];
24     t.push_back(x);
25   } if (t.size()) {
26     vector<vec> f;
27     for (int u = 0; u < free; u++) {
28       vec x(free + 1);
29       for (int i = 0; i < free; i++)
30         for (int j = 0; j < d; j++)
```

```

30 | | | x[i] += t[u][j] * t[i][j];
31 | | | for (int j = 0; j < d; j++)
32 | | | x[free] += t[u][j] * x0[j];
33 | | | f.push_back(x);
34 | | }
35 | | auto [k, tt] = gauss(f, free, free);
36 | | assert (tt.size() == 0);
37 | | for (int x = 0; x < free; x++)
38 | | | for (int i = 0; i < d; i++)
39 | | | x0[i] += k[x] * t[x][i];
40 | } return {x0, t}; }

```

6. 数据结构

6.1 bitset

```

1 template<int len = 1> // 动态size的bitset
2 void solve(int x) {
3 | if(siz[x] > len) return (void) solve<min(len*2,
  ↳ MAXN)>(x);
4 | bitset<len> dp; }

```

6.2 FHQ treap

```

1 struct FHQ{
2 | int val[N], dat[N], siz[N], ls[N], rs[N], tot = 0, rt;
3 | void push_up(int x) {siz[x] = siz[ls[x]] + siz[rs[x]] + 1;}
4 | int merge(int x, int y){
5 | | if(!x || !y) return x+y;
6 | | if(dat[x] < dat[y]) {push_down(x); rs[x] = merge(rs[x],
  ↳ y); push_up(x); return x;}
7 | | else {push_down(y); ls[y] = merge(x, ls[y]); push_up(y);
  ↳ return y;} }
8 | void splitVal(int p, int k, int& x, int& y){
9 | | if(!p) {x = y = 0; return;}
10 | | push_down(p);
11 | | if(val[p] <= k) {x = p; splitVal(rs[x], k, rs[x], y);}
12 | | else {y = p; splitVal(ls[y], k, x, ls[y]);}
13 | | push_up(p);
14 | }
15 | void splitSiz(int p, int k, int& x, int& y){
16 | | if(!p) {x = y = 0; return;}
17 | | push_down(p);
18 | | if(siz[ls[p]] < k) {x = p; splitSiz(rs[p],
  ↳ k-siz[ls[p]]-1, rs[x], y);}
19 | | else {y = p; splitSiz(ls[p], k, x, ls[y]);}
20 | | push_up(p);
21 | }
22 | int new_node(int v){
23 | | siz[++tot] = 1; val[tot] = v;
24 | | dat[tot] = rd(); return tot;}
25 | void ins(int v){
26 | | int x, y; splitVal(rt, v, x, y);
27 | | rt = merge(merge(x, new_node(v)), y); }
28 | void del(int v){
29 | | int x, y, z;
30 | | splitVal(rt, v, x, z), splitVal(x, v-1, x, y);
31 | | y = merge(ls[y], rs[y]); rt = merge(merge(x, y), z); }
32 | int rk(int v){
33 | | int x, y; splitVal(rt, v-1, x, y);
34 | | int t = siz[x] + 1; rt = merge(x, y); return t; }
35 | int kth(int p, int k){
36 | | if(!p) return 0;
37 | | while(true){
38 | | | if(siz[ls[p]] >= k) p = ls[p];
39 | | | else if(siz[ls[p]] + 1 == k) return val[p];
40 | | | else k -= siz[ls[p]] + 1, p = rs[p]; } }
41 | int pre(int v){
42 | | int x, y; splitVal(rt, v-1, x, y);
43 | | int t = kth(x, siz[x]); rt = merge(x, y); return t; }
44 | int nxt(int v){
45 | | int x, y; splitVal(rt, v, x, y);
46 | | int t = kth(y, 1); rt = merge(x, y); return t; }
47 | int s[MAXN]; // 线性建立FHQ
48 | int build(int n){
49 | | int top = 0;
50 | | for(int i=1; i<=n; i++){
51 | | | int tmp = new_node(a[i]), last = 0;
52 | | | while(top && dat[s[top]] > dat[tmp])
53 | | | | push_up(s[top]), last = s[top--];
54 | | | if(top) rs[s[top]] = tmp; ls[tmp] = last, s[++top] =
  ↳ tmp;
55 | | }
56 | | while(top) push_up(s[top--]); return s[1]; }

```

```

57 } tr;

```

6.3 RangeBIT

```

1 struct BIT{
2 | #define lowbit(x) (x & (-x))
3 | ll d[MAXN], d1[MAXN];
4 | void upd(int x, ll k) {
5 | | for (int pos = x; x <= n; x += lowbit(x))
6 | | | d[x] += k, d1[x] += (pos - 1) * k;
7 | | }
8 | void upd(int l, int r, ll k) {upd(l, k), upd(r+1, -k);}
9 | ll qry(int x) {
10 | | ll ret = 0, pos = x;
11 | | for (int pos = x; x; x -= lowbit(x))
12 | | | ret += d[x] * pos - d1[x];
13 | | return ret;
14 | }
15 | ll qry(int l, int r) {return qry(r) - qry(l-1);}
16 };

```

6.4 李超线段树

```

1 namespace Li{ // 默认取max 可持久化李超记得从x转移而非p
2 | int tag[MAXN];
3 | double k[MAXN], b[MAXN];
4 | const double EPS = 1e-10;
5 | double calc(int i, double x) {return x * k[i] + b[i];}
6 | void update(int nl, int nr, int l, int r, int p, int x){
7 | | if(l == r) {if(calc(tag[p], l) < calc(x, l)) tag[p] =
  ↳ x; return;}
8 | | int mid = (l+r)>>1;
9 | | if(nl <= l && r <= nr){
10 | | | if(!tag[p]) {tag[p] = x; return;}
11 | | | double l1 = calc(tag[p], l), l2 = calc(x, l), r1 =
  ↳ calc(tag[p], r), r2 = calc(x, r);
12 | | | if(abs(l1 - l2) < EPS && abs(r1 - r2) < EPS)
  ↳ return;
13 | | | if(l1 > l2 && r1 > r2) return;
14 | | | if(l2 > l1 && r2 > r1) tag[p] = x;
15 | | | if(calc(tag[p], mid) < calc(x, mid)) swap(x,
  ↳ tag[p]);
16 | | | if(k[x] > k[tag[p]]) update(nl, nr, mid+1, r, rs,
  ↳ x);
17 | | | if(k[x] < k[tag[p]]) update(nl, nr, l, mid, ls, x);
  ↳ return;
18 | | }
19 | | if(nl <= mid) update(nl, nr, l, mid, ls, x);
20 | | if(nr > mid) update(nl, nr, mid+1, r, rs, x);
21 | }
22 | double query(int pos, int l, int r, int p){
23 | | if(l == r) return calc(tag[p], pos);
24 | | int mid = (l+r)>>1;
25 | | if(pos <= mid) return max(calc(tag[p], pos),
  ↳ query(pos, l, mid, ls));
26 | | else return max(calc(tag[p], pos), query(pos, mid+1,
  ↳ r, rs));
27 | }
28 | }
29 }

```

6.5 线段树分裂

```

1 void split(int& x, int& y, int nl, int nr, int l, int r){
2 | if(nl <= l && r <= nr) {y = x, x = 0; return;}
3 | y = new_node();
4 | int mid = (l+r)>>1;
5 | if(nl <= mid) split(ls[x], ls[y], nl, nr, l, mid);
6 | if(nr > mid) split(rs[x], rs[y], nl, nr, mid+1, r);
7 | push_up(x), push_up(y);
8 | }

```

6.6 左偏树

```

1 namespace Leftist{
2 | int a[MAXN], ls[MAXN], rs[MAXN], dis[MAXN], fa[MAXN];
3 | bool vis[MAXN]; // dis[0] = -1;
4 | void init() {for(int i=1; i<=n; i++) fa[i] = i;}
5 | int find(int x) {return fa[x] == x ? x : fa[x] =
  ↳ find(fa[x]);}
6 | int _merge(int x, int y){
7 | | if(!x || !y) return x + y;
8 | | if(a[y] < a[x]) swap(x, y);
9 | | rs[x] = _merge(rs[x], y);

```

```

10 | | if(dis[ls[x]] < dis[rs[x]]) swap(ls[x], rs[x]);
11 | | dis[x] = dis[rs[x]] + 1;
12 | | return x;
13 | }
14 | void merge(int x, int y){
15 | | if(vis[x] || vis[y]) return;
16 | | x = find(x), y = find(y);
17 | | if(x != y) fa[x] = fa[y] = _merge(x, y);
18 | | }
19 | int del(int x){
20 | | if(vis[x]) return -1; // deleted flag
21 | | x = find(x); vis[x] = true;
22 | | int ret = a[x].val;
23 | | fa[ls[x]] = fa[rs[x]] = fa[x] = _merge(ls[x], rs[x]);
24 | | ls[x] = rs[x] = dis[x] = 0;
25 | | return ret;
26 | }
27 | }

```

6.7 非递归线段树

6.7.1 区间加，区间求最大值

```

1 void update(int l, int r, int d) {
2   for (l += M-1, r += M+1; l^r^1; l >>= 1, r >>= 1) {
3     if (l < M) {
4       t[l] = max(t[l*2], t[l*2+1]) + mark[l];
5       t[r] = max(t[r*2], t[r*2+1]) + mark[r];
6       if (~l & 1) { t[l^1] += d; mark[l^1] += d; }
7       if (r & 1) { t[r^1] += d; mark[r^1] += d; }
8     } for (; l; l >>= 1, r >>= 1)
9       if (l < M) t[l] = max(t[l*2], t[l*2+1]) + mark[l],
10         t[r] = max(t[r*2], t[r*2+1]) + mark[r];
11 int query(int l, int r) {
12   int maxl = -INF, maxr = -INF;
13   for (l += M-1, r += M+1; l^r^1; l >>= 1, r >>= 1) {
14     maxl += mark[l]; maxr += mark[r];
15     if (~l & 1) maxl = max(maxl, t[l^1]);
16     if (r & 1) maxr = max(maxr, t[r^1]);
17   } while (l) { maxl += mark[l]; maxr += mark[r];
18     l >>= 1; r >>= 1; }
19   return max(maxl, maxr); }

```

6.8 LCT 动态树

```

1 namespace LCT{
2   #define ls(x) c[x][0]
3   #define rs(x) c[x][1]
4   int val[N], ans[N], c[N][2], f[N], rev[N];
5   void push_up(int x){
6     ans[x] = max({ans[ls(x)], ans[rs(x)], val[x]}); }
7   void opt(int x) {swap(ls(x), rs(x)), rev[x] ^= 1;}
8   void push_down(int x) {
9     if(rev[x]) opt(ls(x)), opt(rs(x)), rev[x] = 0;}
10  bool nrt(int x) {return ls(f[x]) == x || rs(f[x]) == x;}
11  bool get(int x) {return x == rs(f[x]);}
12  void rot(int x){
13    int y = f[x], z = f[y], t = get(x), w = c[x][t^1];
14    if(nrt(y)) c[z][get(y)] = x;
15    c[x][t^1] = y, c[y][t] = w;
16    if(w) {f[w] = y; f[y] = x, f[x] = z; push_up(y); }
17  void push_all(int x) {if(nrt(x)) push_all(f[x]);
18    <math>\hookrightarrow</math> push_down(x);}
19  void splay(int x){
20    push_all(x);
21    while(nrt(x)){
22      int y = f[x];
23      if(nrt(y)) rot(get(x) == get(y) ? y : x);
24      rot(x);
25    } push_up(x); }
26  void access(int x) {for(int y=0; y=x, x=f[x]) splay(x),
27    <math>\hookrightarrow</math> rs(x) = y, push_up(x);}
28  void mkrt(int x) {access(x); splay(x); opt(x);}
29  int findrt(int x){
30    access(x), splay(x);
31    while(ls(x)) push_down(x), x = ls(x);
32    splay(x); return x; }
33  void split(int x, int y) {mkrt(x), access(y), splay(y);}
34  bool reach(int x, int y) {mkrt(x); return findrt(y) == x;}
35  void link(int x, int y) {if(!reach(x, y)) f[x] = y;}
36  void cut(int x, int y) {if(reach(x, y) && f[y] == x &&
37    <math>\hookrightarrow</math> ls(y)) f[y] = rs(x) = 0;}
38  // 维护子树信息树的直径，F[x]:splay子树深度最小的点往下最大深度
39  void push_up(int x){
40    siz[x] = siz[ls(x)] + siz[rs(x)] + 1;
41    F[x] = max(F[ls(x)], F[rs(x)] + siz[ls(x)] + 1);

```

```

39 | G[x] = max(G[rs(x)], G[ls(x)] + siz[rs(x)] + 1);
40 | if(s[x].size())
41 | | F[x] = max(F[x], *(--s[x].end()) + 1 + siz[ls(x)]),
42 | | G[x] = max(G[x], *(--s[x].end()) + 1 + siz[rs(x)]); }
43 void opt(int x) {swap(ls(x), rs(x)), swap(F[x], G[x]),
44   <math>\hookrightarrow</math> rev[x] ^= 1;}
45 void access(int x) {
46   for(int y=0; y=x, x=f[x]) {
47     splay(x);
48     if(rs(x)) s[x].insert(F[rs(x)]);
49     if(rs(x) = y) s[x].erase(s[x].find(F[y]));
50     push_up(x); } }
51 void link(int x, int y) {mkrt(x); if(findrt(y) != x) mkrt(y),
52   <math>\hookrightarrow</math> f[x] = y, s[y].insert(F[x]);}
53 #undef ls
54 #undef rs
55 }

```

6.9 Hash Table

```

1 template <class T, int P = 314159/*,451411,1141109,2119969*/>
2 struct hashmap {
3   ULL id[P]; T val[P];
4   int R[P]; // del: few clears
5   hashmap() {memset(id, -1, sizeof id);}
6   T get(const ULL &x) const {
7     for (int i = int(x % P), j = 1; ~id[i]; i = (i + j) % P,
8       <math>\hookrightarrow</math> j = (j + 2) % P /*unroll if needed*/) {
9       if (id[i] == x) return val[i]; }
10    return 0; }
11   T& operator [] (const ULL &x) {
12     for (int i = int(x % P), j = 1; ; i = (i + j) % P, j
13       <math>\hookrightarrow</math> j = (j + 2) % P) {
14       if (id[i] == x) return val[i];
15       else if (id[i] == -1llu) {
16         id[i] = x;
17         R[++R[0]] = i; // del: few clears
18         return val[i]; } } }
19   void clear() { // del: few clears
20     for (int &x = R[0]; x; id[R[x]] = -1, val[R[x]] = 0, --x);
21     void fullclear() {memset(id, -1, sizeof id); R[0] = 0; } }

```

6.10 基数排序

```

1 const int SZ = 1 << 8; // almost always fit in L1 cache
2 void SORT(int a[], int c[], int n, int w) {
3   for(int i=0; i<SZ; i++) b[i] = 0;
4   for(int i=1; i<=n; i++) b[(a[i]>>w) & (SZ-1)]++;
5   for(int i=1; i<SZ; i++) b[i] += b[i-1];
6   for(int i=n; i; i--) c[b[(a[i]>>w) & (SZ-1)]--] = a[i];}
7 void Sort(int *a, int n){
8   SORT(a, c, n, 0); SORT(c, a, n, 8);
9   SORT(a, c, n, 16); SORT(c, a, n, 24); }

```

7. 树上问题

7.1 虚树

```

1 bool cmp(int x, int y) {return id[x] < id[y];}
2 void build(vector<int> v){
3   sort(v.begin(), v.end(), cmp);
4   int top = 0;
5   s[top] = 1; vt::e[1].clear();
6   for(auto x : v){
7     if(x == 1) continue;
8     int l = lca(x, s[top]);
9     if(l != s[top]){
10      while(id[l] < id[s[top-1]]) vt::push(s[top-1],
11        <math>\hookrightarrow</math> s[top]), top--;
12      if(l != s[top-1]) vt::e[1].clear(), vt::push(l,
13        <math>\hookrightarrow</math> s[top]), s[top] = l;
14      else vt::push(l, s[top--]);
15    }
16    s[++top] = x; vt::e[x].clear();
17  }
18  for(int i=1; i<top; i++) vt::push(s[i], s[i+1]);

```

7.2 树哈希

```

1 ULL get(vector <ULL> ha) {
2   sort(ha.begin(), ha.end());
3   ULL ret = 0xdeadbeef;

```

```

4 | for (auto i : ha) {
5 | | ret = ret * P + i, ret ^= ret << 17;
6 | } return ret * 997; }

```

8. 计算几何

8.1 声明与宏

```

1 #define cp const P &
2 #define cl const L &
3 #define cc const C &
4 #define vp vector<P>
5 #define cvp const vector<P> &
6 #define D double
7 #define LD long double
8
9 mt19937 rnd(time(nullptr));
10 const LD eps = 1e-12L;
11 const LD pi = acos(-1.0L);
12 const LD INF = 1e18L;
13
14 int sgn(LD x) {
15 | return x > eps ? 1 : (x < -eps ? -1 : 0);
16 }

```

8.2 点与向量

```

1 struct P {
2 | LD x, y;
3
4 | P(LD x_ = 0, LD y_ = 0) : x(x_), y(y_) {}
5 | P(cp a) : x(a.x), y(a.y) {}
6 | P &operator=(cp a) { x = a.x, y = a.y; return *this; }
7
8 | P rot(LD t) const { // 逆时针旋转 t 弧度
9 | | LD c = cosl(t), s = sinl(t);
10 | | return {x * c - y * s, x * s + y * c};
11 | }
12 | P rot90() const { return {-y, x}; }
13 | P _rot90() const { return {y, -x}; }
14 | LD len() const { return sqrtl(x * x + y * y); }
15 | LD len2() const { return x * x + y * y; }
16 | P unit() const { // 零向量的单位向量约定为零向量
17 | | LD d = len();
18 | | return sgn(d) ? P{x / d, y / d} : P{};
19 | }
20
21 | void read() { if (scanf("%Lf%Lf", &x, &y) != 2) x = y = 0; }
22 | void print() const { printf("%.9Lf, %.9Lf", x, y); }
23 };
24
25 bool operator<(cp a, cp b) { return a.x == b.x ? a.y < b.y :
26 | a.x < b.x; }
27 bool operator>(cp a, cp b) { return b < a; }
28 bool operator==(cp a, cp b) { return !sgn(a.x - b.x) &&
29 | !sgn(a.y - b.y); }
30 bool operator!=(cp a, cp b) { return !(a == b); }
31 P operator+(cp a, cp b) { return {a.x + b.x, a.y + b.y}; }
32 P operator-(cp a, cp b) { return {a.x - b.x, a.y - b.y}; }
33 P operator*(cp a, LD k) { return {a.x * k, a.y * k}; }
34 P operator*(LD k, cp a) { return a * k; }
35 P operator/(cp a, LD k) { return {a.x / k, a.y / k}; }
36 LD operator*(cp a, cp b) { return a.x * b.x + a.y * b.y; }
37 | // 点积
38 LD operator^(cp a, cp b) { return a.x * b.y - a.y * b.x; }
39 | // 叉积
40
41 LD rad(cp a, cp b) { // 夹角 [0, pi]
42 | LD d = a.len() * b.len();
43 | if (!sgn(d)) return 0;
44 | return acosl(clamp((a * b) / d, -1.0L, 1.0L));
45 }
46
47 bool left(cp a, cp b) { return sgn(a ^ b) > 0; }
48 int turn(cp a, cp b, cp c) { return sgn((b - a) ^ (c - a)); }
49 | //
50 int half(cp a) { return a.y > 0 || (a.y == 0 && a.x > 0); }

```

8.3 线

```

1 struct L {
2 | P s, t;
3

```

```

4 | L(cp s_ = P(), cp t_ = P()) : s(s_), t(t_) {}
5 | L(cl a) : s(a.s), t(a.t) {}
6 | L &operator=(cl a) { s = a.s, t = a.t; return *this; }
7 | void read() { s.read(), t.read(); }
8 };
9
10 bool point_on_line(cp a, cl l) {
11 | return !sgn((a - l.s) ^ (l.t - l.s));
12 }
13 bool point_on_segment(cp a, cl l) {
14 | return point_on_line(a, l) && sgn((a - l.s) * (a - l.t))
15 | <= 0;
16 }
17 bool two_side(cp a, cp b, cl l) {
18 | return sgn((a - l.s) ^ (l.t - l.s))
19 | * sgn((b - l.s) ^ (l.t - l.s)) < 0;
20 }
21 bool intersection_judge_strict(cl a, cl b) {
22 | return two_side(a.s, a.t, b) && two_side(b.s, b.t, a);
23 }
24 bool intersection_judge(cl a, cl b) {
25 | return point_on_segment(a.s, b) || point_on_segment(a.t,
26 | b)
27 | || point_on_segment(b.s, a) || point_on_segment(b.t,
28 | a)
29 | || intersection_judge_strict(a, b);
30 }
31 P ll_intersection(cl a, cl b) { // 两直线需不平行
32 | LD s1 = (a.t - a.s) ^ (b.s - a.s);
33 | LD s2 = (a.t - a.s) ^ (b.t - a.s);
34 | return (b.s * s2 - b.t * s1) / (s2 - s1);
35 }
36 bool point_on_ray(cp a, cl b) {
37 | return point_on_line(a, b) && sgn((a - b.s) * (b.t -
38 | b.s)) >= 0;
39 }
40 bool ray_intersection_judge(cl a, cl b) {
41 | if (!sgn((a.t - a.s) ^ (b.t - b.s))) {
42 | | if (!point_on_line(b.s, a)) return false;
43 | | return point_on_ray(b.s, a) || point_on_ray(a.s, b);
44 | }
45 | P p = ll_intersection(a, b);
46 | return point_on_ray(p, a) && point_on_ray(p, b);
47 }
48 LD point_to_line(cp a, cl b) {
49 | if (b.s == b.t) return (a - b.s).len();
50 | return fabsl((b.t - b.s) ^ (a - b.s)) / (b.t -
51 | b.s).len();
52 }
53 P project_to_line(cp a, cl b) {
54 | P d = b.t - b.s;
55 | if (b.s == b.t) return b.s;
56 | return b.s + d * ((a - b.s) * d / d.len2());
57 }
58 LD point_to_segment(cp a, cl b) {
59 | if (b.s == b.t) return (a - b.s).len();
60 | if (sgn((a - b.s) * (b.t - b.s)) >= 0
61 | | && sgn((a - b.t) * (b.s - b.t)) >= 0)
62 | | return point_to_line(a, b);
63 | return min((a - b.s).len(), (a - b.t).len());
64 }
65 LD segment_to_segment(cl a, cl b) {
66 | if (intersection_judge(a, b)) return 0;
67 | return min({point_to_line(a.s, b),
68 | point_to_line(a.t, b),
69 | point_to_line(b.s, a), point_to_line(b.t, a)});

```

8.4 圆

```

1 struct C {
2 | P c;
3 | LD r;
4
5 | C(cp c_ = P(), LD r_ = 0) : c(c_), r(r_) {}
6 | C(cc a) : c(a.c), r(a.r) {}
7 | C &operator=(cc a) { c = a.c, r = a.r; return *this; }
8 | C(cp a, cp b) : c((a + b) / 2), r((a - c).len()) {}
9 | C(cp x, cp y, cp z) { // 三点不能共线
10 | | P p(y - x), q(z - x);
11 | | P s(p.len2() / 2, q.len2() / 2);
12 | | LD d = p ^ q;
13 | | c = x + P(s ^ P(p.y, q.y), P(p.x, q.x) ^ s) / d;
14 | | r = (c - x).len();

```



```

15 | }
16 | };
17
18 bool in_circle(cp a, cc b) { return sgn((b.c - a).len() -
    ↪ b.r) <= 0; }
19
20 C min_circle(vp p) { // 随机增量, O(n) 期望
21 | if (p.empty()) return C();
22 | shuffle(p.begin(), p.end(), rnd);
23 | C ret(p[0], 0);
24 | for (int i = 1; i < (int)p.size(); ++i) if
    ↪ (!in_circle(p[i], ret)) {
25 | | ret = C(p[i], 0);
26 | | for (int j = 0; j < i; ++j) if (!in_circle(p[j], ret))
    ↪ {
27 | | | ret = C(p[i], p[j]);
28 | | | for (int k = 0; k < j; ++k) if (!in_circle(p[k],
    ↪ ret))
29 | | | | ret = C(p[i], p[j], p[k]);
30 | | }
31 | }
32 | return ret;
33 | }
34
35 vp lc_intersection(cl l, cc c) { // 直线与圆交点
36 | LD d = point_to_line(c.c, l);
37 | if (sgn(d - c.r) > 0) return {};
38 | P p = project_to_line(c.c, l);
39 | LD x = sqrtl(max(0.0L, c.r * c.r - d * d));
40 | if (!sgn(x)) return {p};
41 | P v = (l.t - l.s).unit() * x;
42 | return {p - v, p + v};
43 | }
44
45 LD cc_intersection_area(cc a, cc b) {
46 | LD d = (a.c - b.c).len();
47 | if (sgn(d - a.r - b.r) >= 0) return 0;
48 | if (sgn(d - fabsl(a.r - b.r)) <= 0) {
49 | | LD r = min(a.r, b.r);
50 | | return pi * r * r;
51 | }
52 | LD x = (d * d + a.r * a.r - b.r * b.r) / (2 * d);
53 | LD t1 = acosl(clamp(x / a.r, -1.0L, 1.0L));
54 | LD t2 = acosl(clamp((d - x) / b.r, -1.0L, 1.0L));
55 | return a.r * a.r * t1 + b.r * b.r * t2 - d * a.r *
    ↪ sinl(t1);
56 | }
57
58 // 返回交点个数; -1 表示两圆重合, 有无穷多个交点。
59 int cc_intersection_count(cc a, cc b) {
60 | LD d = (a.c - b.c).len();
61 | if (a.c == b.c) {
62 | | if (sgn(a.r - b.r)) return 0;
63 | | return !sgn(a.r) ? 1 : -1;
64 | }
65 | if (sgn(d - a.r - b.r) > 0
66 | | || sgn(d - fabsl(a.r - b.r)) < 0) return 0;
67 | if (!sgn(d - a.r - b.r) || !sgn(d - fabsl(a.r - b.r)))
    ↪ return 1;
68 | return 2;
69 | }
70
71 vp cc_intersection(cc a, cc b) { // 仅返回有限交点; 重合圆返回
    ↪ 空集
72 | int cnt = cc_intersection_count(a, b);
73 | if (cnt <= 0) return {};
74 | if (a.c == b.c) return {a.c}; // 重合的零半径圆
75 | LD d = (a.c - b.c).len();
76 | P v = (b.c - a.c).unit();
77 | LD x = (a.r * a.r - b.r * b.r + d * d) / (2 * d);
78 | LD h = sqrtl(max(0.0L, a.r * a.r - x * x));
79 | P p = a.c + v * x;
80 | if (cnt == 1) return {p};
81 | return {p + v.rot90() * h, p - v.rot90() * h};
82 | }
83
84 vp tangent(cp a, cc b) { // 点到圆的切点
85 | return cc_intersection(b, C(a, b.c));
86 | }
87
88 vector<L> tangent(cc a, cc b, int f) { // f=1 外公切线, f=-1
    ↪ 内公切线
89 | P d = b.c - a.c;
90 | LD dr = b.r - f * a.r, d2 = d.len2();

```

```

91 | if (!sgn(d2) || sgn(d2 - dr * dr) < 0) return {};
92 | LD h = sqrtl(max(0.0L, d2 - dr * dr));
93 | vector<L> ret;
94 | for (int s : {-1, 1}) {
95 | | P v = (d * dr + d.rot90() * (h * s)) / d2;
96 | | ret.push_back({a.c + v * a.r, b.c + v * (f * b.r)});
97 | | if (!sgn(h)) break;
98 | }
99 | return ret;
100 | }

```

8.5 凸包与闵可夫斯基和

```

1 vp convex_hull(vp a) {
2 | sort(a.begin(), a.end()); // 小于号 (y, x) 字典序
3 | a.erase(unique(a.begin(), a.end()), a.end()); // 必要时去重
4 | int n = (int)a.size(), cnt = 0;
5 | vp ret;
6 | for (int i = 0; i < n; ++i) {
7 | | while (cnt > 1
8 | | && turn(ret[cnt - 2], ret[cnt - 1], a[i]) <= 0)
9 | | | --cnt, ret.pop_back(); // 保留凸包边界上的点: < 0
10 | | ++cnt, ret.push_back(a[i]);
11 | for (int i = n - 2, fixed = cnt; i >= 0; --i) {
12 | | while (cnt > fixed
13 | | && turn(ret[cnt - 2], ret[cnt - 1], a[i]) <= 0)
14 | | | --cnt, ret.pop_back(); // 保留凸包边界上的点: < 0
15 | | ++cnt, ret.push_back(a[i]);
16 | if (n > 1) ret.pop_back(); // n <= 2 是凸包吗?
17 | return ret; } // 小于号为 (y, x) 时边 [0, 2pi) 逆时针

```

```

1 vp minkowski_sum(vp a, vp b) {
2 | // size > 0, rotate(begin, min, end), 无重, 小于号 (y, x)
3 | if (a.size() == 1 || b.size() == 1) {
4 | | vp ret;
5 | | for (auto i : a) for (auto j : b) ret.push_back(i+j);
6 | | return ret; }
7 | vp x, y;
8 | for (size_t i = 0; i < a.size(); ++i)
9 | | x.push_back(a[(i + 1) % a.size()] - a[i]);
10 | for (size_t i = 0; i < b.size(); ++i)
11 | | y.push_back(b[(i + 1) % b.size()] - b[i]);
12 | vp ret(x.size() + y.size());
13 | merge(x.begin(), x.end(), y.begin(), y.end(),
14 | | ret.begin(), [](cp u, cp v) {
15 | | return half(u) != half(v) ? half(u) > half(v) : (u ^
    ↪ v) > 0; });
16 | P cur = a[0] + b[0];
17 | for (auto &i : ret) swap(i, cur), cur = cur + i;
18 | return ret; } // ret 可能共线, 但没有重点

```

8.6 旋转卡壳

```

1 LD convex_diameter(cvp p) { // p 为逆时针凸包, 无重复末点
2 | int n = (int)p.size();
3 | if (n < 2) return 0;
4 | if (n == 2) return (p[0] - p[1]).len();
5 | LD ans2 = 0;
6 | for (int i = 0, j = 1; i < n; ++i) {
7 | | int ni = (i + 1) % n;
8 | | while (fabsl((p[ni] - p[i]) ^ (p[(j + 1) % n] - p[i]))
9 | | | > fabsl((p[ni] - p[i]) ^ (p[j] - p[i])) + eps)
10 | | | j = (j + 1) % n;
11 | | ans2 = max({ans2, (p[i] - p[j]).len2(), (p[ni] -
    ↪ p[j]).len2()});
12 | }
13 | return sqrtl(ans2);
14 | }
15
16 vp remove_repeated_convex_vertices(cvp p) {
17 | vp q;
18 | for (cp x : p) if (q.empty() || x != q.back())
    ↪ q.push_back(x);
19 | if (q.size() > 1 && q.front() == q.back()) q.pop_back();
20 | return q;
21 | }
22
23 LD convex_min_width(cvp input) { // input 为逆时针凸包, 允许
    ↪ 相邻重复点
24 | vp p = remove_repeated_convex_vertices(input);
25 | int n = (int)p.size();
26 | if (n < 3) return 0;
27 | LD ans = INF;
28 | for (int i = 0, j = 1; i < n; ++i) {

```



```

29 | int ni = (i + 1) % n;
30 | while (((p[ni] - p[i]) ^ (p[(j + 1) % n] - p[i]))
31 | | > ((p[ni] - p[i]) ^ (p[j] - p[i])) + eps)
32 | | j = (j + 1) % n;
33 | ans = min(ans, ((p[ni] - p[i]) ^ (p[j] - p[i]))
34 | | / (p[ni] - p[i]).len());
35 | }
36 | return ans;
37 | }
38 |
39 | struct MinRectangle {
40 | LD area = 0;
41 | array<P, 4> p{};
42 | };
43 |
44 | MinRectangle min_area_rectangle(cvp input) { // input 为逆时
    ↪ 针凸包, 允许相邻重复点
45 | vp p = remove_repeated_convex_vertices(input);
46 | int n = (int)p.size();
47 | if (!n) return {};
48 | if (n == 1) return {0, {p[0], p[0], p[0], p[0]}};
49 | MinRectangle ans{INF, {}};
50 | P first_u = (p[1] - p[0]).unit(), first_v =
    ↪ first_u.rot90();
51 | int high = 0, right = 0, left = 0;
52 | for (int j = 1; j < n; ++j) {
53 | | if (p[j] * first_v > p[high] * first_v) high = j;
54 | | if (p[j] * first_u > p[right] * first_u) right = j;
55 | | if (p[j] * first_u < p[left] * first_u) left = j;
56 | }
57 | for (int i = 0; i < n; ++i) {
58 | | P u = (p[(i + 1) % n] - p[i]).unit(), v = u.rot90();
59 | | while (p[(high + 1) % n] * v > p[high] * v + eps)
60 | | | high = (high + 1) % n;
61 | | while (p[(right + 1) % n] * u > p[right] * u + eps)
62 | | | right = (right + 1) % n;
63 | | while (p[(left + 1) % n] * u < p[left] * u - eps)
64 | | | left = (left + 1) % n;
65 | | LD min_u = p[left] * u, max_u = p[right] * u;
66 | | LD min_v = p[i] * v, max_v = p[high] * v;
67 | | LD area = (max_u - min_u) * (max_v - min_v);
68 | | if (area < ans.area) ans = {area, {
69 | | | u * min_u + v * min_v, u * max_u + v * min_v,
70 | | | u * max_u + v * max_v, u * min_u + v * max_v}};
71 | | }
72 | return ans;
73 | }

```

8.7 多边形基础

```

1 | vp polygon_cut(cvp p, cl l) { // 保留有向直线 l 左侧 (含边界)
2 | vp ret;
3 | int n = (int)p.size();
4 | if (!n) return ret;
5 | for (int i = 0; i < n; ++i) {
6 | | P a = p[i], b = p[(i + 1) % n];
7 | | int sa = turn(l.s, l.t, a), sb = turn(l.s, l.t, b);
8 | | if (sa >= 0) ret.push_back(a);
9 | | if (sa * sb < 0) ret.push_back(ll_intersection(l, {a,
    ↪ b}));
10 | }
11 | return ret;
12 | }
13 |
14 | LD polygon_signed_area2(cvp p) { // 逆时针为正, 返回两倍有向面
    ↪ 积
15 | LD ret = 0;
16 | for (int i = 0, n = (int)p.size(); i < n; ++i)
17 | | ret += p[i] ^ p[(i + 1) % n];
18 | return ret;
19 | }
20 |
21 | LD polygon_area(cvp p) {
22 | return fabs1(polygon_signed_area2(p)) / 2;
23 | }
24 |
25 | LD polygon_perimeter(cvp p) {
26 | LD ret = 0;
27 | for (int i = 0, n = (int)p.size(); i < n; ++i)
28 | | ret += (p[i] - p[(i + 1) % n]).len();
29 | return ret;
30 | }
31 |
32 | P polygon_centroid(cvp p) { // 退化时返回边界按长度加权的质心
33 | LD area2 = polygon_signed_area2(p);

```

```

34 | if (!sgn(area2)) {
35 | | P ret;
36 | | LD perimeter = 0;
37 | | for (int i = 0, n = (int)p.size(); i < n; ++i) {
38 | | | LD length = (p[i] - p[(i + 1) % n]).len();
39 | | | ret = ret + (p[i] + p[(i + 1) % n]) * length;
40 | | | perimeter += length;
41 | | }
42 | | if (sgn(perimeter)) return ret / (2 * perimeter);
43 | | return p.empty() ? P{} : p[0];
44 | }
45 | P ret;
46 | for (int i = 0, n = (int)p.size(); i < n; ++i) {
47 | | LD cross = p[i] ^ p[(i + 1) % n];
48 | | ret = ret + (p[i] + p[(i + 1) % n]) * cross;
49 | }
50 | return ret / (3 * area2);
51 | }
52 |
53 | // -1: 外部, 0: 边界, 1: 内部; 与顶点顺逆时针无关。
54 | int point_in_polygon(cp u, cvp p) {
55 | bool in = false;
56 | for (int i = 0, n = (int)p.size(); i < n; ++i) {
57 | | P a = p[i], b = p[(i + 1) % n];
58 | | if (point_on_segment(u, {a, b})) return 0;
59 | | if ((a.y > u.y) != (b.y > u.y)
60 | | | && sgn((b - a) ^ (u - a)) == (b.y > a.y ? 1 : -1))
    ↪ in = !in;
61 | }
62 | return in ? 1 : -1;
63 | }

```

8.8 直线半平面交

```

1 | bool turn_left(cl a, cl b, cl c) {
2 | return turn(a.s, a.t, ll_intersection(b, c)) > 0; }
3 | bool is_para(cl a, cl b){return !sgn((a.t-a.s) ^
    ↪ (b.t-b.s));}
4 | bool cmp(cl a, cl b) {
5 | int sign = half(a.t - a.s) - half(b.t - b.s);
6 | int dir = sgn((a.t - a.s) ^ (b.t - b.s));
7 | if (!dir && !sign) return turn(a.s, a.t, b.t) < 0;
8 | else return sign ? sign > 0 : dir > 0; }
9 | vp hpi(vector<L> h) { // 半平面交
10 | sort(h.begin(), h.end(), cmp);
11 | vector<L> q(h.size()); int l = 0, r = -1;
12 | for(auto &i : h) {
13 | | while (l < r && !turn_left(i, q[r - 1], q[r])) --r;
14 | | while (l < r && !turn_left(i, q[l], q[l + 1])) ++l;
15 | | if (l <= r && is_para(i, q[r])) continue;
16 | | q[++r] = i; }
17 | while (r - l > 1 && !turn_left(q[l], q[r - 1], q[r])) --r;
18 | while (r - l > 1 && !turn_left(q[r], q[l], q[l + 1])) ++l;
19 | if (r - l < 2) return {};
20 | vp ret(r - l + 1);
21 | for(int i = l; i <= r; i++)
22 | | ret[i - l] = ll_intersection(q[i], q[i == r ? l : i +
    ↪ 1]);
23 | return ret; }
24 | // 空集会在队列里留下一个开区域; 开区域会被判定为空集。
25 | // 为了保证正确性, 一定要加足够大的框, 尽可能避免零面积区域。
26 | // 实在需要零面积区域边缘, 需要仔细考虑 turn_left 的实现。

```

8.9 整数半平面交

```

1 | struct IL : P { // ax + by + c >= 0, 适合整数半平面
2 | LD z;
3 | IL(LD a = 0, LD b = 0, LD c = 0) : P(a, b), z(c) {}
4 | IL(cp a, cp b) : P((b - a).rot90()), z(a ^ b) {}
5 | LD operator()(cp a) const { return a * static_cast<const
    ↪ P &>(*this) + z; }
6 | };
7 | using cil = const IL &;
8 |
9 | P il_intersection(cil u, cil v) {
10 | return P(P(u.z, u.y) ^ P(v.z, v.y),
11 | | P(u.x, u.z) ^ P(v.x, v.z)) / -(static_cast<const P
    ↪ &>(u)
12 | | ^ static_cast<const P &>(v));
13 | }
14 | LD signed_distance(cil l, cp x = P()) { return l(x) /
    ↪ l.len(); }
15 | bool il_parallel(cil a, cil b) { return !sgn(a ^ b); }
16 | LD det3(cil a, cil b, cil c) {

```

```

17 | return (a ^ b) * c.z + (b ^ c) * a.z + (c ^ a) * b.z;
18 | }
19 | int check(cil a, cil b, cil c) {
20 | | return sgn(det3(b, c, a)) * sgn(b ^ c);
21 | }
22 | bool il_turn_left(cil a, cil b, cil c) { return check(a, b,
    | | c) > 0; }
23 | bool il_cmp(cil a, cil b) {
24 | | if (il_parallel(a, b) && a * b > 0)
25 | | | return signed_distance(a) < signed_distance(b);
26 | | return half(a) == half(b) ? sgn(a ^ b) > 0 : half(b) > 0;
27 | }
28 |
29 | IL perpendicular(cil l) { return {l.y, -l.x, 0}; }
30 | IL parallel_through(cil l, cp o) { return {l.x, l.y, l.z -
    | | l(o)}; }
31 | P project(cp x, cil l) { return x - static_cast<const P
    | | &>(l) * (l(x) / l.len2()); }
32 | P reflect(cp x, cil l) { return x - static_cast<const P
    | | &>(l) * (2 * l(x) / l.len2()); }
33 | bool il_perpendicular(cil a, cil b) { return !sgn(a * b); }
34 | LD triangle_area(cil a, cil b, cil c) {
35 | | LD d = det3(a, b, c);
36 | | return d * d / (2 * (a ^ b) * (b ^ c) * (c ^ a));
37 | }
38 |
39 | vector<IL> cut_integral_hpi(const vector<IL> &o, IL l) {
40 | | vector<IL> ret;
41 | | int n = (int)o.size();
42 | | for (int i = 0; i < n; ++i) {
43 | | | cil u = o[i], v = o[(i + 1) % n], w = o[(i + 2) % n];
44 | | | int va = check(l, u, v), vb = check(l, v, w);
45 | | | if (va > 0 || vb > 0 || (!va && !vb))
    | | | | ret.push_back(v);
46 | | | if (va >= 0 && vb < 0) ret.push_back(l);
47 | | }
48 | | return ret;
49 | }

```

8.10 凸包询问：凸包内、切点、交点、最近点

```

1 | struct ConvexQuery { // a 为逆时针严格凸包
2 | | vp a;
3 | | int n;
4 | | ConvexQuery(vp points = {}) : a(move(points)),
    | | | n((int)a.size()) {}
5 |
6 | | P at(int i) const { // 空凸包没有合法下标
7 | | | if (!n) throw out_of_range("ConvexQuery::at() called
    | | | | on an empty polygon");
8 | | | i %= n;
9 | | | if (i < 0) i += n;
10 | | | return a[i];
11 | | }
12 | | bool inside(cp u) const { // 含边界
13 | | | if (!n) return false;
14 | | | if (n == 1) return u == a[0];
15 | | | if (n == 2) return point_on_segment(u, {a[0], a[1]});
16 | | | if (turn(a[0], a[1], u) < 0 || turn(a[0], a[n - 1], u)
    | | | | < 0)
17 | | | | return false;
18 | | | int l = 1, r = n - 1;
19 | | | while (l + 1 < r) {
20 | | | | int m = (l + r) / 2;
21 | | | | if (turn(a[0], a[m], u) >= 0) l = m;
22 | | | | else r = m;
23 | | | }
24 | | | return turn(a[l], a[(l + 1) % n], u) >= 0;
25 | | }
26 |
27 | | template<class F> int extreme(F better) const {
28 | | | int l = 0, r = n - 1, d = 1;
29 | | | if (better(a[r], a[l])) swap(l, r), d = -1;
30 | | | while (d * (r - l) > 1) {
31 | | | | int m = (l + r) / 2;
32 | | | | if (better(a[m], a[l]) && better(a[m], a[m - d])) l
    | | | | | = m;
33 | | | | else r = m;
34 | | | }
35 | | | return l;
36 | | }
37 |
38 | | pair<int, int> tangents(cp u) const { // u 严格在凸包外, n
    | | | >= 3
39 | | | return {

```

```

40 | | | extreme([&](cp x, cp y) { return turn(u, y, x) > 0;
    | | | | < >}),
41 | | | extreme([&](cp x, cp y) { return turn(u, x, y) > 0;
    | | | | < >});
42 | | }
43 |
44 | | int edge_crossing(cp u, cp v, int l, int r) const {
45 | | | int side = turn(u, v, at(l));
46 | | | while (l + 1 < r) {
47 | | | | int m = (l + r) / 2;
48 | | | | if (side == turn(u, v, at(m))) l = m;
49 | | | | else r = m;
50 | | | }
51 | | | return l % n;
52 | | }
53 | | bool line_crossings(cp u, cp v, int &i, int &j) const {
54 | | | int p0 = extreme([&](cp x, cp y) {
55 | | | | return ((v - u) ^ (x - u)) < ((v - u) ^ (y - u));
56 | | | });
57 | | | int p1 = extreme([&](cp x, cp y) {
58 | | | | return ((v - u) ^ (x - u)) > ((v - u) ^ (y - u));
59 | | | });
60 | | | if (turn(u, v, a[p0]) * turn(u, v, a[p1]) >= 0) return
    | | | | false;
61 | | | if (p0 > p1) swap(p0, p1);
62 | | | i = edge_crossing(u, v, p0, p1);
63 | | | j = edge_crossing(u, v, p1, p0 + n);
64 | | | return true;
65 | | }
66 |
67 | | LD chain_distance(cp u, int l, int r) const {
68 | | | if (l > r) r += n;
69 | | | int side = sgn((u - at(l)) * (at(l + 1) - at(l)));
70 | | | LD ret = point_to_segment(u, {at(l), at(l + 1)});
71 | | | while (l + 1 < r) {
72 | | | | int m = (l + r) / 2;
73 | | | | if (side == sgn((u - at(m)) * (at(m + 1) - at(m))))
    | | | | | l = m;
74 | | | | else r = m;
75 | | | }
76 | | | return min(ret, point_to_segment(u, {at(l), at(l +
    | | | | 1)}));
77 | | }
78 | | LD distance(cp u) const {
79 | | | if (inside(u)) return 0;
80 | | | if (n == 1) return (u - a[0]).len();
81 | | | if (n == 2) return point_to_segment(u, {a[0], a[1]});
82 | | | auto [x, y] = tangents(u);
83 | | | return min(chain_distance(u, x, y), chain_distance(u,
    | | | | y, x));
84 | | }
85 | }

```

8.11 简单多边形内最长线段

```

1 | struct PolygonChordSolver { // 逆时针简单多边形顶点间最长内含
    | | 线段
2 | | vp p;
3 | | int n;
4 | | PolygonChordSolver(vp points) : p(move(points)),
    | | | n((int)p.size()) {}
5 |
6 | | bool segment_inside(cp u, cp v) const {
7 | | | for (int i = 0; i < n; ++i) {
8 | | | | int j = (i + 1) % n, k = (i + 2) % n;
9 | | | | cp a = p[i], b = p[j], c = p[k];
10 | | | | if (intersection_judge_strict({u, v}, {a, b}))
    | | | | | return false;
11 | | | | if (point_on_segment(b, {u, v})) {
12 | | | | | bool good = true, convex = turn(a, b, c) >= 0;
13 | | | | | for (cp x : {u, v}) {
14 | | | | | | if (convex) good &= turn(a, b, x) >= 0 &&
    | | | | | | | turn(b, c, x) >= 0;
15 | | | | | | else good &= !(turn(b, x, c) > 0 && turn(b,
    | | | | | | | c, x) > 0);
16 | | | | | }
17 | | | | | if (!good) return false;
18 | | | | }
19 | | | }
20 | | | return true;
21 | | }
22 |
23 | | LD ray_exit_distance(int uid, int vid) const {

```

```

24 | | cp u = p[uid], v = p[vid];
25 | | LD far = INF;
26 | | for (int i = 0; i < n; ++i) {
27 | | | int j = (i + 1) % n, k = (i + 2) % n;
28 | | | cp a = p[i], b = p[j], c = p[k];
29 | | | if (two_side(a, b, {u, v})) {
30 | | | | LD s1 = (b - a) ^ (u - a), s2 = (b - a) ^ (v -
    | | | | ↪ a);
31 | | | | if (sgn(s1 - s2) && sgn(s1) != sgn(s2 - s1))
32 | | | | | far = min(far, (u - ll_intersection({a, b},
    | | | | | ↪ {u, v})).len());
33 | | | }
34 | | | if (j != uid && point_on_ray(b, {u, v})) {
35 | | | | bool good = turn(a, b, c) <= 0;
36 | | | | for (cp x : {u - (b - u), b + (b - u)})
37 | | | | | good &= !(turn(a, b, x) > 0 && turn(b, c, x)
    | | | | | ↪ > 0);
38 | | | | if (!good) far = min(far, (u - b).len());
39 | | | }
40 | | }
41 | | return far;
42 | }
43
44 | LD solve() const {
45 | | LD ans = 0;
46 | | for (int i = 0; i < n; ++i)
47 | | | for (int j = i + 1; j < n; ++j) {
48 | | | | if (!segment_inside(p[i], p[j])) continue;
49 | | | | ans = max(ans, ray_exit_distance(i, j) +
    | | | | ↪ ray_exit_distance(j, i)
50 | | | | | - (p[i] - p[j]).len());
51 | | | }
52 | | return ans;
53 | }
54 | };

```

8.12 $O(n^2)$ Ear Clipping 三角剖分

```

1 | vector<array<int, 3>> ear_clipping(cvp p) { // 简单多边形,
    | ↪ 0(n^2)
2 | | int n = (int)p.size();
3 | | if (n < 3) return {};
4 | | vector<int> id(n);
5 | | iota(id.begin(), id.end(), 0);
6 | | if (polygon_signed_area2(p) < 0) reverse(id.begin(),
    | | ↪ id.end());
7 | | vector<array<int, 3>> ret;
8 | | for (bool changed = true; changed && id.size() > 3; ) {
9 | | | changed = false;
10 | | | for (int i = 0, m = (int)id.size(); i < m; ++i) {
11 | | | | int a = id[(i + m - 1) % m], b = id[i], c = id[(i +
    | | | | ↪ 1) % m];
12 | | | | if (!turn(p[a], p[b], p[c]) &&
    | | | | ↪ point_on_segment(p[b], {p[a], p[c]})) {
13 | | | | | id.erase(id.begin() + i), changed = true;
14 | | | | | break;
15 | | | | }
16 | | | }
17 | | }
18
19 | | auto in_triangle = [&](cp q, cp a, cp b, cp c) {
20 | | | return turn(a, b, q) >= 0 && turn(b, c, q) >= 0
21 | | | && turn(c, a, q) >= 0;
22 | | };
23 | | while (id.size() > 3) {
24 | | | bool found = false;
25 | | | for (int i = 0, m = (int)id.size(); i < m; ++i) {
26 | | | | int a = id[(i + m - 1) % m], b = id[i], c = id[(i +
    | | | | ↪ 1) % m];
27 | | | | if (turn(p[a], p[b], p[c]) <= 0) continue;
28 | | | | bool empty = true;
29 | | | | for (int x : id) if (x != a && x != b && x != c
30 | | | | | && in_triangle(p[x], p[a], p[b], p[c])) {
31 | | | | | empty = false;
32 | | | | | break;
33 | | | | }
34 | | | | if (!empty) continue;
35 | | | | ret.push_back({a, b, c});
36 | | | | id.erase(id.begin() + i);
37 | | | | found = true;
38 | | | | break;
39 | | | }
40 | | | if (!found) return {}; // 自交、重复点或退化输入
41 | | }

```

```

42 | | ret.push_back({id[0], id[1], id[2]});
43 | | return ret;
44 | }

```

8.13 单插入动态凸包

```

1 | struct hull { // upper hull, left to right
2 | | set<P> a; LD tot;
3 | | hull () {tot = 0;}
4 | | template<class It> LD calc(It it) {
5 | | | auto u = it == a.begin() ? a.end() : prev(it);
6 | | | auto v = next(it);
7 | | | LD ret = 0;
8 | | | if (u != a.end()) ret += *u ^ *it;
9 | | | if (v != a.end()) ret += *it ^ *v;
10 | | | if (u != a.end() && v != a.end()) ret -= *u ^ *v;
11 | | | return ret; }
12 | | void insert (P p) {
13 | | | if (!a.size()) { a.insert (p); return; }
14 | | | auto it = a.lower_bound (p);
15 | | | bool out;
16 | | | if (it == a.begin()) out = (p < *it); // special case
17 | | | else if (it == a.end()) out = true;
18 | | | else out = turn(*prev(it), *it, p) > 0;
19 | | | if (!out) return;
20 | | | while (it != a.begin()) {
21 | | | | auto o = prev(it);
22 | | | | if (o == a.begin() || turn(*prev(o), *o, p) < 0)
    | | | | ↪ break;
23 | | | | else erase(o); }
24 | | | while (it != a.end()) {
25 | | | | auto o = next(it);
26 | | | | if (o == a.end() || turn(p, *it, *o) < 0) break;
27 | | | | else erase(it), it = o; }
28 | | | tot += calc(a.insert(p).first); }
29 | | template<class It> void erase(It it) { tot -= calc(it);
    | | ↪ a.erase(it); } };

```

8.14 Delaunay 三角剖分

```

1 | /* Delaunay Triangulation 随机增量算法 :
2 | | 节点数至少为点数的 6 倍, 空间消耗较大注意计算内存使用
3 | | 建图的过程在 build 中, 注意初始化内存池和初始三角形 (LOTS)
4 | | Triangulation::find 返回包含某点的三角形
5 | | Triangulation::add_point 将某点加入三角剖分
6 | | 某个 Tri 在三角剖分中当且仅当它的 has_ch 为 0
7 | | 如果要找到三角形 u 的邻域, 则枚举它的所有 u.edge[i].tri, 该条
    | | ↪ 边的两个点为 u.p[(i+1)%3], u.p[(i+2)%3] */
8 | | const int N = 100000 + 5, MAX_TRIS = N * 6;
9 | | bool in_circum(cp p1, cp p2, cp p3, cp p4) {
10 | | | LD u11 = p1.x-p4.x, u21 = p2.x-p4.x, u31 = p3.x-p4.x;
11 | | | LD u12 = p1.y-p4.y, u22 = p2.y-p4.y, u32 = p3.y-p4.y;
12 | | | LD u13 = p1.len2()-p4.len2(), u23 = p2.len2()-p4.len2();
13 | | | LD u33 = p3.len2()-p4.len2();
14 | | | LD d = -u13*u22*u31 + u12*u23*u31 + u13*u21*u32
15 | | | | - u11*u23*u32 - u12*u21*u33 + u11*u22*u33;
16 | | | return sgn(d) > 0; }
17 | | LD dir(cp a, cp b, cp p) { return (b-a) ^ (p-a); }
18 | | typedef int SideRef; struct Tri; typedef Tri* TriRef;
19 | | struct Edge {
20 | | | TriRef tri; SideRef side; Edge() : tri(0), side(0) {}
21 | | | Edge(TriRef tri_, SideRef side_) : tri(tri_), side(side_)
    | | | ↪ {} };
22 | | struct Tri { // Triangle
23 | | | P p[3]; Edge edge[3]; TriRef ch[3]; Tri(){}
24 | | | Tri(cp p0, cp p1, cp p2){
25 | | | | p[0] = p0; p[1] = p1; p[2] = p2;
26 | | | | | ch[0] = ch[1] = ch[2] = 0; }
27 | | | | bool has_ch() const { return ch[0] != 0; }
28 | | | | int num_ch() const {
29 | | | | | return ch[0] == 0 ? 0
30 | | | | | : ch[1] == 0 ? 1
31 | | | | | : ch[2] == 0 ? 2 : 3; }
32 | | | | bool contains(cp q) const {
33 | | | | | LD a=dir(p[0],p[1],q), b=dir(p[1],p[2],q),
    | | | | | ↪ c=dir(p[2],p[0],q);
34 | | | | | return sgn(a) >= 0 && sgn(b) >= 0 && sgn(c) >= 0; }
35 | | } triange_pool[MAX_TRIS], *tot_tri;
36 | | void set_edge(Edge a, Edge b) {
37 | | | if (a.tri) a.tri->edge[a.side] = b;
38 | | | if (b.tri) b.tri->edge[b.side] = a; }
39 | | class Triangulation {
40 | | | public:
41 | | | Triangulation() {

```

```

42 | | | tot_tri = triange_pool;
43 | | | const LD LOTS = 1e6; // NOTE: below base triangle
44 | | | the_root = new(tot_tri++) Tri (P(-LOTS,-LOTS),
    | | |   ↳ P(+LOTS,-LOTS), P(0,+LOTS)); }
45 | | | TriRef find(P p) const { return find(the_root,p); }
46 | | | void add_point(cp p) { add_point(find(the_root,p),p);
    | | |   ↳ }
47 | | | void build(vp p) {
48 | | | | shuffle(p.begin(), p.end(), rnd);
49 | | | | for (cp x : p) add_point(x);
50 | | | }
51 | private:
52 | | | TriRef the_root;
53 | | | static TriRef find(TriRef root,cp p){
54 | | | | for( ; ; ) {
55 | | | | | if (!root->has_ch()) return root;
56 | | | | | else for (int i = 0; i < 3 && root->ch[i] ; ++i)
57 | | | | | | if (root->ch[i]->contains(p))
58 | | | | | | | {root = root->ch[i]; break;} } }
59 | | | void add_point(TriRef root, cp p) {
60 | | | | TriRef tab,tbc,tca;
61 | | | | tab = new(tot_tri++) Tri(root->p[0], root->p[1],
    | | | |   ↳ p);
62 | | | | tbc = new(tot_tri++) Tri(root->p[1], root->p[2],
    | | | |   ↳ p);
63 | | | | tca = new(tot_tri++) Tri(root->p[2], root->p[0],
    | | | |   ↳ p);
64 | | | | set_edge(Edge(tab,0),Edge(tbc,1));
65 | | | | set_edge(Edge(tbc,0),Edge(tca,1));
66 | | | | set_edge(Edge(tca,0),Edge(tab,1));
67 | | | | set_edge(Edge(tab,2),root->edge[2]);
68 | | | | set_edge(Edge(tbc,2),root->edge[0]);
69 | | | | set_edge(Edge(tca,2),root->edge[1]);
70 | | | | root->ch[0]=tab;root->ch[1]=tbc;root->ch[2]=tca;
71 | | | | flip(tab,2); flip(tbc,2); flip(tca,2); }
72 | | | void flip(TriRef tri, SideRef side_index) {
73 | | | | TriRef trj = tri->edge[side_index].tri;
74 | | | | int pj = tri->edge[side_index].side;
75 | | | | if(!trj ||
    | | | |   ↳ lin_circum(tri->p[0],tri->p[1],tri->p[2],trj->p[pj])
    | | | |   ↳ return;
76 | | | | TriRef trk = new(tot_tri++)
    | | | |   ↳ Tri(tri->p[(side_index+1)%3], trj->p[pj],
    | | | |   ↳ tri->p[side_index]);
77 | | | | TriRef trl = new(tot_tri++) Tri(trj->p[(pj+1)%3],
    | | | |   ↳ tri->p[side_index], trj->p[pj]);
78 | | | | set_edge(Edge(trk,0), Edge(trl,0));
79 | | | | set_edge(Edge(trk,1), tri->edge[(side_index+2)%3]);
80 | | | | set_edge(Edge(trk,2), trj->edge[(pj+1)%3]);
81 | | | | set_edge(Edge(trl,1), trj->edge[(pj+2)%3]);
82 | | | | set_edge(Edge(trl,2), tri->edge[(side_index+1)%3]);
83 | | | | tri->ch[0]=trk; tri->ch[1]=trl; tri->ch[2]=0;
84 | | | | trj->ch[0]=trk; trj->ch[1]=trl; trj->ch[2]=0;
85 | | | | flip(trk,1); flip(trk,2); flip(trl,1); flip(trl,2);
    | | | |   ↳ }
86 | }
87 | //The Euclidean minimum spanning tree of a set of points is
    | | ↳ a subset of the Delaunay triangulation of the same
    | | ↳ points.
88 | //Connecting the centers of the circumcircles produces the
    | | ↳ Voronoi diagram.
89 | //No point in P is inside the circumcircle of any triangle.
90 | //Maximize the minimum angle of all the angles of the
    | | ↳ triangles.

```

8.15 圆反演, 阿波罗尼茨圆

所有关于两点 A, B 满足 $PA/PB = k$ 且不等于 1 的点 P 的轨迹是一个圆。

圆幂: 半径为 R 的圆 O , 任意一点 P 到 O 的幂为 $h = OP^2 - R^2$

圆幂定理: 过 P 的直线交圆在 A 和 B 两点, 则 $PA \cdot PB = |h|$

根轴: 到两圆等幂点的轨迹是一条垂直于连心线的直线

反演: 已知一圆 C , 圆心为 O , 半径为 r , 如果 P 与 P' 在过圆心 O 的直线上, 且 $OP \cdot OP' = r^2$, 则称 P 与 P' 关于 O 互为反演。一般 C 取单位圆。

反演的性质:

不过反演中心的直线反形是过反演中心的圆, 反之亦然。

不过反演中心的圆, 它的反形是一个不过反演中心的圆。

两条直线在交点 A 的夹角, 等于它们的反形在相应点 A' 的夹角, 但方向相反。

两个相交圆周在交点 A 的夹角等于它们的反形在相应点 A' 的夹角, 但方向相反。

直线和圆周在交点 A 的夹角等于它们的反演图形在相应点 A' 的夹角, 但方向相反。

正交圆反形也正交。相切圆反形也相切, 当切点为反演中心时, 反形为两条平行线。

两两相切的圆 r_1, r_2, r_3 , 求与他们都相切的圆 r_4 。分母取负号, 答案再取绝对值, 为外切圆半径。分母取正号为内切圆半径。 $r_4^{\pm} = \frac{r_1 r_2 r_3}{r_1 r_2 + r_1 r_3 + r_2 r_3 \pm 2\sqrt{r_1 r_2 r_3 (r_1 + r_2 + r_3)}}$

```

1 | C inv_c2c(P O, LD R, C A) {
2 | | LD OA = (A.c - O).len();
3 | | LD RB = 0.5 * R * R * (1 / (OA - A.r) - 1 / (OA + A.r));

```

```

4 | | LD OB = OA * RB / A.r;
5 | | P B = O + (A.c - O) * (OB / OA);
6 | | return {B, RB};
7 | } // 点 O 在圆 A 外, 求圆 A 的反演圆 B, R 是反演半径
8 | C inv_l2c(P O, LD R, L l) {
9 | | P p = project_to_line(O, l);
10 | | LD d = (O - p).len();
11 | | LD RB = R * R / (2 * d);
12 | | P VB = (p - O) / d * RB;
13 | | return {O + VB, RB};
14 | } // 不过 O 点的直线反演为过 O 点的圆, R 是反演半径
15 | L inv_c2l(P O, LD R, C A) {
16 | | LD t = R * R / (2 * A.r);
17 | | P p = O + (A.c - O).unit() * t;
18 | | return {p, p + (O - p).rot90()};
19 | } // 过 O 点的圆反演为不过 O 点的直线, R 是反演半径

```

8.16 圆并

```

1 | int circle_count;
2 | C circles[MAXN];
3 | LD coverage_area[MAXN]; // coverage_area[k]: 恰被 k 个圆覆盖
    | ↳ 的面积
4 |
5 | struct Event {
6 | | LD ang;
7 | | int delta;
8 | | bool operator<(const Event &o) const { return ang <
    | |   ↳ o.ang; }
9 | };
10 |
11 | bool same_circle(cc a, cc b) {
12 | | return a.c == b.c && !sgn(a.r - b.r);
13 | }
14 | bool contains(cc a, cc b) { // 圆 a 包含圆 b
15 | | return sgn(a.r - b.r - (a.c - b.c).len()) >= 0;
16 | }
17 |
18 | void add_interval(LD l, LD r, vector<Event> &e, int &base) {
19 | | const LD tau = 2 * pi;
20 | | while (l < 0) l += tau, r += tau;
21 | | while (l >= tau) l -= tau, r -= tau;
22 | | if (r < tau) e.push_back({l, 1}), e.push_back({r, -1});
23 | | else {
24 | | | ++base;
25 | | | e.push_back({r - tau, -1});
26 | | | e.push_back({l, 1});
27 | | }
28 | }
29 |
30 | void circle_union() {
31 | | fill(coverage_area, coverage_area + circle_count + 2, 0);
32 | | vector<LD> delta(circle_count + 2);
33 | | for (int i = 0; i < circle_count; ++i) {
34 | | | bool duplicate = false;
35 | | | for (int j = 0; j < i; ++j)
36 | | | | if (same_circle(circles[i], circles[j])) duplicate
    | | | |   ↳ = true;
37 | | | if (duplicate) continue;
38 | |
39 | | | int multiplicity = 0;
40 | | | for (int j = 0; j < circle_count; ++j)
41 | | | | multiplicity += same_circle(circles[i],
    | | | |   ↳ circles[j]);
42 | | | int cnt = multiplicity;
43 | | | vector<Event> e;
44 | | | for (int j = 0; j < circle_count; ++j) if (i != j) {
45 | | | | if (same_circle(circles[i], circles[j])) continue;
46 | | | | if (contains(circles[j], circles[i])) { ++cnt;
    | | | |   ↳ continue; }
47 | | | | if (contains(circles[i], circles[j])) continue;
48 | | | | LD d = (circles[j].c - circles[i].c).len();
49 | | | | if (sgn(d - circles[i].r - circles[j].r) >= 0)
    | | | |   ↳ continue;
50 | | | | LD a = atan2(circles[j].c.y - circles[i].c.y,
    | | | |   ↳ circles[j].c.x - circles[i].c.x);
51 | | | | LD b = acos(1/clamp((circles[i].r * circles[i].r + d
    | | | |   ↳ * d
52 | | | |   ↳ - circles[j].r * circles[j].r) / (2 *
    | | | |   ↳ circles[i].r * d),
    | | | |   ↳ -1.0L, 1.0L));
53 | | | | add_interval(a - b, a + b, e, cnt);
54 | | | }
55 | | | e.push_back({0, 0});
56 | | }
57 | }

```



```

58 | | e.push_back({2 * pi, 0});
59 | | sort(e.begin(), e.end());
60 | | for (int j = 0; j + 1 < (int)e.size(); ++j) {
61 | | | cnt += e[j].delta;
62 | | | LD l = e[j].ang, r = e[j + 1].ang, d = r - l;
63 | | | P p = circles[i].c + P(cosl(l), sinl(l)) *
    | | |   ↪ circles[i].r;
64 | | | P q = circles[i].c + P(cosl(r), sinl(r)) *
    | | |   ↪ circles[i].r;
65 | | | LD area = (p ^ q) / 2
66 | | | | + circles[i].r * circles[i].r * (d - sinl(d)) /
    | | |   ↪ 2;
67 | | | delta[cnt - multiplicity + 1] += area;
68 | | | delta[cnt + 1] -= area;
69 | | }
70 | }
71 | for (int k = 1; k <= circle_count; ++k)
72 | | coverage_area[k] = coverage_area[k - 1] + delta[k];
73 | for (int k = 1; k < circle_count; ++k)
74 | | coverage_area[k] -= coverage_area[k + 1];
75 | }

```

```

20 | | | tmp.begin(), [](cp a, cp b) {
21 | | | | return a.y == b.y ? a.x < b.x : a.y < b.y;
22 | | | });
23 | | copy(tmp.begin(), tmp.begin() + r - 1, p.begin() + 1);
24 | | vector<P> strip;
25 | | for (int i = 1; i < r; ++i) if ((p[i].x - mid_x) *
    | | |   ↪ (p[i].x - mid_x) < ret) {
26 | | | | for (int j = (int)strip.size() - 1; j >= 0
27 | | | | | && (p[i].y - strip[j].y) * (p[i].y - strip[j].y)
    | | | | |   ↪ < ret; --j)
28 | | | | | ret = min(ret, (p[i] - strip[j]).len2());
29 | | | | strip.push_back(p[i]);
30 | | | }
31 | | return ret;
32 | };
33 | return sqrtl(solve(0, n));
34 | }

```

8.17 多边形与圆交

```

1 | LD angle (cp u, cp v) {
2 | | return atan2l(fabsl(u ^ v), u * v); }
3 | LD circle_edge_area2(cp s, cp t, LD r) { // 2 * area
4 | | if (!sgn(s ^ t)) return 0;
5 | | LD theta = angle(s, t);
6 | | LD d = point_to_segment({0, 0}, {s, t});
7 | | if (sgn(d - r) >= 0) return theta * r * r;
8 | | auto q = lc_intersection({s, t}, C({0, 0}, r));
9 | | if (q.size() < 2) return theta * r * r;
10 | | P lo = sgn(s ^ q[0]) >= 0 ? q[0] : s;
11 | | P hi = sgn(q[1] ^ t) >= 0 ? q[1] : t;
12 | | return (lo ^ hi) + (theta - angle(lo, hi)) * r * r; }
13 | LD polygon_circle_intersection_area(cvp p, cc c) {
14 | | LD ret = 0;
15 | | for (int i = 0; i < (int) p.size(); ++i) {
16 | | | auto u = p[i] - c.c;
17 | | | auto v = p[(i + 1) % p.size()] - c.c;
18 | | | int s = sgn(u ^ v);
19 | | | if (s > 0) ret += circle_edge_area2(u, v, c.r);
20 | | | else if (s < 0) ret -= circle_edge_area2(v, u, c.r);
21 | | } return fabsl(ret) / 2; } // ret 在 p 逆时针为正

```

8.20 圆上整点

```

1 | vector <LL> solve(LL r) {
2 | | vector <LL> ret; // non-negative Y pos
3 | | ret.push_back(0);
4 | | LL l = 2 * r, s = sqrt(l);
5 | | for (LL d=1; d<=s; d++) if (l%d==0) {
6 | | | LL lim=LL(sqrt(l/(2*d)));
7 | | | for (LL a = 1; a <= lim; a++) {
8 | | | | LL b = sqrt(l/d-a*a);
9 | | | | if (a*a+b*b==l/d && __gcd(a,b)==1 && a!=b)
10 | | | | | ret.push_back(d*a*b);
11 | | | } if (d*d==l) break;
12 | | | lim = sqrt(d/2);
13 | | | for (LL a=1; a<=lim; a++) {
14 | | | | LL b = sqrt(d - a * a);
15 | | | | if (a*a+b*b==d && __gcd(a,b)==1 && a!=b)
16 | | | | | ret.push_back(1/d*a*b);
17 | | } } return ret; }

```

8.18 球面基础, 经纬度球面距离

球面距离: 连接球面两点的大圆劣弧 (所有曲线中最短)

球面角: 球面两个大圆所在半平面形成的二面角

球面凸多边形: 把一个球面多边形任意一边向两方无限延长成大圆, 其余边都在此大圆的同旁。

球面角盈 E : 球面凸 n 边形的内角和与 $(n-2)\pi$ 的差

离北极夹角 θ , 距离 h 的球冠: $S = 2\pi R h = 2\pi R^2(1 - \cos \theta)$, $V = \frac{\pi h^2}{3}(3R - h)$

球面凸 n 边形面积: $S = ER^2$

```

1 | // longitude 经度范围:  $\pm\pi$ , latitude 纬度范围:  $\pm\pi/2$ 
2 | LD sphereDis(LD lon1, LD lat1, LD lon2, LD lat2, LD R) {
3 | | LD d = cosl(lat1) * cosl(lat2) * cosl(lon1 - lon2) +
    | | |   ↪ sinl(lat1) * sinl(lat2);
4 | | return R * acosl(clamp(d, -1.0L, 1.0L)); }

```

8.19 最近点对

```

1 | LD closest_pair(vp p) { //  $O(n \log n)$ , 返回欧氏距离
2 | | int n = (int)p.size();
3 | | if (n < 2) return INF;
4 | | sort(p.begin(), p.end());
5 | | vector<P> tmp(n);
6 | | function<LD(int, int)> solve = [&](int l, int r) -> LD {
7 | | | if (r - l <= 3) {
8 | | | | LD ret = INF;
9 | | | | for (int i = 1; i < r; ++i)
10 | | | | | for (int j = i + 1; j < r; ++j)
11 | | | | | | ret = min(ret, (p[i] - p[j]).len2());
12 | | | | sort(p.begin() + 1, p.begin() + r,
13 | | | | | [](cp a, cp b) { return a.y == b.y ? a.x < b.x :
    | | | | |   ↪ a.y < b.y; });
14 | | | | return ret;
15 | | | }
16 | | | int m = (l + r) / 2;
17 | | | LD mid_x = p[m].x;
18 | | | LD ret = min(solve(l, m), solve(m, r));
19 | | | merge(p.begin() + 1, p.begin() + m, p.begin() + m,
    | | |   ↪ p.begin() + r,

```

8.21 三角形

```

1 | P incenter(cp a, cp b, cp c) {
2 | | LD p = (a-b).len() + (b-c).len() + (c-a).len();
3 | | return (a*(b-c).len() + b*(c-a).len() + c*(a-b).len()) /
    | | |   ↪ p; }
4 | P circumcenter(cp a, cp b, cp c) { return C(a, b, c).c; }
5 | P orthocenter(cp a, cp b, cp c) {
6 | | return a + b + c - circumcenter(a, b, c) * 2; }
7 | P fermt_point(cp a, cp b, cp c) {
8 | | if (a == b) return a;
9 | | if (b == c) return b;
10 | | if (c == a) return c;
11 | | LD ab = (a-b).len(), bc = (b-c).len(), ca = (c-a).len();
12 | | LD cosa = ((b-a)*(c-a)) / ab / ca;
13 | | LD cosb = ((a-b)*(c-b)) / ab / bc;
14 | | LD cosc = ((b-c)*(a-c)) / ca / bc;
15 | | LD sq3 = pi / 3; P mid;
16 | | if (sgn(cosa + 0.5) < 0) mid = a;
17 | | else if (sgn(cosb + 0.5) < 0) mid = b;
18 | | else if (sgn(cosc + 0.5) < 0) mid = c;
19 | | else if (sgn((b-a)^(c-a)) < 0)
20 | | | mid = ll_intersection({a, b+(c-b).rot(sq3)}, {b,
    | | |   ↪ c+(a-c).rot(sq3)});
21 | | else mid = ll_intersection({a, c+(b-c).rot(sq3)}, {c,
    | | |   ↪ b+(a-b).rot(sq3)});
22 | | return mid; } // minimize(|A-x|+|B-x|+|C-x|)

```

8.22.6 高维球体积

```

1 struct p3 {
2 | LD x, y, z;
3 | p3(LD x_ = 0, LD y_ = 0, LD z_ = 0) : x(x_), y(y_), z(z_)
4 | {}
5 | LD &operator[](int i) { return i == 0 ? x : (i == 1 ? y :
6 | ↪ z); }
7 | LD operator[](int i) const { return i == 0 ? x : (i == 1
8 | ↪ ? y : z); }
9 | LD len2() const { return x*x + y*y + z*z; }
10 | LD len() const { return sqrt(len2()); }
11 | p3 unit() const { LD d = len(); return {x/d, y/d, z/d}; }
12 | void read() { if (scanf("%Lf%Lf%Lf", &x, &y, &z) != 3) x
13 | ↪ = y = z = 0; }
14 };
15 p3 operator+(p3 a, p3 b) { return {a.x+b.x, a.y+b.y,
16 | ↪ a.z+b.z}; }
17 p3 operator-(p3 a, p3 b) { return {a.x-b.x, a.y-b.y,
18 | ↪ a.z-b.z}; }
19 p3 operator*(p3 a) { return {-a.x, -a.y, -a.z}; }
20 p3 operator*(p3 a, LD k) { return {a.x*k, a.y*k, a.z*k}; }
21 p3 operator*(LD k, p3 a) { return {a.x*k, a.y*k, a.z*k}; }
22 p3 operator/(p3 a, LD k) { return {a.x/k, a.y/k, a.z/k}; }
23 bool operator==(p3 a, p3 b) {
24 | return !sgn(a.x-b.x) && !sgn(a.y-b.y) && !sgn(a.z-b.z);
25 }
26 LD dot(p3 a, p3 b) { return a.x*b.x + a.y*b.y + a.z*b.z; }
27 p3 cross(p3 a, p3 b) {
28 | return {a.y*b.z-a.z*b.y, a.z*b.x-a.x*b.z,
29 | ↪ a.x*b.y-a.y*b.x};
30 }
31 LD mix(p3 a, p3 b, p3 c) { return dot(cross(a, b), c); }
32
33 LD rotation[3][3]; // 右手系, 绕单位向量 axis 旋转
34 void rotation_matrix(p3 axis, LD angle) {
35 | axis = axis.unit();
36 | LD c = cosl(angle), s = sinl(angle);
37 | for (int i = 0; i < 3; ++i) {
38 | | int j = (i + 1) % 3, k = (i + 2) % 3;
39 | | LD x = axis[i], y = axis[j], z = axis[k];
40 | | rotation[i][i] = (y*y + z*z)*c + x*x;
41 | | rotation[i][j] = x*y*(1-c) + z*s;
42 | | rotation[i][k] = x*z*(1-c) - y*s;
43 | }
44 }
45
46 struct l3 { p3 s, t; };
47 struct plane {
48 | p3 normal; LD offset; // dot(normal, x) = offset, normal
49 | ↪ 为单位向量
50 | plane(p3 normal_ = {1,0,0}, p3 point = {})
51 | : normal(normal_.unit()), offset(dot(normal, point))
52 | {}
53 };
54 p3 project_to_plane(p3 a, const plane &b) {
55 | return a + b.normal * (b.offset - dot(a, b.normal));
56 }
57 pair<p3, p3> closest_points(l3 x, l3 y) { // 两直线需异面且不
58 | ↪ 平行
59 | p3 u = x.t-x.s, v = y.t-y.s, r = x.s-y.s;
60 | LD a = dot(u,u), b = dot(u,v), e = dot(v,v), d = a*e-b*b;
61 | LD c = dot(u,r), f = dot(v,r);
62 | LD s = (b*f-c*e)/d, t = (a*f-b*c)/d;
63 | return {x.s+u*s, y.s+v*t};
64 }
65 p3 line_plane_intersection(const plane &a, l3 b) {
66 | LD t =
67 | ↪ (a.offset-dot(a.normal,b.s))/dot(a.normal,b.t-b.s);
68 | return b.s+(b.t-b.s)*t;
69 }
70 l3 plane_plane_intersection(const plane &a, const plane &b)
71 | ↪ {}

```

```

60 | p3 d = cross(a.normal,b.normal);
61 | LD d2 = d.len2();
62 | p3 s = cross(b.normal*a.offset-a.normal*b.offset,d)/d2;
63 | return {s,s+d};
64 | }
65 | p3 three_plane_intersection(const plane &a, const plane &b,
    |   ↪ const plane &c) {
66 |   LD d = mix(a.normal,b.normal,c.normal);
67 |   return (cross(b.normal,c.normal)*a.offset
    |     + cross(c.normal,a.normal)*b.offset
68 |     + cross(a.normal,b.normal)*c.offset)/d;
69 | }
70 | }

```

8.24 三维凸包

```

1 // 随机增量；输入需不存在重复点且仿射维数为 3。
2 // 点数不足、共线或全部共面时 build 返回 false, face 为空。
3 struct ConvexHull3D {
4 |   using Face = array<int,3>;
5 |   vector<p3> p;
6 |   vector<Face> face;
7 |
8 |   LD volume(Face f, int x) const {
9 |     return
    |       ↪ mix(p[f[1]]-p[f[0]],p[f[2]]-p[f[0]],p[x]-p[f[0]]);
10 |   }
11 |   Face outward(int a,int b,int c,int inside) const {
12 |     Face f{a,b,c};
13 |     if (sgn(volume(f,inside))>0) swap(f[1],f[2]);
14 |     return f;
15 |   }
16 |   bool build(vector<p3> points) {
17 |     p=move(points), face.clear();
18 |     if (p.size()<4) return false;
19 |     shuffle(p.begin(),p.end(),rnd);
20 |     int n=(int)p.size(),b=1,c=-1,d=-1;
21 |     for (int i=2;i<n;++i) if
    |       ↪ (cross(p[b]-p[0],p[i]-p[0]).len()>eps)
    |       ↪ {c=i;break;}
22 |     if (c<0) return false;
23 |     swap(p[2],p[c]);
24 |     for (int i=3;i<n;++i) if
    |       ↪ (fabs1(mix(p[1]-p[0],p[2]-p[0],p[i]-p[0]))>eps)
    |       ↪ {d=i;break;}
25 |     if (d<0) return false;
26 |     swap(p[3],p[d]);
27 |     face={outward(0,1,2,3),outward(0,3,1,2),
    |       | outward(0,2,3,1),outward(1,3,2,0)};
28 |     for (int x=4;x<n;++x) {
29 |       set<pair<int,int>> horizon;
30 |       vector<Face> keep;
31 |       for (Face f:face) if (sgn(volume(f,x))>0) {
32 |         for (int k=0;k<3;++k) {
33 |           pair<int,int>
    |             ↪ e={f[k],f[(k+1)%3]},rev={e.second,e.first};
34 |           if (horizon.count(rev)) horizon.erase(rev);
    |             ↪ else horizon.insert(e);
35 |         }
36 |       } else keep.push_back(f);
37 |       for (auto [u,v]:horizon) keep.push_back({u,v,x});
38 |       face.swap(keep);
39 |     }
40 |     return true;
41 |   }
42 | }
43 | };

```

8.25 最小覆盖球

```

1 struct Sphere {
2 |   p3 c; LD r;
3 | };
4 bool in_sphere(p3 p,const Sphere &s){return
    |   ↪ sgn((p-s.c).len()-s.r)<=0;}
5 Sphere sphere(p3 a){return {a,0};}
6 Sphere sphere(p3 a,p3 b){p3 c=(a+b)/2;return {c,
    |   ↪ (a-c).len()};}
7 Sphere sphere3(p3 a,p3 b,p3 c){
8 |   p3 u=b-a,v=c-a,n=cross(u,v);
9 |   if (!sgn(n.len2())) {
10 |     Sphere s=sphere(a,b);
11 |     array<pair<p3,p3>,2> candidate{{{a,c},{b,c}}};
12 |     for (auto [x,y]:candidate) {
13 |       Sphere t=sphere(x,y);
14 |       if (t.r>s.r) s=t;
15 |     }

```

```

16 |   return s;
17 | }
18 | p3
    |   ↪ o=a+(cross(n,u)*v.len2()+cross(v,n)*u.len2())/(2*n.len2());
19 | return {o,(a-o).len()};
20 | }
21 Sphere sphere4(p3 a,p3 b,p3 c,p3 d){
22 |   p3 u=b-a,v=c-a,w=d-a;
23 |   LD det=mix(u,v,w);
24 |   if (!sgn(det)) {
25 |     array<p3,4> q{a,b,c,d}; Sphere best{{{},{},INF}};
26 |     for (int i=0;i<4;++i) for (int j=i+1;j<4;++j) {
27 |       Sphere s=sphere(q[i],q[j]); bool ok=true;
28 |       for (p3 x:q) ok&=in_sphere(x,s);
29 |       if (ok&&s.r<best.r) best=s;
30 |     }
31 |     for (int i=0;i<4;++i) {
32 |       Sphere s=sphere3(q[(i+1)%4],q[(i+2)%4],q[(i+3)%4]);
    |       ↪ bool ok=true;
33 |       for (p3 x:q) ok&=in_sphere(x,s);
34 |       if (ok&&s.r<best.r) best=s;
35 |     }
36 |     return best;
37 |   }
38 |   p3 o=a+(cross(v,w)*(u.len2()/2)+cross(w,u)*(v.len2()/2)
    |     +cross(u,v)*(w.len2()/2))/det;
39 |   return {o,(a-o).len()};
40 | }
41 |
42 Sphere min_sphere(vector<p3> p){ // 随机增量, O(n) 期望
43 |   if (p.empty()) return {{{},{},0};
44 |   shuffle(p.begin(),p.end(),rnd); Sphere ret=sphere(p[0]);
45 |   for (int i=1;i<(int)p.size();++i) if
    |     ↪ (!in_sphere(p[i],ret)) {
46 |     ret=sphere(p[i]);
47 |     for (int j=0;j<i;++j) if (!in_sphere(p[j],ret)) {
48 |       ret=sphere(p[i],p[j]);
49 |       for (int k=0;k<j;++k) if (!in_sphere(p[k],ret)) {
50 |         ret=sphere3(p[i],p[j],p[k]);
51 |         for (int l=0;l<k;++l) if (!in_sphere(p[l],ret))
52 |           ret=sphere4(p[i],p[j],p[k],p[l]);
53 |       }
54 |     }
55 |   }
56 |   return ret;
57 | }

```

8.26 黄金三分

```

1 constexpr LD R = (sqrt(5) - 1) / 2;
2 auto split = [](LD l, LD r){ return l + (r - l) * R; };
3 LD solve(LD a, LD c, auto f){
4 |   LD b = split(a, c), bv = f(b);
5 |   for (int _ = T; _; _--) {
6 |     LD x = split(a, b), xv = f(x);
7 |     if (xv < bv) c = b, b = x, bv = xv; // 最小化, 注意符号
8 |     else a = c, c = x;
9 |   } return bv; }

```

8.27 自适应 Simpson

```

1 // Adaptive Simpson's method : LD simpson::solve (LD (*f)
    |   ↪ (LD), LD l, LD r, LD eps) : integrates f over (l, r)
    |   ↪ with error eps.
2 struct simpson {
3 |   LD area (LD (*f) (LD), LD l, LD r) {
4 |     LD m = l + (r - l) / 2;
5 |     return (f(l) + 4 * f(m) + f(r)) * (r - l) / 6;
6 |   }
7 |   LD solve (LD (*f) (LD), LD l, LD r, LD eps, LD a) {
8 |     LD m = l + (r - l) / 2;
9 |     LD left = area (f, l, m), right = area (f, m, r);
10 |    if (abs (left + right - a) <= 15 * eps) // TLE: || eps <
    |      ↪ EPS ** 2
11 |      return left + right + (left + right - a) / 15.0;
12 |    return solve (f, l, m, eps / 2, left) + solve (f, m, r,
    |      ↪ eps / 2, right);
13 |  }
14 LD solve (LD (*f) (LD), LD l, LD r, LD eps) {
15 |   return solve (f, l, r, eps, area (f, l, r));
16 | };

```

9. 图论

9.1 Hopcroft-Karp $O(\sqrt{VE})$

```

1 vector<int> E[N];
2 vector<int> ml, mr, a, p;
3 void match(int nl, int nr) { // 1-based
4     ml.assign(nl + 1, 0);
5     mr.assign(nr + 1, 0); // nr
6     while (true) {
7         bool ok = 0;
8         a.assign(nl + 1, 0);
9         p.assign(nl + 1, 0); // nl
10        static queue<int> q;
11        for (int i = 1; i <= nl; i++)
12            if (!ml[i]) a[i] = p[i] = i, q.push(i);
13        while (!q.empty()) {
14            int x = q.front(); q.pop();
15            if (ml[a[x]]) continue;
16            for (auto y : E[x]) {
17                if (!mr[y]) {
18                    for (ok = 1; y; x = p[x])
19                        mr[y] = x, swap(ml[x], y);
20                    break;
21                } else if (!p[mr[y]])
22                    q.push(y = mr[y]), p[y] = x, a[y] = a[x];
23            } // while (!q.empty())
24            if (!ok) break; } }
25 array<vector<int>, 2> min_edge_cover(int nl, int nr) {
26     match(nl, nr); vector<int> l, r;
27     for (int i = 1; i <= nl; i++) if (!a[i]) l.push_back(i);
28     for (int i = 1; i <= nr; i++) if (a[mr[i]]) r.push_back(i);
29     return {l, r}; }

```

9.2 Hungarian $O(VE/w)$

```

1 using B = bitset<N>; B edge[N];
2 bool dfs(int x, B& unvis, vector<int>& match) {
3     for (B z = edge[x];;) {
4         z &= unvis;
5         int y = z._Find_first();
6         if (y == N) return 0;
7         unvis.reset(y);
8         if (!match[y] || dfs(match[y], unvis, match))
9             return match[y] = x, 1; } }
10 vector<int> match(int nl, int nr) {
11     B unvis; unvis.set();
12     vector<int> match(nr + 1, ret(nl + 1));
13     for (int i = 1; i <= nl; i++)
14         if (dfs(i, unvis, match)) unvis.set();
15     for (int i = 1; i <= nr; i++) ret[match[i]] = i;
16     return ret; }

```

9.3 Shuffle 一般图最大匹配 $O(VE)$

```

1 // 该算法的正确性无法保证
2 int n, m, mat[N], vis[N]; vector<int> E[N];
3 bool dfs(int tim, int x) {
4     shuffle(E[x].begin(), E[x].end(), rng);
5     vis[x] = tim;
6     for (auto y : E[x]) {
7         int z = mat[y]; if (vis[z] == tim) continue;
8         mat[x] = y, mat[y] = x, mat[z] = 0;
9         if (!z || dfs(tim, z)) return true;
10        mat[x] = 0, mat[y] = z, mat[z] = y; }
11     return false; }
12 int main() { // 暗含二分图性质跑一次即可
13     for (int _ = 0; _ < 10; _++) {
14         fill(vis + 1, vis + n + 1, 0);
15         for (int i = 1; i <= n; i++) if (!mat[i]) dfs(i, i); } }

```

9.4 极大团计数

```

1 // 0下标, 需删除自环 (即确保  $E_{ii} = 0$ , 补图要特别注意)
2 // 极大团计数, 最坏情况  $O(3^{n/3})$ 
3 ll ans; ull E[64]; #define bit(i) (1ULL << (i))
4 void dfs(ull P, ull X, ull R) { // 不要方案时可去掉R
5     if (!P && !X) { ++ans; sol.pb(R); return; }
6     ull Q = P & ~E[__builtin_ctzll(P | X)];
7     for (int i; i = __builtin_ctzll(Q), Q; Q &= ~bit(i)) {
8         dfs(P & E[i], X & E[i], R | bit(i));
9         P &= ~bit(i), X |= bit(i); } }
10 ans = 0; dfs(n == 64 ? ~0ULL : bit(n) - 1, 0, 0);

```

9.5 KM 最大权匹配 $O(V^3)$

```

1 ll e[N][M]; // O(1^2*r)
2 vector<int> KM(int n, int m) {
3     vector<ll> l(n + 1), r(m + 1);
4     vector<int> p(m + 1), ans(n + 1);
5     for (int i = 1; i <= n; ++i) {
6         vector<ll> d(m + 1, 1e18);
7         vector<int> pre(m + 1), done(m + 1);
8         int x, v, u = 0;
9         for (p[0] = i; x = p[u]; u = v) {
10            done[u] = 1;
11            ll min = 1e18;
12            for (int j = 1; j <= m; ++j) if (!done[j]) {
13                auto w = e[x][j] - l[x] - r[j];
14                if (w < d[j]) d[j] = w, pre[j] = u;
15                if (d[j] < min) min = d[j], v = j;
16            }
17            for (int j = 0; j <= m; ++j) {
18                if (done[j]) l[p[j]] += min, r[j] -= min;
19                else d[j] -= min;
20            }
21        }
22        for (int v; u; u = v) v = pre[u], p[u] = p[v];
23    }
24    for (int j = 1; j <= m; ++j) ans[p[j]] = j;
25    return ans; }

```

9.6 欧拉回路

```

1 /* comment : directed */
2 int e, cur[N]/*, deg[N]*/;
3 vector<int> E[N];
4 int id[M]; bool vis[M];
5 stack<int> stk;
6 void dfs(int u) {
7     for (cur[u]; cur[u] < E[u].size(); cur[u]++) {
8         int i = cur[u];
9         if (vis[abs(E[u][i])]) continue;
10        int v = id[abs(E[u][i])] ^ u;
11        vis[abs(E[u][i])] = 1; dfs(v);
12        stk.push(E[u][i]);
13    } // dfs for all when disconnect
14    void add(int u, int v) {
15        id[+e] = u ^ v; // s = u
16        E[u].push_back(e); E[v].push_back(-e);
17        /* E[u].push_back(e); deg[v]++; */
18    } bool valid() {
19        for (int i = 1; i <= n; i++)
20            if (E[i].size() & 1) return 0;
21        /* if (E[i].size() != deg[i]) return 0; */
22        return 1; }

```

9.7 Blossom 一般图最大匹配 $O(|V||E|)$

```

1 int mch[MAXN], pre[MAXN], fa[MAXN], col[MAXN];
2 int hscnt = 0, hs[MAXN]; queue<int> q;
3 int find(int x) { return fa[x] == x ? x : fa[x] = find(fa[x]); }
4 int lca(int x, int y) {
5     for (++hscnt; x; y ? swap(x, y) : (void)0) {
6         x = find(x);
7         if (hs[x] == hscnt) return x;
8         hs[x] = hscnt, x = pre[mch[x]]; } }
9 void blossom(int x, int y, int rt) {
10    for (int z; find(x) != rt; y = z, x = pre[y]) {
11        pre[x] = y, z = mch[x];
12        if (col[z] == 1) q.push(z), col[z] = 0;
13        if (find(x) == x) fa[x] = rt;
14        if (find(z) == z) fa[z] = rt; } }
15 bool bfs(int st) {
16    while (!q.empty()) q.pop();
17    q.push(st), col[st] = 0;
18    while (!q.empty()) {
19        int x = q.front(); q.pop();
20        for (auto v : e[x]) {
21            if (col[v] == -1) {
22                pre[v] = x, col[v] = 1;
23                if (!mch[v]) {
24                    for (int y; v; swap(mch[y], v))
25                        mch[v] = y, y = pre[v];
26                    return true; }
27                q.push(mch[v]); col[mch[v]] = 0;
28            } else if (col[v] == 0 && find(x) != find(v)) {
29                int f = lca(x, v); blossom(x, v, f);

```



```

30 | | | blossom(v, x, f); } } }
31 | return false; }
32 int solve() {
33 | int ans = 0;
34 | for(int i=1; i<=n; i++) {
35 | | for(int j=1; j<=n; j++) col[j] = -1, fa[j] = j;
36 | | if(!mch[i] && bfs(i)) ans++; }
37 | return ans; }

```

9.8 2-SAT, 强连通分量 / Bitset Kosaraju

```

1 int stp, sccs, top; // N 开 **两倍**
2 int dfn[N], low[N], scc[N], stk[N], ans[N];
3 void add(int x, int a, int y, int b) { // 注意连边对称
4 | E[x << 1 | a].push_back(y << 1 | b); }
5 void tarjan(int x) {
6 | dfn[x] = low[x] = ++stp;
7 | stk[top++] = x;
8 | for (auto y : E[x]) {
9 | | if (!dfn[y])
10 | | | tarjan(y), low[x] = min(low[x], low[y]);
11 | | else if (!scc[y])
12 | | | low[x] = min(low[x], dfn[y]); }
13 | if (low[x] == dfn[x]) {
14 | | sccs++;
15 | | do scc[stk[--top]] = sccs;
16 | | while (stk[top] != x); } }
17 bool solve() {
18 | int cnt = n + n; stp = top = sccs = 0;
19 | fill(dfn, dfn + cnt + 1, 0); fill(scc, scc + cnt + 1, 0);
20 | for (int i = 0; i < cnt; ++i) if (!dfn[i]) tarjan(i);
21 | for (int i = 0; i < n; ++i) {
22 | | if (scc[i << 1] == scc[i << 1 | 1]) return false;
23 | | ans[i] = (scc[i << 1 | 1] < scc[i << 1]); }
24 | return true; }

```

```

1 bitset<N> e[N], re[N], vis; vector<int> sta;
2 void dfs0(int x, bitset<N> e[]) {
3 | vis.reset(x);
4 | while (true) {
5 | | int go = (e[x] & vis)._Find_first();
6 | | if (go == N) break;
7 | | dfs0(go, e); }
8 | sta.push_back(x); }
9 vector<vector<int>> solve() { // re 需要连好反向边
10 | vis.set();
11 | for(int i = 1; i <= n; ++i) if (vis.test(i)) dfs0(i, e);
12 | vis.set();
13 | auto s = sta;
14 | vector<vector<int>> ret;
15 | for(int i = n - 1; i >= 0; --i) if (vis.test(s[i])) {
16 | | sta.clear(), dfs0(s[i], re), ret.push_back(sta); }
17 | return ret; }

```

9.9 Tarjan 点双, 边双

```

1 // 强连通分量
2 void tar(int x) {
3 | dfn[x] = low[x] = ++tot;
4 | S[++top] = x, vis[x] = 1;
5 | for(auto v : e[x]) {
6 | | if(!dfn[v]) tar(v), low[x] = min(low[x], low[v]);
7 | | else if(vis[v]) low[x] = min(low[x], dfn[v]); }
8 | if(dfn[x] == low[x]) {
9 | | ++bcnt;
10 | | while(S[top] != x) {
11 | | | int v = S[top--];
12 | | | vis[v] = 0;
13 | | | bel[v] = bcnt; }
14 | | top--, vis[x] = 0, bel[x] = bcnt; } }
15 /*边双*/
16 // 注意! 建立边双树或者圆方树后【边表大小】是否开够
17 // 求边双: 无视掉 bri 后搜出每个连通块, 记得多测
18 int dfn[N], low[N], dfscnt; // clear dfn low bri dfscnt
19 bool bri[M << 1]; // 注意此处是边数
20 void tarjan(int x, int last) { // last 是边
21 | dfn[x] = low[x] = ++dfscnt;
22 | for (int i = head[x]; i; i = e[i].next) {
23 | | int v = e[i].to;
24 | | if (!dfn[v]) {
25 | | | tarjan(v, i);
26 | | | low[x] = min(low[x], low[v]);
27 | | | if (low[v] > dfn[x]) bri[i] = bri[i ^ 1] = 1;
28 | | } else if (dfn[v] < dfn[x] && ((i ^ 1) != last))

```

```

29 | | | low[x] = min(low[x], dfn[v]); } }
30 /** 建立圆方树+求割点 **/
31 int is_cut[N], dfn[N], low[N], dfscnt, pcnt;
32 int stk[N], dep; // clear dfn low is_cut dfscnt, let pcnt=n
33 void tarjan(int x, int fa) {
34 | int child = 0;
35 | dfn[x] = low[x] = ++dfscnt; stk[++dep] = x;
36 | for (auto v : e[x]) {
37 | | if (!dfn[v]) {
38 | | | ++child; tarjan(v, x);
39 | | | low[x] = min(low[x], low[v]);
40 | | | if (low[v] >= dfn[x]) {
41 | | | | is_cut[x] = true;
42 | | | | ++pcnt; // square node index
43 | | | | int j = 0, sz = 1;
44 | | | | do {
45 | | | | | j = stk[dep--];
46 | | | | | addedge(pcnt, j);
47 | | | | | ++sz;
48 | | | | } while (j != v);
49 | | | | addedge(pcnt, x); }
50 | | } else if low[x] == min(low[x], dfn[v]); }
51 | if (!fa && child == 1) is_cut[x] = false; }

```

9.10 Dominator Tree 支配树

```

1 vector<int> G[MAXN], R[MAXN], son[MAXN];
2 int ufs[MAXN]; // R 是反图, son 存的是 sdom 树上的儿子
3 int idom[MAXN], sdom[MAXN], anc[MAXN];
4 // anc: sdom的dfn最小的祖先
5 int pr[MAXN], dfn[MAXN], id[MAXN], stamp;
6 int findufs(int x) { if (ufs[x] == x) return x;
7 | int t = ufs[x]; ufs[x] = findufs(ufs[x]);
8 | if (dfn[sdom[anc[x]]] > dfn[sdom[anc[t]]])
9 | | anc[x] = anc[t];
10 | return ufs[x]; }
11 void dfs(int x) {
12 | dfn[x] = ++stamp; id[stamp] = x; sdom[x] = x;
13 | for (int y : G[x]) if (!dfn[y]) { pr[y] = x; dfs(y); } }
14 void get_dominator(int n) {
15 | for (int i = 1; i <= n; ++i) ufs[i] = anc[i] = i;
16 | dfs(1);
17 | for (int i = n; i > 1; i--) { int x = id[i];
18 | | for (int y : R[x]) if (dfn[y]) { findufs(y);
19 | | | if (dfn[sdom[x]] > dfn[sdom[anc[y]]])
20 | | | | sdom[x] = sdom[anc[y]]; } }
21 | | son[sdom[x]].push_back(x); ufs[x] = pr[x];
22 | | for (int u : son[pr[x]]) { findufs(u);
23 | | | idom[u] = (sdom[u] == sdom[anc[u]]) ?
24 | | | pr[x] : anc[u]; }
25 | | son[pr[x]].clear(); }
26 | for (int i = 2; i <= n; ++i) { int x = id[i];
27 | | if (idom[x] != sdom[x]) idom[x] = idom[idom[x]];
28 | | son[idom[x]].push_back(x); } }

```

9.11 环计数

```

1 // Time complexity:  $O(m\sqrt{m})$ 
2 for(int i=1; i<=m; i++) {
3 | if(deg[x[i]] > deg[y[i]] || (deg[x[i]] == deg[y[i]] &&
4 | | x[i] > y[i])) swap(x[i], y[i]);
5 | e[x[i]].push_back(y[i]); }
6 // 四元环计数
7 for(int i=1; i<=n; i++) {
8 | for(auto x : e[i]) for(auto y : e[x])
9 | | if(deg[i] < deg[y] || (deg[i] == deg[y] && i <= y))
10 | | | ans += cnt[y]++; // 注意判等
11 | for(auto x : e[i]) for(auto y : e[x]) cnt[y] = 0; }

```

9.12 Dinic 最大流

单位流量网络 在 0-1 流量图上 Dinic 有更好的性质。

- 复杂度为 $O(\min\{V^{2/3}, E^{1/2}\}E)$ 。
- $\text{dist}(s, t) = d$, 残量网络上至多还存在 E/d 的流。
- 每个点只有一个入/出度时复杂度 $O(V^{1/2}E)$, 例如 Hopcroft-Karp。

```

1 bool bfs() { // 约定s=n+1, t=s+1
2 | for(int i=1; i<=t; i++) dep[i] = 0, cur[i] = head[i];
3 | queue<int> q; q.push(s); dep[s] = 1;
4 | while(!q.empty()) {
5 | | int x = q.front(); q.pop();
6 | | for(int i=head[x]; i; i=e[i].next) {
7 | | | int v = e[i].to;

```

```

8 | | | if(dep[v] || !e[i].w) continue;
9 | | | dep[v] = dep[x] + 1; q.push(v);}}
10 | return dep[t];}
11 int dfs(int x, int in) {
12 | if(x == t) return in;
13 | int out = 0;
14 | for(int &i=cur[x];i;i=e[i].next) {
15 | | int v = e[i].to;
16 | | if(dep[v] != dep[x] + 1 || !e[i].w) continue;
17 | | int res = dfs(v, min(in, e[i].w));
18 | | in -= res, out += res;
19 | | e[i].w -= res, e[i^1].w += res;
20 | | if(!in) break;}
21 | if(!out) dep[x] = 0;
22 | return out;}

```

9.13 原始对偶费用流

```

1 void dij() { // reversed: from t to s
2 | for (int i = 1; i <= t; i++) dis[i] = inf;
3 | for (int i = 1; i <= t; i++) vis[i] = 0;
4 | dis[t] = 0;
5 | priority_queue<Node> q; q.push({t, 0});
6 | while(!q.empty()){
7 | | auto [x, t] = q.top(); q.pop();
8 | | if(vis[x]) continue;
9 | | vis[x] = 1;
10 | | for(int i=head[x];i;i=e[i].next){
11 | | | int v = e[i].to;
12 | | | if(vis[v] || !e[i^1].w) continue;
13 | | | ll cost = -e[i].cost + h[x] - h[v];
14 | | | if(dis[v] > dis[x] + cost) {
15 | | | | dis[v] = dis[x] + cost;
16 | | | | q.push({v, dis[v]});
17 | | } } }
18 bool bfs(){
19 | for(int i=1;i<=t;i++) dep[i] = 0, cur[i] = head[i];
20 | queue<int> q; q.push(s); dep[s] = 1;
21 | while(!q.empty()){
22 | | int x = q.front(); q.pop();
23 | | for(int i=head[x];i;i=e[i].next){
24 | | | int v = e[i].to;
25 | | | ll cost = e[i].cost + h[v] - h[x];
26 | | | if(dep[v] || !e[i].w || dis[v] != dis[x] - cost)
27 | | | | continue;
28 | | | dep[v] = dep[x] + 1;
29 | | | q.push(v);
30 | } } return dep[t]; }
31 int dfs(int x, ll in){
32 | if(x == t) return in;
33 | ll out = 0;
34 | for(int &i=cur[x];i;i=e[i].next){
35 | | int v = e[i].to;
36 | | ll cost = e[i].cost + h[v] - h[x];
37 | | if(dep[v] != dep[x] + 1 ||
38 | | | dis[v] != dis[x] - cost || !e[i].w) continue;
39 | | ll res = dfs(v, min(in, e[i].w));
40 | | in -= res, out += res;
41 | | e[i].w -= res, e[i^1].w += res;
42 | | if(!in) break;}
43 | if(!out) dep[x] = 0;
44 | return out; }
45 pair<ll, ll> solve() { // first_sssp: DAG最短路或SPFA
46 | ll flow = 0, cost = 0;
47 | for (first_sssp(); dis[s] != inf; dij()) {
48 | | while (bfs()) {
49 | | | ll tmp = dfs(s, inf);
50 | | | flow += tmp, cost += tmp * (dis[s] + h[s]); }
51 | | for(int i=1;i<=t;i++) h[i] += min(dis[i], dis[s]);
52 | } return {flow, cost}; }

```

9.14 网络流总结

9.15 网络流总结

最小割集, 最小割必须边以及可行边

最小割集 从 S 出发, 在残余网络中BFS所有权值非 0 的边 (包括反向边), 得到点集 $\{S\}$, 另一集为 $\{V\} - \{S\}$.

最小割集必须点 残余网络中 S 直接连向的点必在 S 的割集中, 直接连向 T 的点必在 T 的割集中; 若这些点的并集为全集, 则最小割方案唯一.

最小割可行边 在残余网络中求强联通分量, 将强联通分量缩点后, 剩余的边即为最小割可行边, 同时这些边也必然满流.

最小割必须边 在残余网络中求强联通分量, 若 S 出发可到 u , T 出发可到 v , 等价于 $scc_S = scc_u$ 且 $scc_T = scc_v$, 则该边为必须边.

常见问题

最大权闭合子图 点权, 限制条件形如: 选择 A 则必须选择 B , 选择 B 则必须选择 C , D . 建图方式: B 向 A 连边, CD 向 B 连边. 求解: S 向正权点连边, 负权点向 T 连边, 其余边容量 ∞ , 求最小割, 答案为 S 所在最小割集.

二次布尔型 (文理分科) n 个点分为两类, i 号点有 l_i 或 r_i 的代价, i, j 同属一侧分别获得 l_{ij} 或 r_{ij} 的代价, 问最小代价. $L \rightarrow i: (l_i + 1/2 \sum_j l_{ij})$, $i \rightarrow R: (r_i + 1/2 \sum_j r_{ij})$, $i \leftrightarrow j: 1/2(l_{ij} + r_{ij})$. 实现时边权乘 2 为整数, 求解后答案除 2 为整数. 图拆点可以看作二分图.

如果是二元限制是分类不同时有一个代价 d_{ij} , 建图可以简化为 $L \rightarrow i: l_i$, $i \rightarrow R: r_i$, $i \leftrightarrow j: d_{ij}$. 经典例子: xor 最小值, 按位拆开建图.

混合图欧拉回路 把无向边随便定向, 计算每个点的入度和出度, 如果有某个点出入度之差 $\deg_i = in_i - out_i$ 为奇数, 肯定不存在欧拉回路. 对于 $\deg_i > 0$ 的点, 连接边 $(i, T, \deg_i/2)$; 对于 $\deg_i < 0$ 的点, 连接边 $(S, i, -\deg_i/2)$. 最后检查是否满流即可.

二物流 水源 S_1 , 水汇 T_1 , 油源 S_2 , 油汇 T_2 , 每根管道流量共用. 求流量和最大. 建超级源 SS_1 汇 TT_1 , 连边 $SS_1 \rightarrow S_1, SS_1 \rightarrow S_2, T_1 \rightarrow TT_1, T_2 \rightarrow TT_1$, 设最大流为 x_1 . 建超级源 SS_2 汇 TT_2 , 连边 $SS_2 \rightarrow S_1, SS_2 \rightarrow T_2, T_1 \rightarrow TT_2, S_2 \rightarrow TT_2$, 设最大流为 x_2 . 则最大流中水流量 $\frac{x_1+x_2}{2}$, 油流量 $\frac{x_1-x_2}{2}$.

无源汇有上下界可行流 每条边 (u, v) 有一个上界容量 $C_{u,v}$ 和下界容量 $B_{u,v}$. 我们让下界变为 0, 上界变为 $C_{u,v} - B_{u,v}$. 但这样做流量不守恒. 建立超级源点 SS 和超级汇点 TT , 用 du_i 来记录每个节点的流量情况, $du_i = \sum B_{j,i} - \sum B_{i,j}$. 添加一些附加弧. 当 $du_i > 0$ 时, 连边 (SS, i, du_i) ; 当 $du_i < 0$ 时, 连边 $(i, TT, -du_i)$. 最后对 (SS, TT) 求一次最大流即可, 当所有附加边全部满流时 (即 $\maxflow ==$ 所有 $du_i > 0$ 之和) 时有可行解.

有源汇有上下界最大可行流 建立超级源点 SS 和超级汇点 TT , 首先判断是否存在可行流, 用无源汇有上下界可行流的方法判断. 增设一条从 T 到 S 没有下界容量为无穷的边, 那么原图就变成了一个无源汇有上下界可行流问题. 同样地建图后, 对 (SS, TT) 进行一次最大流, 判断是否有可行解. 如果有可行解, 删除超级源点 SS 和超级汇点 TT , 并删去 T 到 S 的这条边, 再对 (S, T) 进行一次最大流, 此时得到的 $\maxflow$ 即为有源汇有上下界最大可行流.

有源汇有上下界最小可行流 建立超级源点 SS 和超级汇点 TT , 和无源汇有上下界可行流一样新增一些边, 然后从 SS 到 TT 跑最大流. 接着加上边 (T, S, ∞) , 再从 SS 到 TT 跑一遍最大流. 如果所有新增边都是满的, 则存在可行流, 此时 T 到 S 这条边的流量即为最小可行流.

有上下界费用流 如果求无源汇有上下界最小费用可行流或有源汇有上下界最小费用最大可行流, 用 1.6.3.1/1.6.3.2 的构图方法, 给边加上费用即可. 求有源汇有上下界最小费用最小可行流, 要先用 1.6.3.3 的方法建图, 先求出一个保证必要边满流情况下的最小费用. 如果费用全部非负, 那么这时的费用就是答案. 如果费用有负数, 那么流多了可能更好, 继续做从 S 到 T 的流量任意的最小费用流, 加上原来的费用就是答案.

费用流消负环 新建超级源 SS 汇 TT , 对于所有流量非空的负权边 e , 先满流 ($ans+=e.f*e.c$, $e.rev.f+=e.f$, $e.f=0$), 再连边 $SS \rightarrow e.to$, $e.from \rightarrow TT$, 流量均为 $e.f(>0)$, 费用均为 0. 再连边 $T \rightarrow S$ 流量 ∞ 费用 0. 此时没有负环了. 做一遍 SS 到 TT 的最小费用最大流, 将费用累加 ans , 拆掉 $T \rightarrow S$ 的那条边 (此边的流量为残量网络中 $S \rightarrow T$ 的流量). 此时负环已消, 再继续跑最小费用最大流.

整数线性规划转费用流

首先将约束关系转化为所有变量下界为 0, 上界没有要求, 并满足一些等式, 每个变量在均在等式左边且出现恰好两次, 系数为 $+1$ 和 -1 , 优化目标为 $\max \sum v_i x_i$ 的形式. 将等式看做点, 等式 i 右边的值 b_i 若为正, 则 S 向 i 连边 $(b_i, 0)$, 否则向 T 连边 $(-b_i, 0)$. 将变量看做边, 记变量 x_i 的上界为 m_i (无上界则 $m_i = \text{inf}$), 将 x_i 系数为 $+1$ 的那个等式 u 向系数为 -1 的等式 v 连边 (m_i, v_i) .

9.16 Gomory-Hu 无向图最小割树 $O(V^3E)$

每次随便找两个点 s, t 求在原图的最小割, 在最小割树上连 (s, t, w_{cut}) , 递归对由割集隔开的部分继续做. 在得到的树上, 两点最小割即为树上瓶颈路. 实现时, 由于是随意找点, 可以写为分治的形式.

9.17 Stoer-Wagner 无向图最小割 $O(VE + V^2 \log V)$

```

1 const int N = 601;
2 int f[N], siz[N], G[N][N];
3 int getf(int x) {return f[x] == x ? x : f[x] = getf(f[x]);}
4 int dis[N], vis[N], bin[N];
5 int n, m;
6 int contract(int &s, int &t) { // Find s,t
7 | memset(vis, 0, sizeof(vis));
8 | memset(dis, 0, sizeof(dis));
9 | int i, j, k, mincut, maxc;
10 | for (i = 1; i <= n; i++) {
11 | | k = -1; maxc = -1;
12 | | for (j = 1; j <= n; j++)
13 | | | if (!bin[j] && !vis[j] && dis[j] > maxc) {
14 | | | | k = j;
15 | | | | maxc = dis[j]; }
16 | | if (k == -1) return mincut;
17 | | s = t; t = k; mincut = maxc; vis[k] = true;
18 | | for (j = 1; j <= n; j++)
19 | | | if (!bin[j] && !vis[j]) dis[j] += G[k][j];

```

```

20 | } return mincut; }
21 const int inf = 0x3f3f3f3f;
22 int solve() {
23 | int mincut, i, j, s, t, ans;
24 | for (mincut = inf, i = 1; i < n; i++) {
25 | | ans = contract(s, t);
26 | | bin[t] = true;
27 | | if (mincut > ans) mincut = ans;
28 | | if (mincut == 0) return 0;
29 | | for (j = 1; j <= n; j++)
30 | | | if (!bin[j]) G[s][j] = (G[j][s] += G[j][t]);
31 | } return mincut; }
32 int main() {
33 | cin >> n >> m;
34 | for (int i = 1; i <= n; ++i) f[i] = i, siz[i] = 1;
35 | for (int i = 1, u, v, w; i <= m; ++i) {
36 | | cin >> u >> v >> w;
37 | | int fu = getf(u), fv = getf(v);
38 | | if (fu != fv) {
39 | | | if (siz[fu] > siz[fv]) swap(fu, fv);
40 | | | f[fu] = fv, siz[fv] += siz[fu]; }
41 | | G[u][v] += w, G[v][u] += w; }
42 | cout << (siz[getf(1)] != n ? 0 : solve()); }

```

9.18 弦图

弦图的定义 连接环中不相邻的两个点的边称为弦。一个无向图称为弦图，当图中任意长度都大于 3 的环都至少有一个弦。

单纯点 一个点称为单纯点当 $\{v\} \cup A(v)$ 的导出子图为一个团。任何一个弦图都至少有一个单纯点，不是完全图的弦图至少有两个不相邻的单纯点。

完美消除序列 一个序列 $v_1, v_2, \dots, v_n$ 满足 v_i 在 $v_i, \dots, v_n$ 的诱导子图中为一个单纯点。一个无向图是弦图当且仅当它有一个完美消除序列。

最大势算法 从 n 到 1 的顺序依次给点标号。设 $label_i$ 表示第 i 个点与多少个已标号的点相邻，每次选择 $label$ 最大的未标号的点进行标号。用桶维护优先队列可以做到 $O(n + m)$ 。

弦图的判定 判定最大势算法输出是否合法即可。如果依次判断是否构成团，时间复杂度为 $O(nm)$ 。考虑优化，设 $v_{i+1}, \dots, v_n$ 中所有与 v_i 相邻的点依次为 $N(v_i) = \{v_{j_1}, \dots, v_{j_k}\}$ 。只需判断 v_{j_1} 是否与 $v_{j_2}, \dots, v_{j_k}$ 相邻即可。时间复杂度 $O(n + m)$ 。

弦图的染色 完美消除序列从后往前染色，染上出度的 mex 。

最大独立集 完美消除序列从前往后能选就选。

团数 最大团的点数。一般团团数 $\leq$ 色数，弦图团数 = 色数。

极大团 弦图的极大团一定为 $\{x\} \cup N(x)$ 。

最小团覆盖 用最少的团覆盖所有的点。设最大独立集为 $\{p_1, \dots, p_t\}$ ，则 $\{p_1 \cup N(p_1), \dots, p_t \cup N(p_t)\}$ 为最小团覆盖。

弦图 k 染色计数 $\prod_{v \in V} k - N(v) + 1$ 。

区间图 每个顶点代表一个区间，有边当且仅当区间有交。区间图是弦图，一个完美消除序列是右端点排序。

```

1 vector<int> L[N];
2 int seq[N], lab[N], col[N], id[N], vis[N];
3 void mcs() {
4 | for (int i = 0; i < n; i++) L[i].clear();
5 | fill(lab + 1, lab + n + 1, 0);
6 | fill(id + 1, id + n + 1, 0);
7 | for (int i = 1; i <= n; i++) L[0].push_back(i);
8 | int top = 0;
9 | for (int k = n; k; k--) {
10 | | int x = -1;
11 | | for ( ; ; ) {
12 | | | if (L[top].empty()) top--;
13 | | | else {
14 | | | | x = L[top].back(), L[top].pop_back();
15 | | | | if (lab[x] == top) break;
16 | | | }
17 | | }
18 | | seq[k] = x; id[x] = k;
19 | | for (auto v : E[x]) {
20 | | | if (!id[v]) {
21 | | | | L[++lab[v]].push_back(v);
22 | | | | top = max(top, lab[v]);
23 | | | } } }
24 bool check() {
25 | fill(vis + 1, vis + n + 1, 0);
26 | for (int i = n; i; i--) {
27 | | int x = seq[i];
28 | | vector<int> to;
29 | | for (auto v : E[x])
30 | | | if (id[v] > i) to.push_back(v);
31 | | if (to.empty()) continue;
32 | | int w = to.front();
33 | | for (auto v : to) if (id[v] < id[w]) w = v;

```

```

34 | | for (auto v : E[w]) vis[v] = i;
35 | | for (auto v : to)
36 | | | if (v != w && vis[v] != i) return false;
37 | | } return true; }
38 void color() {
39 | fill(vis + 1, vis + n + 1, 0);
40 | for (int i = n; i; i--) {
41 | | int x = seq[i];
42 | | for (auto v : E[x]) vis[col[v]] = x;
43 | | for (int c = 1; !col[x]; c++)
44 | | | if (vis[c] != x) col[x] = c;
45 | | } }

```

9.19 Minimum Mean Cycle 最小平均值环 $O(n^2)$

```

1 // 点标号为 1, 2, ..., n, 0 为虚拟源点向其他点连权值为 0 的单向边。
2 // f[i][v] : 从 0 到 v 恰好经过 i 条路的最短路
3 ll f[N][N] = {Inf}; int u[M], v[M], w[M]; f[0][0] = 0;
4 for (int i = 1; i <= n + 1; i++)
5 | for (int j = 0; j < m; j++)
6 | | f[i][v[j]] = min(f[i][v[j]], f[i - 1][u[j]] + w[j]);
7 double ans = Inf;
8 for (int i = 1; i <= n; i++) {
9 | double t = -Inf;
10 | for (int j = 1; j <= n; j++)
11 | | t = max(t, (f[n][i] - f[j][i]) / (double)(n - j));
12 | ans = min(t, ans); }

```

9.20 斯坦纳树

```

1 LL d[1 << 10][N]; int c[15];
2 priority_queue<pair<LL, int>> q;
3 void dij(int S) {
4 | for (int i = 1; i <= n; i++) q.push(mp(-d[S][i], i));
5 | while (!q.empty()) {
6 | | pair<LL, int> o = q.top(); q.pop();
7 | | if (-o.x != d[S][o.y]) continue;
8 | | int x = o.y;
9 | | for (auto v : E[x]) if (d[S][v.v] > d[S][x] + v.w) {
10 | | | d[S][v.v] = d[S][x] + v.w;
11 | | | q.push(mp(-d[S][v.v], v.v)); } } }
12 void solve() {
13 | for (int i = 1; i < (1 << K); i++)
14 | | for (int j = 1; j <= n; j++) d[i][j] = INF;
15 | for (int i = 0; i < K; i++) read(c[i]), d[1 << i][c[i]] = -o;
16 | for (int S = 1; S < (1 << K); S++) {
17 | | for (int k = S; k > (S >> 1); k = (k - 1) & S) {
18 | | | for (int i = 1; i <= n; i++) {
19 | | | | d[S][i] = min(d[S][i], d[k][i] + d[S ^ k][i]);
20 | | | } } } dij(S); }

```

9.21 图论结论

9.21.1 最小乘积问题原理

每个元素有两个权值 $\{x_i\}$ 和 $\{y_i\}$ ，要求在某个限制下（例如生成树，二分图匹配）使得 $\sum x_i y_i$ 最小。对于任意一种符合限制的选取方法，记 $X = \sum x_i, Y = \sum y_i$ ，可看做平面内一点 (X, Y) 。答案必在下凸壳上，找出该下凸壳所有点，即可枚举获得最优答案。可以递归求出此下凸壳所有点，分别找出距 x, y 轴最近的点 A, B ，分别对应于 $\sum y_i, \sum x_i$ 最小。找出距离线段最远的点 C ，则 C 也在下凸壳上， C 点满足 $AB \times AC$ 最小，也即

$$(X_B - X_A)Y_C + (Y_A - Y_B)X_C - (X_B - X_A)Y_A - (Y_B - Y_A)X_A$$

最小，后两项均为常数，因此将所以权值改成 $(X_B - X_A)y_i + (Y_B - Y_A)x_i$ ，求同样问题（例如最小生成树，最小权匹配）即可。求出 C 点以后，递归 AC, BC 。

9.21.2 最小环

无向图最小环：每次floyd到 k 时，判断 1 到 $k - 1$ 的每一个 i, j ：

$$\text{ans} = \min\{\text{ans}, d(i, j) + G(i, k) + G(k, j)\}.$$

有向图最小环：做完floyd后， $d(i, i)$ 即为经过 i 的最小环。

9.21.3 度序列的可图性

判断一个度序列是否可转化为简单图，除了一种贪心构造的方法外，下列方法更快速。EG定理：将度序列从大到小排序得到 $\{d_i\}$ ，此序列可转化为简单图当且仅当 $\sum d_i$ 为偶数，且对于任意的 $1 \leq k \leq n - 1$ 满足 $\sum_{i=1}^k d_i \leq k(k - 1) + \sum_{i=k+1}^n \min(k, d_i)$ 。

9.21.4 切比雪夫距离与曼哈顿距离转化

曼哈顿转切比雪夫： $(x + y, x - y)$ ，适用于一些每次只能向四联通的格子走一格的问题。切比雪夫转曼哈顿： $(\frac{x+y}{2}, \frac{x-y}{2})$ ，适用于统计距离。

9.21.5 树链的交

```

1 bool cmp(int a, int b){return dep[a]<dep[b];}
2 path merge(path u, path v){
3     int d[4], c[2];
4     if (!u.x || !v.x) return path(0, 0);
5     d[0]=lca(u.x, v.x); d[1]=lca(u.x, v.y);
6     d[2]=lca(u.y, v.x); d[3]=lca(u.y, v.y);
7     c[0]=lca(u.x, u.y); c[1]=lca(v.x, v.y);
8     sort(d, d+4, cmp); sort(c, c+2, cmp);
9     if (dep[c[0]] <= dep[d[0]] && dep[c[1]] <= dep[d[2]])
10        return path(d[2], d[3]);
11    else return path(0, 0); }

```

9.21.6 带修改MST

维护少量修改的最小生成树，可以缩点缩边使暴力复杂度变低。(银川 21: 求有 16 个‘某两条边中至少选一条’的限制条件的最小生成树)

找出必须边 将修改边标 $-\infty$ ，在MST上的其余边为必须边，以此缩点。

找出无用边 将修改边标 ∞ ，不在MST上的其余边为无用边，删除之。

假设修改边数为 k ，操作后图中最多剩下 $k+1$ 个点和 $2k$ 条边。

9.21.7 差分约束

$x_r - x_l \leq c$: add(1, r, c) $x_r - x_l \geq c$: add(r, 1, -c)

9.21.8 李超线段树

添加若干条线段或直线 $(a_i, b_i) \rightarrow (a_j, b_j)$ ，每次求 $[l, r]$ 上最上面的那条线段的值。思想是让线段树中一个节点只对应一条直线，如果在这个区间加入一条直线，如果一段比原来的优，一段比原来的劣，那么判断一下两条线的交点，判断哪条直线可以完全覆盖一段一半的区间，把它保留，另一条直线下传到另一半区间。时间复杂度 $O(n \log n)$ 。

9.21.9 Segment Tree Beats

区间 min, max, 区间求和。以区间取 min 为例，额外维护最大值 m ，严格次大值 s 以及最大值个数 t 。现在假设我们要让区间 $[L, R]$ 对 x 取 min，先在线段树中定位若干个节点，对于每个节点分三种情况讨论：1, 当 $m \leq x$ 时，显然这一次修改不会对这个节点产生影响，直接退出；2, 当 $se < x < ma$ 时，显然这一次修改只会影响到所有最大值，所以把 num 加上 $t \cdot (x - ma)$ ，把 ma 更新为 x ，打上标记退出；3, 当 $se \geq x$ 时，无法直接更新着一个节点的信息，对当前节点的左儿子和右儿子递归处理。单次操作均摊复杂度 $O(\log^2 n)$ 。

9.21.10 二分图

最小点覆盖=最大匹配数。独立集与覆盖集互补。最小点覆盖构造方法：对二分图流图求割集，跨过的边指示最小点覆盖。Hall定理 $G = (X, Y, E)$, $|M| = |X| \Leftrightarrow \forall S \subseteq X, |S| \leq |A(S)|$ 。

9.21.11 稳定婚姻问题

男士按自己喜欢程度从高到底依次向每位女士求婚，女士遇到更喜欢男士时就接受他，并抛弃以前的配偶。被抛弃的男士继续按照列表向剩下的女士依次求婚，直到所有人都有配偶。算法一定能得到一个匹配，而且这个匹配一定是稳定的。时间复杂度 $O(n^2)$ 。

9.21.12 三元环

对于无向边 (u, v) ，如果 $\deg_u < \deg_v$ ，那么连有向边 (u, v) (以点标号为第二关键字)。枚举 x 暴力即可。时间复杂度 $O(m\sqrt{m})$ 。

9.21.13 图同构

令 $F_t(i) = (F_{t-1}(i) * A + \sum_{j \rightarrow i} F_{t-1}(j) * B + \sum_{j \rightarrow i} F_{t-1}(j) * C + D * (i - a)) \bmod P$ ，枚举点 a ，迭代 K 次后求得的就是 a 点所对应的 $hash$ 值，其中 K, A, B, C, D, P 为 $hash$ 参数，可自选。

9.21.14 竞赛图 Landau's Theorem

n 个点竞赛图点按出度按升序排序，前 i 个点的出度之和不小于 $\frac{i(i-1)}{2}$ ，度数总和等于 $\frac{n(n-1)}{2}$ 。否则可以用优先队列构造出方案。

9.21.15 Ramsey Theorem $R(3,3)=6, R(4,4)=18$

6 个人中存在 3 人相互认识或者相互不认识。

9.21.16 树的计数 Prufer序列

树和其prufer编码一一对应，一颗 n 个点的树，其prufer编码长度为 $n-2$ ，且度数为 d_i 的点在prufer 编码中出现 d_i-1 次。

由树得到序列：总共需要 $n-2$ 步，第 i 步在当前的树中寻找具有最小标号的叶子节点，将与其相连的点的标号设为Prufer序列的第 i 个元素 p_i ，并将此叶子节点从树中删除，直到最后得到一个长度为 $n-2$ 的Prufer 序列和一个只有两个节点的树。

由序列得到树：先将所有点的度赋初值为 1，然后加上它的编号在Prufer序列中出现的次数，得到每个点的度；执行 $n-2$ 步，第 i 步选取具有最小标号的度为 1 的点 u 与 $v = p_i$ 相连，得到树中的一条边，并将 u 和 v 的度减一。最后再把剩下的两个度为 1 的点连边，加入到树中。

相关结论： n 个点完全图，每个点度数依次为 $d_1, d_2, \dots, d_n$ ，这样生成树的棵数为： $\frac{(n-2)!}{(d_1-1)!(d_2-1)!\dots(d_n-1)!}$ 。

左边有 n_1 个点，右边有 n_2 个点的完全二分图的生成树棵数为 $n_1^{n_2-1} \times n_2^{n_1-1}$ 。

m 个连通块，每个连通块有 c_i 个点，把他们全部连通的生成树方案数： $(\sum c_i)^{m-2} \prod c_i$

9.21.17 有根树计数 1,1,2,4,9,20,48,115,286,719,1842,4766

无标号 $a_{n+1} = 1/n \sum_{k=1}^n (\sum_{d|k} d \cdot a(d)) \cdot a(n-k+1)$

9.21.18 无根树计数

n 是奇数时，有 $a_n - \sum_i^{n/2} a_i a_{n-i}$ 种不同的无根树。

n 是偶数时，有 $a_n - \sum_i^{n/2} a_i a_{n-i} + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$ 种不同的无根树。

9.21.19 生成树计数 Kirchhoff's Matrix-Tree Theorem

Kirchhoff Matrix $T = Deg - A$, Deg 是度数对角阵, A 是邻接矩阵。无向图度数矩阵是每个点度数；有向图度数矩阵是每个点入度。

邻接矩阵 $A[u][v]$ 表示 $u \rightarrow v$ 边个数，重边按照边数计算，自环不计入度数。

无向图生成树计数: $c = |K|$ 的任意 1 个 $n-1$ 阶主子式

有向图外向树计数: $c =$ 去掉根所在的那阶得到的主子式

9.21.20 有向图欧拉回路计数 BEST Theorem

$$ec(G) = t_w(G) \prod_{v \in V} (\deg(v) - 1)!$$

其中 $\deg$ 为入度 (欧拉图中等于出度), $t_w(G)$ 为以 w 为根的外向树的个数。相关计算参考生成树计数。

欧拉连通图中任意两点外向树个数相同: $t_v(G) = t_w(G)$ 。

9.21.21 Tutte Matrix

Tutte matrix A of a graph $G = (V, E)$:

$$A_{ij} = \begin{cases} x_{ij} & \text{if } (i, j) \in E \text{ and } i < j \\ -x_{ij} & \text{if } (i, j) \in E \text{ and } i > j \\ 0 & \text{otherwise} \end{cases}$$

where x_{ij} are indeterminates. The determinant of this skew-symmetric matrix is then a polynomial (in the variables $x_{ij}, i < j$): this coincides with the square of the pfaffian of the matrix A and is non-zero (as a polynomial) if and only if a perfect matching exists.

9.21.22 Edmonds Matrix

Edmonds matrix A of a balanced ($|U| = |V|$) bipartite graph $G = (U, V, E)$:

$$A_{ij} = \begin{cases} x_{ij} & (u_i, v_j) \in E \\ 0 & (u_i, v_j) \notin E \end{cases}$$

where the x_{ij} are indeterminates. G 有完美匹配当且仅当关于 x_{ij} 的多项式 $\det(A_{ij})$ 不为 0。完美匹配的个数等于多项式中单项式的个数。

9.21.23 有向图无环定向, 色多项式

图的色多项式 $P_G(q)$ 对图 G 的 q -染色计数。

Triangle K_3 : $x(x-1)(x-2)$

Complete graph K_n : $x(x-1)(x-2) \cdots (x-(n-1))$

Tree with n vertices: $x(x-1)^{n-1}$

Cycle C_n : $(x-1)^n + (-1)^n (x-1)$

acyclic orientations of an n -vertex graph G is $(-1)^n P_G(-1)$ 。

9.21.24 拟阵交问题

拟阵定义: S, T 是独立集, 则 S 子集是, 若 $|S| > |T|$, 则 S 能扩充 T 。最大带权拟阵交问题: 全集 U 中每个元素都有权值 w_i 。设同一个全集 U 上有两个满足拟阵性质的集族 $\mathcal{F}_1, \mathcal{F}_2$ 。对于 $k = 1..|U|$, 分别求出一个集合 S , 满足 $S \in \mathcal{F}_1 \cap \mathcal{F}_2$ 且 $|S|$ 恰好为 k 的前提下, S 中元素权值和最小。

设集合大小为 k 时已经求出了答案 S 。现在希望求出集合大小为 $k+1$ 的答案。 U 中所有元素分为两个集合: 当前答案集合 S , 和剩余集合 $T = U \setminus S$ 。考虑 T 中的某个元素 x_i 。记 $A = \{x_i | S \cup \{x_i\} \in \mathcal{F}_1\}$, $B = \{x_i | S \cup \{x_i\} \in \mathcal{F}_2\}$ 。如果 T 中某个元素 $x_i \notin A$, 说明 x_i 加进 S 中形成了某个“环”, 从而不满足 $\mathcal{F}_1$ 的限制。考虑这个“环”上每个元素 y_j , 满足 $S \setminus \{y_j\} \cup \{x_i\} \in \mathcal{F}_1$, 将 y_j 向每个 x_i 连边。如果 T 中某个元素 $x_i \notin B$, 同理找出 S 中每一个元素 y_j 使得 $S \setminus \{y_j\} \cup \{x_i\} \in \mathcal{F}_2$, 将 x_i 向 y_j 连边。现在求出从 A 到 B 的多源多汇最短路, 权值在点上, 若点属于 T 则权值为正, 否则属于 S , 权值为负。最短路上的每个 T 中的点放进 S , S 中的点放进 T , 则完成了一次增广。由于每次增广路的起点和终点都在 T 中, 所以每次增广都会使得 $|S|$ 增加 1。

最大拟阵交问题可以去掉权值直接求增广路。

9.21.25 双极定向

```

1 //无向图每条边定向后成为DAG, 使得s可达所有点可达t
2 //topo为定向后DAG的拓扑序, u->v: 拓扑序中u在v的前面.
3 int n, dfn[N], low[N], stp, p[N], ord[N], topo[N];
4 bool fail = 0, sign[N]; vector<int> e[N];
5 void dfs(int x, int fa, int s, int t){
6     dfn[x] = low[x] = ++stp;
7     ord[stp] = x, p[x] = fa;
8     if (x == s) dfs(t, x, s, t);
9     for (int y : e[x]){
10        if (x == s && y == t) continue;
11        if (!dfn[y]){
12            if (x == s) fail = true;
13            dfs(y, x, s, t);
14            low[x] = min(low[x], low[y]); }
15        else if (dfn[y] < dfn[x] && y != fa)
16            low[x] = min(low[x], dfn[y]); } }
17 bool bipolar_orientation(int s, int t){

```



```

18 | e[s].push_back(t), e[t].push_back(s);
19 | stp = fail = 0, dfs(s, s, s, t);
20 | for (int i = 1; i <= n; i++)
21 | | if (i != s && (!dfn[i] || low[i] >= dfn[i]))
22 | | | fail = true;
23 | if (fail) return false;
24 | sign[s] = 0; //memset sign[] is not necessary
25 | int pre[n + 5], suf[n + 5]; // list
26 | suf[0] = s; pre[s] = 0, suf[s] = t;
27 | pre[t] = s, suf[t] = n + 1; pre[n + 1] = t;
28 | for (int i = 3; i <= n; i++){
29 | | int v = ord[i];
30 | | if (!sign[ord[low[v]]]){ // insert before p[v]
31 | | | int P = pre[p[v]];
32 | | | pre[v] = P, suf[v] = p[v];
33 | | | suf[P] = pre[p[v]] = v; }
34 | | else{ // insert after p[v]
35 | | | int S = suf[p[v]];
36 | | | pre[v] = p[v], suf[v] = S;
37 | | | suf[p[v]] = pre[S] = v; }
38 | | sign[p[v]] = !sign[ord[low[v]]]; }
39 | for (int x = s, cnt = 0; x != n + 1; x = suf[x])
40 | | topo[++cnt] = x;
41 | return true; }

```

```

19 | for i in combinations_with_replacement('ABCD', repeat=2):
20 | | pass # AA AB AC AD BB BC BD CC CD DD
21 | def FractionOperation():
22 | | from fractions import Fraction
23 | | a.numerator, a.denominator, str(a)
24 | | a = Fraction(0.233).limit_denominator(1000)
25 | def DecimalOperation():
26 | | from decimal import Decimal, getcontext, FloatOperation
27 | | getcontext().prec = 100
28 | | getcontext().rounding = getattr(decimal, 'ROUND_HALF_EVEN')
29 | | # default; other: FLOOR, CEILING, DOWN, ...
30 | | getcontext().traps[FloatOperation] = True
31 | | Decimal((0, (1, 4, 1, 4), -3)) # 1.414
32 | | a = Decimal(1<<31) / Decimal(100000)
33 | | print(f"{a:.9f}") # 21474.83648
34 | | print(a.sqrt(), a.ln(), a.log10(), a.exp(), a ** 2)
35 | def Complex():
36 | | a = 1-2j
37 | | print(a.real, a.imag, a.conjugate())
38 | def FastIO():
39 | | import sys, atexit, io
40 | | _INPUT_LINES = sys.stdin.read().splitlines()
41 | | input = iter(_INPUT_LINES).__next__
42 | | _OUTPUT_BUFFER = io.StringIO()
43 | | sys.stdout = _OUTPUT_BUFFER
44 | | atexit.register
45 | | def write():
46 | | | sys.__stdout__.write(_OUTPUT_BUFFER.getvalue())

```

9.21.26 图中的环

没有奇环的图是二分图, 没有偶环的图是仙人掌. 判定没有奇环仅用深度奇偶性判即可; 判定没有偶环的图需要记录覆盖次数判定是否存在奇环有交.

10. 离线算法 (如 CDQ、莫队)

10.1 莫队二次离线

```

1 | for(int i=1;i<=n;i++){sum[i] = query(arr[i]) + sum[i-1];
  |   ↳ add(i);}
2 | sort(Q+1, Q+1+m, cmp);
3 | int l = 1, r = 0;
4 | for(int i=1;i<=m;i++){
5 | | int ql = Q[i].l, qr = Q[i].r;
6 | | if(l > ql) v[r].push_back({ql, l-1, i}), Q[i].ans -=
  |   ↳ sum[l-1] - sum[ql-1], l = ql;
7 | | if(r < qr) v[l-1].push_back({r+1, qr, -i}), Q[i].ans +=
  |   ↳ sum[qr] - sum[r], r = qr;
8 | | if(l < ql) v[r].push_back({l, ql-1, -i}), Q[i].ans +=
  |   ↳ sum[ql-1] - sum[l-1], l = ql;
9 | | if(r > qr) v[l-1].push_back({qr+1, r, i}), Q[i].ans -=
  |   ↳ sum[r] - sum[qr], r = qr;
10 | }
11 | // 清空贡献
12 | for(int i=1;i<=n;i++){
13 | | add(i);
14 | | for(auto& [l, r, id] : v[i]){
15 | | | ll tmp = 0;
16 | | | for(int j=l;j<=r;j++){
17 | | | | tmp += query(arr[j]);
18 | | | | // 注意如果j <= i且会产生重复贡献的时候要扣去
19 | | | }
20 | | | if(id > 0) Q[id].ans += tmp;
21 | | | else Q[-id].ans -= tmp;
22 | | }
23 | }
24 | for(int i=1;i<=m;i++) Q[i].ans += Q[i-1].ans;
25 | for(int i=1;i<=m;i++) ans[Q[i].id] = Q[i].ans;

```

11.2 Hacks: 指令集, 读入优化, Bitset, builtin, 随机种子

```

1 | //fast = O3 + ffast-math + fallow-store-data-races
2 | #pragma GCC optimize("Ofast")
3 | //下: 加include后面| 位运算 |浮点| SIMD | 优化寄存器|
4 | #pragma GCC target("abm,bmi,bmi2,fma,sse2,avx2,tune=native")
5 | const int SZ = 1 << 16; int getc() {
6 | | static char buf[SZ], *ptr = buf, *top = buf;
7 | | if (ptr == top) {
8 | | | ptr = buf, top = buf + fread(buf, 1, SZ, stdin);
9 | | | if (top == buf) return -1; }
10 | | return *ptr++; }
11 | idx=b._Find_first();idx!=b.size();idx=b._Find_next(idx);
12 | struct HashFunc{size_t operator()(const KEY &key)const{};
13 | |__builtin_uaddll_overflow(a, b, &c) // binary big int
14 | void GspersHack(int k, int n) {
15 | | for (int s = (1 << k) - 1, c, r; s < (1 << n);
16 | | | c = s & -s, r = s + c, s = ((r ^ s) >> 2) / c) | r); }
17 | chrono::steady_clock::now().time_since_epoch().count());

```

11.3 试机赛与纪律文件

- 一人场检查—检查身份证件: 胸牌。
- 检查: 手机、智能手表、金属 (钥匙) 等等。
- 完整测试键盘、鼠标、显示器和座椅。热身赛发现联系工作人员。
- 一环境检查—测试所有可能的提交方式, 找到所有题面、排行榜位置。
- 测试 -fsanitize address, undefined, define _GLIBCXX_DEBUG
- 测试 time 命令是否能显示内存占用。/usr/bin/time -v ./a.out
- 一OJ检查—测试编译器版本。C++20 cin >> (s + 1); C++17 auto [x, y]: a; C++14 [(auto x, auto y); C++11 auto; bits/stdc++.h; pb_ds。

```

#include <ext/rope>
using namespace __gnu_cxx;
rope <int> R; R.insert(y, x); R[x]; R.erase(x, 1);
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
tree <int, null_type, less<int>, rb_tree_tag,
tree_order_statistics_node_update> s;
s.insert(1); s.find_by_order(0); s.order_of_key(5);

```

- __int128, __float128, long double。
- pragma target CE; 代码长度限制 (256K), 尝试 NO-OUTPUT, OLE, MLE。
- for i in 1..11; do pc2submit a.cpp -y; done 有没有提交限制。
- 默认一分钟提交 10 次; 优先队列, 上一发没有测完, 下一发优先级 -1。
- clock() 是否能够正常工作; 测试本地性能与提交性能。

```

const int N = 1 << 20;
for (int T = 4; T; T--) {
    int a[N], b[N], c[N];
    for (int i = 0; i < N; i++) a[i] = i, b[i] = N - i;
    ntt_init(N); ntt(a, N, 1); ntt(b, N, 1);
    for (int i = 0; i < N; i++) c[i] = a[i] * b[i];
    ntt(c, N, -1); assert (c[0] == 0xad223d); }
// 机房: 190ms; (1s T = 21)
// Q03: 340+-40ms; (1s T = 11.9)
// CF: 1050+-100ms; (1s T = 3.9)

import sys
sys.set_int_max_str_digits(0) # >= Python 3.11
T = 3
for _ in range(T): # 本地 assert failed 检查下数字抄错没
    assert str(3 ** 100000 * 10 ** 100000)[9] == "133497141"
# 机房 Pypy: 130ms; (1s T = 39)
# Q03: 201ms; (1s T = 16)
# CF Python3: 1100ms; (1s T = 2.9) Pypy3: 310ms; (1s T = 12)

```

11. 随机化与博弈论

11.1 Python Hint

```

1 | from itertools import combinations, combinations_with_replacement,
  |   ↳ permutations, product
2 |
3 |
4 | def RandomAndList():
5 | | import random
6 | | random.normalvariate(0.5, 0.1)
7 | | l = [str(i) for i in range(9)]
8 | | sorted(l, min(1), max(1), len(1)
9 | | random.shuffle(l)
10 | | l.sort(key=lambda x: x ^ 1, reverse=True)
11 | | from functools import cmp_to_key
12 | | l.sort(key=cmp_to_key(lambda x, y: (y^1)-(x^1)))
13 | | for i in product('ABCD', repeat=2):
14 | | | pass # AA AB AC AD BA BB BC BD CA CB CC CD DA DB DC DD
15 | | for i in permutations('ABCD', repeat=2):
16 | | | pass # AB AC AD BA BC BD CA CB CD DA DB DC
17 | | for i in combinations('ABCD', repeat=2):
18 | | | pass # AB AC AD BC BD CD

```

1. **一提交前检查**— 保存, 编译, 测过样例, 测过 $n=0/1$;
2. 数组大小: N/M ; 时间空间限制; 输入实数? MOD 多少? 开 LL
3. **调用 `init`**, 清空时数组大小 (0, 1, n 还是 $m, n+1$)
4. 输出取模取到正值, 输出格式
5. 多组没读入完就 `break`
6. 关流同步, 注意不要关流同步混用
7. 加上需要怀疑的 `assert`
8. 开 `Ofast`, 位运算开 `target("abm")`

n	$\log_{10} n$	$n!$	$C(n, n/2)$	$\text{LCM}(1 \dots n)$	P_n
2	0.30102999	2	2	2	2
3	0.47712125	6	3	6	3
4	0.60205999	24	6	12	5
5	0.69897000	120	10	60	7
6	0.77815125	720	20	60	11
7	0.84509804	5040	35	420	15
8	0.90308998	40320	70	840	22
9	0.95424251	362880	126	2520	30
10	1	3628800	252	2520	42
11	1.04139269	39916800	462	27720	56
12	1.07918125	479001600	924	27720	77
15	1.17609126	1.31e12	6435	360360	176
20	1.30103000	2.43e18	184756	232792560	627
25	1.39794001	1.55e25	5200300	26771144400	1958
30	1.47712125	2.65e32	155117520	1.444e14	5604
P_n	37338 ₄₀	204226 ₅₀	966467 ₆₀	190569292 ₁₀₀	1e9 ₁₁₄

$n \leq$	10	100	1e3	1e4	1e5	1e6
$\max \omega(n)$	2	3	4	5	6	7
$\max d(n)$	4	12	32	64	128	240
$\arg \max d$	6	60	840	7560	83160	720720
$\pi(n)$	4	25	168	1229	9592	78498
$n \leq$	1e7	1e8	1e9	1e10	1e11	1e12
$\max \omega(n)$	8	8	9	10	10	11
$\max d(n)$	448	768	1344	2304	4032	6720
$\arg \max d$	8648640	73513440	$\times 10$	$\times 9.5$	$\times 14$	...
$\pi(n)$	664579	5761455	5.08e7	4.55e8	4.12e9	3.7e10
$n \leq$	1e13	1e14	1e15	1e16	1e17	1e18
$\max \omega(n)$	12	12	13	13	14	15
$\max d(n)$	10752	17280	26880	41472	64512	103680
$\arg \max d$	1e18: 897612484786617600 = $2^8 3^4 5^2 7^2 11 \dots 37$					
$\pi(n)$	Prime number theorem: $\pi(x) \sim x/\log(x)$					

1. **一提交后检查**— 变量复用后忘记改回去
2. `sum` 多组输入: 应只用了输入内容 (`memset TL`, 错下标 `WA`), 用前清空还是用后
3. `int/LL` 溢出, `INF/-1` 大小, 浮点数 `eps` 和 `= 0`, `__builtin_popcountll`
4. 类似 `pair<LL, int> x = pair<int, int>()`; 不会报警告
5. 离散化与二分: `lower_bound upper_bound +-1, begin(), end()`
6. 自定义排序: 排序方向, 比较函数为小于, 考虑坏点: 如叉积 (0, 0)
7. 样例是否为对称/回文的? 考虑构造不对称的情况, 正序逆序
8. 复制过的代码对应位置正确修改
9. 检查 `shadow` 变量重名
10. 分类讨论: `if` 嵌套, `break` 位置
11. 图: 边标号初始是否为 1, 单双向边, 反向边边权
12. 几何: 共线, `sqrt(-0.0)`, `nan / inf`, 重点, 除零 `det/dot`, 旋转方向
13. 浮点数: `eps` 量纲: 应当至少取到 (最短线段 $\times$ 精度要求)。
14. 二分, 三分: 次数。

```
1 export CXXFLAGS='-g -Wall -Wextra -Wconversion -Wshadow
   ↪ -std=gnu++20'
2 ulimit -s 1048576
```